МАТЕРИАЛЫ ДЛЯ ГОСУДАРСТВЕННОЙ РЕГИСТРАЦИИ ПРОГРАММЫ ДЛЯ ЭВМ
АРМ
МОДУЛЬ «СПРАВОЧНИКИ»
ОПИСАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ, ФРАГМЕНТЫ ИСХОДНОГО ТЕКСТА И ЭКРАННЫЕ ФОРМЫ

Адрес модуля: http://msk-arm-gs01.ntcees.ru/refdata/

Дата подготовки материалов: 29 октября 2026 года

Состав: главная страница, 25 страниц списков, 25 форм добавления, служебные операции и взаимодействия с другими модулями.

Документ подготовлен как отдельный файл, предусмотренный пунктом «Исходный текст программы (листинг)» шаблона «Сведения для регистрации программы». Приведены фрагменты, а не полный исходный текст программы.

Основание описания: текущие исходные файлы рабочего проекта и экранные формы действующего сайта. Номер релиза серверного приложения отдельно не устанавливался: совпадение серверной сборки со всеми локальными исходными файлами не утверждается.

СОДЕРЖАНИЕ

1. Сведения по форме предоставленного шаблона 3
2. Назначение и архитектура модуля 6
2.1. Слои реализации 6
2.2. Версии БД и межверсионная идентичность 7
2.3. Общие правила обработки и ограничения 8
3. Главная страница и периоды планирования 9
3.1. Модели и маршрут главной страницы 10
3.2. Изменение текущего года 11
3.3. Служебные операции периодов и взаимодействия 13
4. Страницы справочников и формы добавления 14
4.1. Годы 14
4.2. Федеральные округа 19
4.3. Субъекты РФ 24
4.4. Зарубежные страны, имеющие общие границы с РФ 30
4.5. Объединенные энергосистемы 34
4.6. Региональные энергосистемы 39
4.7. Энергорайоны 44
4.8. Энергоузлы 49
4.9. Типы энергосистем 53
4.10. Синхронные зоны 57
4.11. Энергозоны 62
4.12. Генерирующие компании 66
4.13. Типы топлива 70
4.14. Виды топлива 74
4.15. Типы электростанций 81
4.16. Типы состояния оборудования 85
4.17. Типы агрегатов 89
4.18. Типы ТЭС 93
4.19. Типы агрегатов ТЭС 97
4.20. Типы агрегатов ПГУ 101
4.21. Типы групп оборудования 105
4.22. Типы технологий 111
4.23. Доступность технологии 115
4.24. Типы затрат на производство ЭЭ и ТЭ 119
4.25. Виды экономической деятельности (ВЭД) 123
5. Взаимодействие с другими модулями 128
5.1. Пример сквозного взаимодействия: субъект РФ → энергозона → потребление 129
5.2. Пример сквозного взаимодействия: тип группы → технология → оборудование 129
5.3. Граница модуля: управление версиями БД 130
6. Полнота материалов и перечень страниц 131
6.1. Сведения об источниках и проверке 132

1. Сведения по форме предоставленного шаблона

№	Сведения по шаблону	Содержание для модуля «Справочники»
1	Название программы (полное и сокращенное)	Программа: АРМ (обозначение, предоставленное заявителем). Полное регистрационное наименование подлежит уточнению. Описываемая часть: модуль «Справочники» (refdata).
2	Назначение, область применения, функциональные возможности (в реферативной форме); ~ 10 строк.	Модуль «Справочники» программы АРМ предназначен для централизованного ведения нормативно-справочной информации, используемой при планировании развития электроэнергетики. Обеспечивает ведение административных территорий, энергосистем и энергозон, компаний, топлива, типов станций, агрегатов, групп оборудования, технологий, видов затрат и ВЭД. Реализует поиск, фильтрацию, сортировку, постраничный просмотр, табличное редактирование, добавление и удаление записей. Для предусмотренных маршрутов поддерживаются импорт и экспорт Excel. Управляет текущим годом, признаками годов и периодами планирования. Работает в контексте выбранной версии БД; для поддерживаемых справочников предусмотрено изменение во всех версиях по ref_uuid. Предоставляет единые классификаторы модулям генерации, топлива, экономики, электропотребления, потребности в мощности, энергетических балансов, экологии и рациональной структуры. Доступ осуществляется через веб-интерфейс после аутентификации.
3	Тип ЭВМ	Клиент-серверная веб-система. Сервер и клиентские персональные компьютеры; работа пользователя через веб-браузер. Точная аппаратная конфигурация заявляемого экземпляра подлежит уточнению.
4	Язык программирования	Python; JavaScript; шаблоны HTML/Jinja2 и CSS. Доступ к реляционным данным через SQLAlchemy (СУБД PostgreSQL).
5	Тип и версия используемой ОС	В документации проекта описано серверное развертывание на Astra Linux 1.8.4. Текущий рабочий каталог находится в среде Windows. ОС и версия заявляемого экземпляра подтверждаются заявителем.
6	Объем программы для ЭВМ в машиночитаемой форме в единицах, кратных числу байт	Объем всей регистрируемой программы и состав поставки указываются заявителем. Размер выборки исходных файлов этого описания не является объемом программы.
7	Дата окончательного завершения программы	Дата окончательного завершения программы указывается заявителем. 29.10.2026 – дата

№	Сведения по шаблону	Содержание для модуля «Справочники»
	(число, месяц, год)	подготовки описания, не подтверждение завершения разработки.
8	ФИО каждого автора; дата рождения; адрес местожительства (включая индекс)	Заполняется заявителем: сведения не предоставлены.
9	В каком порядке указывать ФИО авторов?	Заполняется заявителем: сведения не предоставлены.
10	Краткое описание авторского вклада в данную программу (Написание исходного текста программного модуля …; или Написание исходного текста подпрограммы …; или Разработка программ …; или Написание исходного текста программы – 00%)	Авторский вклад и доли определяются заявителем. Документ описывает реализацию модуля, но не устанавливает авторство конкретных лиц.
11	Исходный текст программы (листинг) (можно не подряд, а фрагментами). Допускается включать в состав этих мат-лов подготовительные материалы, полученные в ходе ее разработки, а также порождаемые ею аудиовизуальные отображения в любой визуально воспринимаемой форме. Экранные формы включаются в состав документов заявки в том случае, если они охраноспособны по нормам авторского права, т.е. являются результатом творческой деятельности автора.	Фрагменты исходного текста приведены в разделах 2–6, с указанием файлов и строк; экранные формы включены по месту описания. Представлены фрагменты, не полный текст.
12	Контактное лицо из числа авторов, тел.	Заполняется заявителем: сведения не предоставлены.

Сведения об авторах, их персональных данных, контактах, окончательном названии, объеме и дате завершения не определяются по имени пользователя в интерфейсе или по дате изменения файлов. Вопрос о включении экранных форм в состав охраняемых материалов решается заявителем с учетом прав на используемые элементы интерфейса.

2. Назначение и архитектура модуля

Модуль «Справочники» программы АРМ предназначен для централизованного ведения нормативно-справочной информации, используемой при планировании развития электроэнергетики. Обеспечивает ведение административных территорий, энергосистем и энергозон, компаний, топлива, типов станций, агрегатов, групп оборудования, технологий, видов затрат и ВЭД. Реализует поиск, фильтрацию, сортировку, постраничный просмотр, табличное редактирование, добавление и удаление записей. Для предусмотренных маршрутов поддерживаются импорт и экспорт Excel. Управляет текущим годом, признаками годов и периодами планирования. Работает в контексте выбранной версии БД; для поддерживаемых справочников предусмотрено изменение во всех версиях по ref_uuid. Предоставляет единые классификаторы модулям генерации, топлива, экономики, электропотребления, потребности в мощности, энергетических балансов, экологии и рациональной структуры. Доступ осуществляется через веб-интерфейс после аутентификации.

2.1. Слои реализации

Слой	Реализация и роль
Модели	SQLAlchemy-модели app/refdata/models: таблицы классификаторов, внешние ключи, ограничения уникальности, привязка к версии БД и устойчивый ref_uuid.
Маршруты	Flask Blueprint refdata_bp; обработка GET/POST, получение параметров формы, вызов сервисов, flash-сообщения, перенаправление или render_template.
Сервисы	app/refdata/services: выборка, фильтры, валидация, изменение данных, импорт/экспорт и синхронизация между версиями.
Формы	WTForms-классы app/refdata/forms: поля, подписи и валидаторы форм фильтрации и добавления.
Представление	Jinja2/HTML app/templates/refdata, общий base.html, клиентские скрипты для таблиц, поиска и иерархии топлива.
Общие подсистемы	Версии БД, аутентификация и права, журналирование, кэш вариантов выбора, транзакции и разрешение внешних ключей.

Основной поток просмотра: браузер → GET-маршрут → сервис выборки с контекстом версии → SQLAlchemy/БД → пагинация и варианты выбора → шаблон Jinja2 → HTML. Поток изменения: POST → чтение и проверка значений → сервис изменения → проверка зависимостей и сохранение → сообщение → перенаправление на список.

Регистрация Blueprint и маршрутов. Источник: app/refdata/routes/__init__.py, строки 1–18.

```
from flask import Blueprint

refdata_bp = Blueprint('refdata_bp', __name__)

from .refdata_routes import *
from .territories.federal_district_routes import *
from .territories.regional_district_routes import *
from .territories.foreign_border_country_routes import *
from .energy_systems.union_energy_system_routes import *
from .energy_systems.regional_energy_system_routes import *
from .energy_systems.energy_area_routes import *
from .energy_systems.energy_zone_routes import *
from .energy_systems.energy_system_type_routes import *
from .energy_systems.synchronous_area_routes import *
from .energy_systems.energy_unit_routes import *
from .gen_companies.gen_company_routes import *
from .fuels.fuel_routes import *
from .fuels.fuel_type_routes import *
```

2.2. Версии БД и межверсионная идентичность

Записи классификаторов имеют database_version_id. Первичный ключ id идентифицирует строку. У справочников с RefdataUuidMixin поле ref_uuid позволяет сопоставить логически одну и ту же запись в разных версиях. Числовой id связанной записи нельзя безусловно копировать в другую версию: для межверсионной операции используется разрешение внешнего ключа по ref_uuid.

Разрешение связей между версиями. Источник: app/refdata/services/refdata_all_versions_common.py, строки 43–64.

```
def fk_id_for_version(model_cls, source_pk_id: int | None, target_version_id: int | None) -> int |
None:
    """
    Находит id строки с тем же ref_uuid в указанной версии БД,
    по исходному pk записи из текущей (или любой) версии.

    При активном fk_id_resolution_cache() один раз подгружает все версии
    по ref_uuid и дальше отвечает из памяти.
    """
    if source_pk_id is None:
        return None
    if target_version_id is None:
        raise ValueError(
            f"Нельзя сопоставить {model_cls.__name__} с версией БД id=None: "
            "записи без версии не участвуют в синхронизации «во всех версиях»."
        )

    source_pk = int(source_pk_id)
    target_vid = int(target_version_id)
    cache = _fk_id_cache.get()
    cache_key = (model_cls.__name__, source_pk, target_vid)
    if cache is not None and cache_key in cache:
        return cache[cache_key]
```

Операции с признаком all_versions=1 вызываются только там, где они реализованы в маршруте и сервисе. Для справочника годов такой режим не предусмотрен. Проверки has_admin и is_admin различаются: конкретный маршрут может требовать общие административные полномочия либо именно роль admin. Документ не приравнивает эти признаки.

Серверная проверка межверсионной операции. Источник: app/refdata/routes/refdata_all_versions_guard.py, строки 11–23.

```
  11  def block_all_versions_without_admin(current_user) -> bool:
  12      """
  13      True – запрос следует заблокировать (нужен redirect и flash).
  14      """
  15      if not all_versions_requested():
  16          return False
  17      if getattr(current_user, "has_admin", False):
  18          return False
  19      flash(
  20          "Операции во всех версиях базы данных доступны только пользователям с ролью
администратора.",
  21          "danger",
  22      )
  23      return True
```

2.3. Общие правила обработки и ограничения

Большинство списков совмещает просмотр и изменение: GET получает фильтры, сортировку и номер страницы, POST передает массивы идентификаторов и значений строк. Удаление обычно задается флажками и обрабатывается при отправке формы. Фактический результат зависит от проверок сервиса, ограничений БД и наличия зависимых записей; возможность безусловного удаления не заявляется.

Наличие login_required означает требование входа в систему, но не доказывает отдельного административного ограничения для каждой операции. Для годов предусмотрен has_admin_required; изменение текущего года требует roles_required(["admin"]). Видимость кнопки в шаблоне и серверная проверка прав описываются раздельно.

Сохранение выполняется транзакционными средствами проекта. В сервисах применяются commit/rollback и общий _commit_with_retry; отдельные операции используют дополнительную обработку конфликтов и ограничений целостности. Изменения и ошибки регистрируются через log_to_db. Журналирование не является отдельной пользовательской страницей справочника.

3. Главная страница и периоды планирования

Адрес: GET /refdata/. Страница предоставляет навигацию по группам справочников и показывает текущий год, период СиПР и горизонт Генеральной схемы. Текущий год выбирается из справочных годов выбранной версии. При отсутствии выбранной версии маршрут передает нулевые значения и пустой список вариантов выбора.

Для текущего года 2025 на действующем сайте показаны СиПР 2026–2031 и Генеральная схема 2026–2042. Это значения конкретного состояния данных, а не постоянные параметры программы.



Рисунок 1. Главная страница модуля «Справочники». Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

3.1. Модели и маршрут главной страницы

Используются Year (календарный год и его признак), YearFeature (факт, текущий/оценка, план), YearService (границы СиПР и даты), DatabaseVersion (контекст выбранной версии).

Модель параметров периода. Источник: app/refdata/models/years/year_service_model.py, строки 13–34.

```
13  class YearService(db.Model, AuditMixin, VersionedModelMixin):
14      __tablename__ = 'gs_sys_year_service'
15      __table_args__ = {
16          "schema": SCHEMA_REFDATA,
17          "extend_existing": True
18      }
19
20      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
21
22      # Годы СиПР
23      year_sipr_start = db.Column(db.Integer, nullable=True)  # Год начала СиПР
24      year_sipr_end = db.Column(db.Integer, nullable=True)    # Год конца СиПР
25
26      # Даты СиПР
27      date_sipr_start = db.Column(db.Date, nullable=True)  # Дата начала СиПР
28      date_sipr_end = db.Column(db.Date, nullable=True)    # Дата конца СиПР
29
30      # Аудит
31      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
32      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
33
34      # FK -> DatabaseVersion
```

Маршрут главной страницы. Источник: app/refdata/routes/refdata_routes.py, строки 19–44.

```
19  @refdata_bp.route("/")
20  @login_required
21  def refdata():
22      """Страница справочников, в т.ч. блок управления годами/СиПР."""
23      version_id = get_current_version()
24
25      if version_id:
26          year_info = get_current_year_info(version_id)
27      else:
28          # Если версия не выбрана – по ТЗ показываем нули по годам
29          year_info = {
30              "current_year": 0,
31              "sipr_start": 0,
32              "sipr_end": 0,
33              "date_sipr_start": None,
34              "date_sipr_end": None,
35          }
36
37      year_select_years = get_reference_years() if version_id else []
38
39      return render_template(
40          "refdata/refdata.html",
41          year_info=year_info,
42          year_select_years=year_select_years,
43          general_scheme_end_year=get_general_scheme_end_year(year_info["current_year"]),
44      )
```

3.2. Изменение текущего года

POST /refdata/update_current_year проверяет роль admin, выбранную версию и наличие года в get_reference_years(). Сервис update_current_year проставляет признаки: годы меньше выбранного – «факт», равный – «текущий (оценка)», большие – «план». Границы СиПР сохраняются как current_year + 1 и current_year + 6. После сохранения очищаются связанные кэши годов.

Проверки маршрута изменения года. Источник: app/refdata/routes/refdata_routes.py, строки 47–69.

```
47  @refdata_bp.route("/update_current_year", methods=["POST"])
48  @login_required
49  @roles_required(["admin"])
50  def update_current_year_refdata():
51      """Обновление текущего года из формы на странице справочников."""
52      version_id = get_current_version()
53      if not version_id:
54          flash("Текущая версия БД не установлена", "danger")
55          return redirect(url_for("refdata_bp.refdata"))
56
57      try:
58          year = int(request.form.get("current_year", 0))
59      except (TypeError, ValueError):
60          flash("Некорректное значение года", "danger")
61          return redirect(url_for("refdata_bp.refdata"))
62
63      if year not in get_reference_years():
64          flash("Некорректное значение года", "danger")
65          return redirect(url_for("refdata_bp.refdata"))
66
67      success, message = update_current_year(version_id, year, current_user)
68      flash(message, "success" if success else "danger")
69
```

Изменение признаков годов и периода СиПР. Источник: app/refdata/services/year_management_services.py, строки 156–188.

```
156
157         # Правило: меньше текущего -> факт, текущий -> текущий (оценка), больше -> план
158         years = (
159             db.session.query(Year)
160             .filter(Year.database_version_id == version_id)
161             .all()
162         )
163         for y in years:
164             if y.number < new_year:
165                 y.id_year_feature = fact_feature.id
166             elif y.number > new_year:
167                 y.id_year_feature = plan_feature.id
168             else:
169                 y.id_year_feature = current_year_feature.id
170
171         # Автоматически обновляем SIPR годы в таблице YearService
172         year_service = db.session.query(YearService).filter_by(
173             database_version_id=version_id
174         ).first()
175
176         if not year_service:
177             # Создаем новую запись YearService
178             year_service = YearService(
179                 database_version_id=version_id,
180                 year_sipr_start=new_year + 1,
181                 year_sipr_end=new_year + 6
182             )
183             db.session.add(year_service)
184         else:
185             # Обновляем существующую запись
186             year_service.year_sipr_start = new_year + 1
187             year_service.year_sipr_end = new_year + 6
188
```

Горизонт Генеральной схемы определяется по шестилетнему циклу разработки, начиная с 2024 года, на 18 лет. Для current_year < 2024 или отсутствующего года конец не определяется; для 2024–2029 возвращается 2042, для 2030–2035 – 2048. Начальная граница в шаблоне выводится из sipr_start.

Правило горизонта Генеральной схемы. Источник: app/refdata/services/year_management_services.py, строки 19–24.

```
19  def get_general_scheme_end_year(current_year: int | None) -> int | None:
20      """Разработка с 2024 года каждые 6 лет на 18 лет; актуализация не сдвигает конец."""
21      if current_year is None or current_year < 2024:
22          return None
23      cycle = (current_year - 2024) // 6
24      return 2024 + cycle * 6 + 18
```

HTML выбора текущего года. Источник: app/templates/refdata/refdata.html, строки 17–35.

```
  17                            <form method="post" action="{{
url_for('refdata_bp.update_current_year_refdata') }}" class="d-flex align-items-center gap-2 mb-0 flex-
fill">
  18                                <span class="fw-bold text-nowrap">Текущий год:</span>
  19                                <div class="input-group flex-grow-1">
  20                                    <select class="form-select" id="current-year-select"
name="current_year"
  21                                            {% if not current_user.has_admin %}disabled{% endif %}>
  22                                        <option value="0" {% if year_info.current_year == 0
%}selected{% endif %}>
  23                                            не установлен
  24                                        </option>
  25                                        {% for y in year_select_years %}
  26                                            <option value="{{ y }}" {% if year_info.current_year == y
%}selected{% endif %}>
  27                                                {{ y }}
  28                                            </option>
  29                                        {% endfor %}
  30                                    </select>
  31                                    {% if current_user.has_admin %}
  32                                        <button type="submit" class="btn btn-primary">
  33                                            <i class="bi bi-check-lg"></i>
  34                                        </button>
  35                                    {% endif %}
```

HTML периода Генеральной схемы. Источник: app/templates/refdata/refdata.html, строки 46–55.

```
  46                            <div class="d-flex align-items-center gap-2 flex-fill">
  47                                <span class="fw-bold text-nowrap">Генеральная схема:</span>
  48                                <div class="input-group flex-grow-1">
  49                                    <div class="form-control bg-light fw-bold fs-5 text-center">
  50                                        {% if general_scheme_end_year is not none %}
  51                                            {{ year_info.sipr_start }} – {{
general_scheme_end_year }}
  52                                        {% else %}
  53                                            не установлен
  54                                        {% endif %}
  55                                    </div>
```

3.3. Служебные операции периодов и взаимодействия

Метод и адрес	Назначение
POST /refdata/update_sipr_dates	Сохранение дат СиПР через HTML-маршрут; отдельной формы на текущей главной странице нет.
GET /refdata/api/current_year_info	JSON-информация о годах текущей версии; без выбранной версии возвращается набор нулевых значений.
POST /refdata/api/update_current_year	Изменение текущего года через JSON. Маршрут проверяет диапазон 2000–2100, в отличие от HTML-маршрута с проверкой get_reference_years().
POST /refdata/api/update_sipr_years	Изменение начала и конца СиПР через JSON, с проверками маршрута и сервиса.
POST /refdata/api/update_sipr_dates	Изменение дат СиПР через JSON.

Данные о периодах передаются общим сервисам годов и далее используются в годовых фильтрах и расчетных представлениях. Смена текущего года изменяет классификацию временной оси; она не означает автоматического выполнения всех расчетов АРМ. Сервисные адреса возвращают JSON либо перенаправление и не являются дополнительными самостоятельными HTML-страницами.

4. Страницы справочников и формы добавления

Ниже описаны все 25 справочников, доступных из навигации модуля, включая административный справочник годов. Для каждого отдельно рассматриваются список и форма добавления. Технические адреса приведены с префиксом /refdata. Номера строк соответствуют файлам рабочей копии на дату подготовки. Листинги фрагментарные: промежуточные части функций не выдаются за полный исполняемый модуль. В русскоязычном тексте листингов нормализована типографика.

4.1. Годы

4.1.1. Страница списка

Адрес: GET, POST /refdata/years. Обработчик: years_list().

Задает календарные годы и их признаки. Таблица позволяет просматривать, искать и редактировать годы; отдельная форма создает новый год. Признак года выбирается из YearFeature. Календарный номер года не равен внутреннему первичному ключу id.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, year_filter, year_ids[], year_numbers[], year_features[], year_delete[].

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Доступ к списку, добавлению и экспорту годов ограничен has_admin_required. Межверсионных кнопок у этого справочника нет.

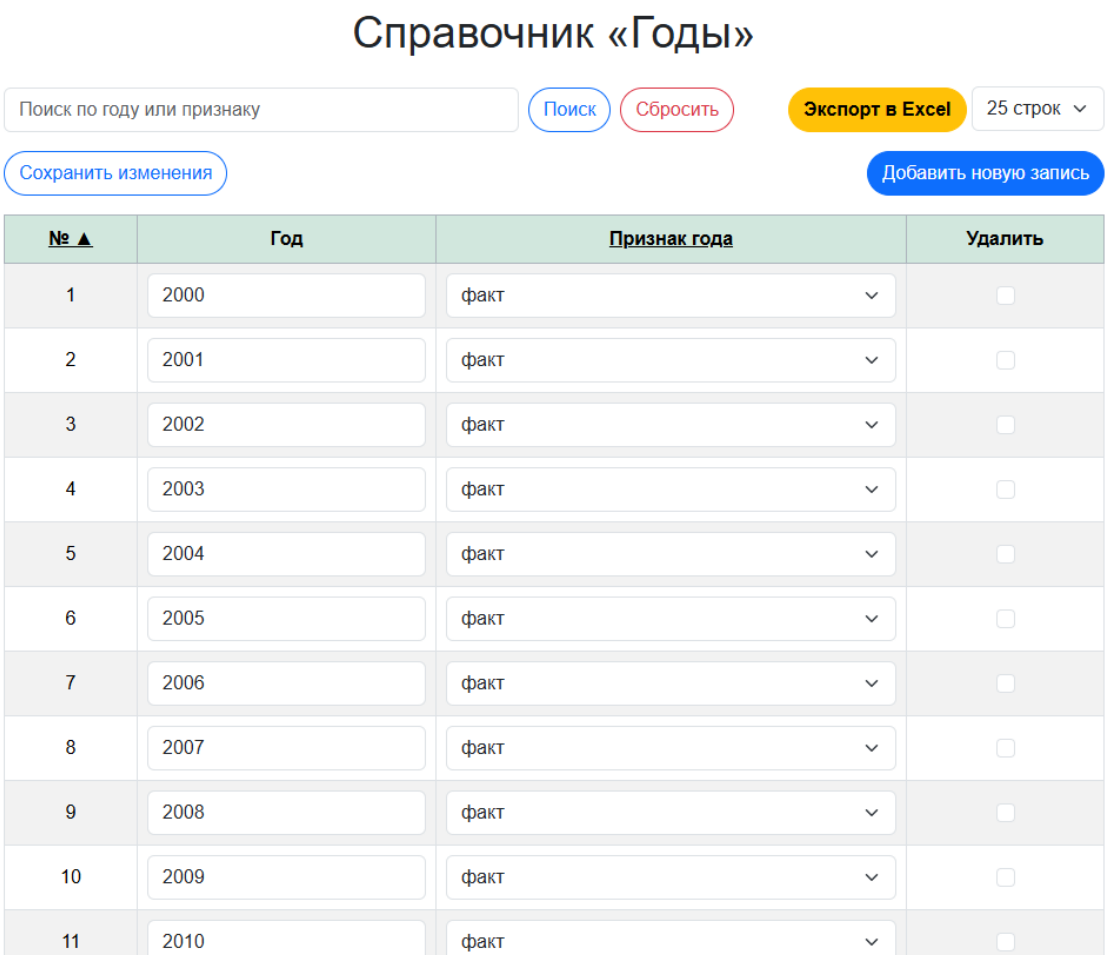


Рисунок 2. Годы. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Модель: Year. Поля SQLAlchemy: id, number, id_year_feature, year_feature, database_version_id, created_at, updated_at, machine_powers, machine_fuels, machine_tes_types, machine_names, pgu_machine_names, fd_economic_activity_consumption_parameters, rf_economic_activity_consumption_parameters, fd_economic_activity_accum_fixed_capital_parameters, rf_economic_activity_accum_fixed_capital_parameters, fd_economic_activity_product_output_parameters, rf_economic_activity_product_output_parameters, fd_electrical_intensity_year_parameters, rf_electrical_intensity_year_parameters, fd_population_parameters, fd_accum_monetary_income_parameters, fd_population_consumption_year_parameters, station_energy_generations, espp_energy_generations, regional_energy_system_energy_generations.

Модель данных – фрагмент объявления. Источник: app/refdata/models/years/year_model.py, строки 12–35.

```
class Year(db.Model, AuditMixin, VersionedModelMixin):
    __tablename__ = 'gs_sys_years'
    __table_args__ = (
        db.UniqueConstraint(
            "number",
            "database_version_id",
            name="uq_gs_years_number_version",
        ),
        {
            "schema": SCHEMA_REFDATA,
            "extend_existing": True,
        },
    )

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    number = db.Column(db.Integer, nullable=False, index=True)

    # FK -> YearFeature
    id_year_feature = db.Column(
        db.Integer,
        db.ForeignKey(f'{SCHEMA_REFDATA}.gs_sys_year_features.id', ondelete='RESTRICT'),
        nullable=True,
        index=True
    )
```

Связь модели с другой сущностью. Источник: app/refdata/models/years/year_model.py, строки 36–36.

```
    year_feature = db.relationship('YearFeature', back_populates='years')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/years/year_routes.py, строки 38–53.

```
@refdata_bp.route("/years", methods=["GET", "POST"])
@login_required
@has_admin_required
def years_list():
    """Страница справочника «Годы» (таблица + редактирование)."""

    user = session.get("username", "Неизвестный пользователь")
    log_to_db(user, "Открыта страница: годы", entity_type="year")

    # Для dropdown (строго по активной версии)
    version_id = get_current_version()

    # Форма
    form = YearFilterForm()

    # Параметры отображения
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/years/year_routes.py, строки 135–140.

```
                raise ValueError(f"Обнаружены дублирующиеся ID годов: {duplicates}")

            update_year_service(payload, user)
            flash("Изменения успешно сохранены.", "success")
        except ValueError as e:
            flash(str(e), "danger")
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/years/year_services.py, строки 64–72.

```
@no_autoflush
def get_year_list(page, per_page, year_filter=None, sort_by="number", sort_dir="asc"):
    """Получает список годов с пагинацией, фильтрацией и сортировкой."""
    query = year_query(
        year_filter=year_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/years/year_services.py, строки 149–158.

```
 149
 150          db.session.flush()
 151
 152      try:
 153          _commit_with_retry()
 154          return updated_ids
 155      except IntegrityError as e:
 156          db.session.rollback()
 157          raise ValueError(
 158              "Ошибка сохранения данных. Возможно, нарушены уникальные ограничения или внешние
ключи."
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/years/year/year.html, строки 15–28.

```
  15      {% endif %}
  16
  17      <form method="GET" action="{{ url_for('refdata_bp.years_list') }}" class="mb-3"
id="filterForm">
  18          <div class="row g-2 align-items-center">
  19              <div class="col-12 col-lg-8">
  20                  <div class="d-flex align-items-center gap-2 flex-wrap flex-lg-nowrap">
  21                      <div class="flex-grow-1" style="min-width: 220px;">
  22                          {{ form.year_filter(class="form-control", value=year_filter,
placeholder="Поиск по году или признаку") }}
  23                      </div>
  24                      <button type="submit" class="btn btn-outline-primary"
onclick="this.form.page.value=1">
  25                          Поиск
  26                      </button>
  27                      <a href="{{ url_for('refdata_bp.years_list') }}" class="btn btn-outline-danger"
data-no-preloader>
  28                          Сбросить
```

Взаимодействие с другими модулями. Годы используются как общая временная ось мощности, топлива, экономики, электропотребления и балансов. Ссылки id_year ведут на Year.id; в отдельных экономических моделях применяется сочетание year_number и database_version_id. Изменение текущего года выполняется отдельно на главной странице и не тождественно добавлению записи Year.

Подтверждающие ссылки на потребителей модели:
app/ecology/models/new_tes_norm_model.py:45 – "Year",
app/economics/models/federal_district_accum_fixed_capital_parameter_model.py:52 – "Year",
app/electrical_intensity/models/federal_district_electrical_intensity_year_parameter_model.py:54 – "Year",
app/energy_balance/models/espp_energy_generation_model.py:6 – - year_number – календарный год (ссылка на Year.number вместе с database_version_id);
app/fuel/models/fue_distribution_parameter_model.py:42 – year = db.relationship("Year", foreign_keys=[id_year])

4.1.2. Форма добавления

Адрес: GET, POST /refdata/add_year. Обработчик: add_year().

Назначение: создание записи справочника «Годы». Вводимые поля формы: number, year_feature. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – Year, модель и ее связи приведены выше в подразделе 4.1.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); number = IntegerField('Год', validators=[DataRequired(message='Поле «Год» обязательно.'),

NumberRange(min=1900, max=2200, message='Год должен быть в диапазоне 1900–2200.')]); year_feature = SelectField('Признак года', choices=[], coerce=int, validators=[DataRequired(message='Выберите признак года.')]).

Отдельной кнопки добавления во всех версиях в этой форме нет.

Модель добавляемого объекта Year – основной реквизит. Источник: app/refdata/models/years/year_model.py, строки 27–27.

```
27      number = db.Column(db.Integer, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/years/year_routes.py, строки 220–232.

```
220 @refdata_bp.route("/add_year", methods=["GET", "POST"])
221 @login_required
222 @has_admin_required
223 def add_year():
224     """Страница добавления записи в справочник «Годы»."""
225
226     user = session.get("username", "Неизвестный пользователь")
227     log_to_db(user, "Открыта страница добавления года", entity_type="year")
228
229     version_id = get_current_version()
230     if not version_id:
231         flash("Текущая версия БД не установлена", "danger")
232         return redirect(url_for("refdata_bp.years_list"))
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/years/year_routes.py, строки 251–260.

```
251                 {
252                     "number": form.number.data,
253                     "year_feature_id": form.year_feature.data,
254                 }
255             ]
256             add_year_service(payload, user)
257             flash("Новая запись успешно добавлена.", "success")
258
259             total_records = year_query(year_filter=year_filter).count()
260             last_page = (total_records + per_page - 1) // per_page
```

Сервис добавления – создание сущности. Источник: app/refdata/services/years/year_services.py, строки 201–210.

```
201             if exists:
202                 raise ValueError(f"Год «{number}» уже существует в текущей версии БД.")
203
204             obj = Year(number=number, id_year_feature=feature_id)
205             set_db_version_on_create(obj)
206
207             db.session.add(obj)
208             db.session.flush()
209             created_ids.append(obj.id)
210
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/years/year/year_add.html, строки 19–33.

```
19          <div class="mb-3">
20              <label for="number" class="form-label">Год</label>
21              {{ form.number(class="form-control", id="number") }}
22              {% if form.number.errors %}
23                  <div class="text-danger">{{ form.number.errors|join(', ') }}</div>
24              {% endif %}
25          </div>
26
27          <div class="mb-3">
28              <label for="year_feature" class="form-label">Признак года</label>
29              {{ form.year_feature(class="form-select", id="year_feature") }}
30              {% if form.year_feature.errors %}
31                  <div class="text-danger">{{ form.year_feature.errors|join(', ') }}</div>
32              {% endif %}
33          </div>
```

Взаимодействие при добавлении. Годы используются как общая временная ось мощности, топлива, экономики, электропотребления и балансов. Ссылки id_year ведут на Year.id; в отдельных экономических моделях применяется

сочетание year_number и database_version_id. Изменение текущего года выполняется отдельно на главной странице и не тождественно добавлению записи Year. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.1.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_years	export_years	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.2. Федеральные округа

4.2.1. Страница списка

Адрес: GET, POST /refdata/federal_district. Обработчик: federal_district_list().

Ведет перечень федеральных округов и варианты их наименований. Предусмотрены поиск, изменение строк таблицы, добавление записей, импорт и экспорт Excel. Округа являются верхним уровнем административной группировки субъектов РФ.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, federal_district_filter, all_versions, federal_district_ids[], display_orders[], federal_district_names[], federal_district_full_names[], federal_district_abr_names[], federal_district_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: FederalDistrict. Поля SQLAlchemy: id, display_order, name, name_full, name_abr, created_at, updated_at, database_version_id, regional_districts, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/territories/federal_district_model.py, строки 13–35.

```
class FederalDistrict(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_federal_districts'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)

    # Порядок отображения
    display_order = db.Column(db.Integer, nullable=True)

    name = db.Column(db.String(80), nullable=False, index=True)
    name_full = db.Column(db.String(80), nullable=True)
    name_abr = db.Column(db.String(80), nullable=True)

    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)

    # Поле для связи с версией БД
    database_version_id = db.Column(
        db.Integer,
        db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
        nullable=True,
        index=True
    )
```

Связь модели с другой сущностью. Источник: app/refdata/models/territories/federal_district_model.py, строки 44–44.

```
    regional_districts = db.relationship('RegionalDistrict', back_populates='federal_district')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/territories/federal_district_routes.py, строки 36–51.

```
@refdata_bp.route("/federal_district", methods=["GET", "POST"])
@login_required
def federal_district_list():
    """Маршрут для отображения списка федеральных округов."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(user, "Открыта страница федеральных округов", entity_type="federal_district")

    # Создание формы
    form = FederalDistrictFilterForm()

    # Получение параметров запроса
    page = request.args.get("page", 1, type=int)
    per_page = request.args.get("per_page", 25, type=int)
    federal_district_filter = request.args.get("federal_district_filter", "").strip()
    sort_by = request.args.get("sort_by", "display_order")
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/territories/federal_district_routes.py, строки 146–151.

```
                            sort_dir=sort_dir,
                    ))
                update_federal_district_all_versions_service(federal_district_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_federal_district_service(federal_district_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/territories/federal_district_services.py, строки 153–171.

```
153  @no_autoflush
154  def get_federal_district_list(
155      page,
156      per_page,
157      federal_district_filter=None,
158      sort_by="display_order",
159      sort_dir="asc"
160  ):
161      """ Получает список федеральных округов с пагинацией, фильтрацией и сортировкой. """
162
163      # Базовый запрос
164      query = federal_district_query(
165          federal_district_filter=federal_district_filter,
166          sort_by=sort_by,
167          sort_dir=sort_dir,
168      )
169
170      # Пагинация
171      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/territories/federal_district_services.py, строки 263–272.

```
263
264      try:
265          # Сохранение изменений в базе данных
266          # Фиксация транзакции (устойчивый коммит)
267          _commit_with_retry()
268
269          if updated_ids:
270              log_to_db(user, "Сохранены изменения по федеральным округам", f"Измененных записей:
{len(updated_ids)} (id: {updated_ids})", entity_type="federal_district")
271          else:
272              log_to_db(user, "Изменений по федеральным округам не обнаружено", "",
entity_type="federal_district")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/territories/federal_district/federal_district.html, строки 236–249.

```
236                                  <!-- Строки таблицы. Столбец "Наименование ФО" -->
237                                  <td>
238                                      <input type="text" name="federal_district_names[]" class="form-
control" value="{{ item.name }}">
239                                  </td>
240
241                                  <!-- Строки таблицы. Столбец "Наименование полное ФО" -->
242                                  <td>
243                                      <input type="text" name="federal_district_full_names[]"
class="form-control" value="{{ item.name_full }}">
244                                  </td>
245
246                                  <!-- Строки таблицы. Столбец "Наименование сокращенное ФО" -->
247                                  <td>
248                                      <input type="text" name="federal_district_abr_names[]"
class="form-control" value="{{ item.name_abr }}">
249                                  </td>
```

Взаимодействие с другими модулями. RegionalDistrict.id_federal_district связывает субъект РФ с округом. Территориальная группировка используется в экономике, электропотреблении, электроемкости и потребности в мощности. Изменение наименования справочника отражается при последующем построении списков и отчетов, но само по себе не является запуском расчета.

Подтверждающие ссылки на потребителей модели:
app/economics/models/federal_district_accum_fixed_capital_parameter_model.py:35 – "FederalDistrict",
app/electrical_intensity/models/federal_district_electrical_intensity_coefficient_model.py:24 – "FederalDistrict",
app/energy_consumption/models/territories/federal_district_energy_consumption_parameter_model.py:26 – "FederalDistrict",
app/power_demand/models/territories/federal_district_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для федерального округа (FederalDistrict)."""

app/fuel/services/fuel_imports/import_fuel_refdata_services.py:46 – from app.refdata.models.territories.federal_district_model import FederalDistrict

4.2.2. Форма добавления

Адрес: GET, POST /refdata/add_federal_district. Обработчик: add_federal_district().

Назначение: создание записи справочника «Федеральные округа». Вводимые поля формы: name, name_full, name_abr. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – FederalDistrict, модель и ее связи приведены выше в подразделе 4.2.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Короткое наименование ФО', validators=[DataRequired(message='Поле «Короткое наименование ФО» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_full = StringField('Полное наименование ФО', validators=[DataRequired(message='Поле «Полное наименование ФО» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); name_abr = StringField('Сокращенное наименование ФО', validators=[DataRequired(message='Поле «Сокращенное наименование ФО» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта FederalDistrict – основной реквизит. Источник: app/refdata/models/territories/federal_district_model.py, строки 22–22.

```
22      name = db.Column(db.String(80), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/territories/federal_district_routes.py, строки 190–202.

```
190  @refdata_bp.route("/add_federal_district", methods=["GET", "POST"])
191  @login_required
192  def add_federal_district():
193      """ Маршрут для добавления нового федерального округа. """
194
195      user = session.get('username', 'Неизвестный пользователь')
196      log_to_db(user, "Открыта страница добавления федерального округа",
entity_type="federal_district")
197
198      # Создание формы
199      form = AddFederalDistrictForm()
200
201      # Сохранение текущих фильтров и параметров отображения
202      page                         = request.args.get("page", 1, type=int)
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/territories/federal_district_routes.py, строки 235–245.

```
                                            sort_dir=sort_dir,
                        ))
                    add_federal_district_all_versions_service(payload, user)
                    flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
                else:
                    add_federal_district_service(payload, user)
                    flash("Новая запись успешно добавлена.", "success")

                # Перенаправление на список с сохранением параметров и переходом к новой записи
                total_records = federal_district_query(
                                    federal_district_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/territories/federal_district_services.py, строки 329–350.

```
                    raise ValueError(f"Запись с сокращенным наименованием «{name_abr}» уже
существует.")

                # Создаем новую запись
                obj = FederalDistrict(
                    name=name,
                    name_full=name_full or None,
                    name_abr=name_abr,
                )
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()  # получить id без полного коммита

                log_to_db(
                    user,
                    "Создан федеральный округ",
                    (
                        f"Наименование: {name};"
                        f"Полное наименование: {_dash(name_full)};"
                        f"Сокращенное наименование: {_dash(name_abr)}",
                    ),
                    entity_type="federal_district",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/territories/federal_district/federal_district_add.html, строки 22–36.

```
        <div class="mb-3">
            <label for="name" class="form-label">Наименование ФО</label>
            {{ form.name(class="form-control", id="name") }}

            <label for="name" class="form-label">Полное наименование ФО</label>
            {{ form.name_full(class="form-control", id="name_full") }}

            <label for="name" class="form-label">Сокращенное наименование ФО</label>
            {{ form.name_abr(class="form-control", id="name_abr") }}
            {% if form.name.errors %}
                <div class="text-danger">
                    {{ form.name.errors|join(', ') }}
                </div>
            {% endif %}
        </div>
```

Взаимодействие при добавлении. RegionalDistrict.id_federal_district связывает субъект РФ с округом. Территориальная группировка используется в экономике, электропотреблении, электроемкости и потребности в мощности. Изменение наименования справочника отражается при последующем построении списков и отчетов, но само по себе не является запуском расчета. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.2.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат

Адрес и метод	Обработчик	Результат
POST /refdata/import_federal_district	import_federal_district	Обработка загруженного файла и сообщение о результате
GET /refdata/export_federal_district	export_federal_district	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

4.3. Субъекты РФ

4.3.1. Страница списка

Адрес: GET, POST /refdata/regional_district. Обработчик: regional_district_list().

Хранит порядковый номер, краткое и полное наименования субъекта РФ, формы наименования в родительном и дательном падежах, связи с федеральным округом, энергозоной и синхронной зоной. В списке доступны текстовый поиск и фильтры по связанным классификаторам; региональные энергосистемы связаны с субъектами отношением многие-ко-многим.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, federal_district_filter, energy_zone_filter, synchronous_area_filter, regional_district_filter, all_versions, regional_district_ids[], region_ids[], regional_district_names[], regional_district_full_names[], regional_district_rp_names[], regional_district_dp_names[], federal_districts[], energy_zones[], synchronous_areas[], regional_district_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

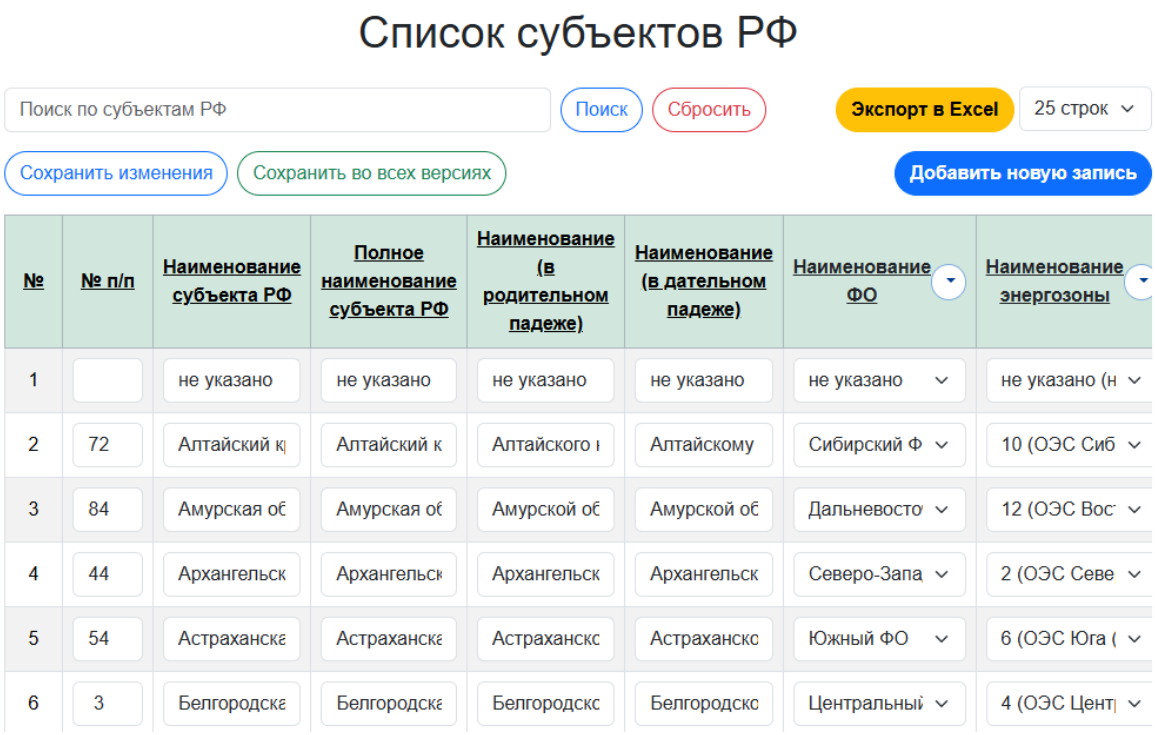


Рисунок 3. Субъекты РФ. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Модель: RegionalDistrict. Поля SQLAlchemy: id, region_number, region_id, name, name_full, name_rp, name_dp, id_federal_district, federal_district, regional_energy_systems, stations, energy_areas, energy_units, id_energy_zone, energy_zone, id_synchronous_area, synchronous_area, created_at, updated_at, database_version_id, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/territories/regional_district_model.py, строки 14–32.

```
14  class RegionalDistrict(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
15      __tablename__ = 'gs_sys_regional_districts'
16      __table_args__ = {"schema": SCHEMA_REFDATA}
17
18      # Идентификатор субъекта РФ
19      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
20
21      # Номер региона РФ (может быть пустым)
22      region_number = db.Column(db.String(3), nullable=True, index=True)
23
24      # Номер порядковый (может быть пустым)
25      region_id = db.Column(db.String(3), nullable=True, index=True)
26
27      # Наименование сокращенное
28      name = db.Column(db.String(80), nullable=False, index=True)
29
30      # Полное наименование
31      name_full = db.Column(db.String(255), nullable=False)
32
```

Связь модели с другой сущностью. Источник: app/refdata/models/territories/regional_district_model.py, строки 46–46.

```
46      federal_district = db.relationship('FederalDistrict', back_populates='regional_districts')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/territories/regional_district_routes.py, строки 42–57.

```
@refdata_bp.route("/regional_district", methods=["GET", "POST"])
@login_required
def regional_district_list():
    """Маршрут для отображения списка субъектов РФ."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(user, "Открыта страница субъектов РФ", entity_type="regional_district")

    # Создание формы
    form = RegionalDistrictFilterForm()

    # Получение параметров запроса
    page                        = request.args.get("page", 1, type=int)
    per_page                    = request.args.get("per_page", 25, type=int)
    sort_by                     = request.args.get("sort_by", "id")
    sort_dir                    = request.args.get("sort_dir", "asc")
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/territories/regional_district_routes.py, строки 175–180.

```
                            synchronous_area_filter=synchronous_area_filter,
                        ))
                    update_regional_district_all_versions_service(regional_district_data, user)
                    flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
                else:
                    update_regional_district_service(regional_district_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/territories/regional_district_services.py, строки 193–214.

```
@no_autoflush
def get_regional_district_list(
    page,
    per_page,
    regional_district_filter=None,
    federal_district_filter=None,
    energy_zone_filter=None,
    synchronous_area_filter=None,
    sort_by="id",
    sort_dir="asc",
):
    """ Получает список субъектов РФ с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = regional_district_query(
        regional_district_filter=regional_district_filter,
        federal_district_filter=federal_district_filter,
        energy_zone_filter=energy_zone_filter,
        synchronous_area_filter=synchronous_area_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/territories/regional_district_services.py, строки 386–395.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(user, "Сохранены изменения по субъектам РФ",
                      f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
entity_type="regional_district")
        else:
```

HTML/Jinja2 списка – элементы формы. Источник:
app/templates/refdata/territories/regional_district/regional_district.html, строки 493–506.

```
 493                                    <!-- Строки таблицы. Столбец "Субъект РФ" -->
 494                                    <td>
 495                                        <input type="text" name="regional_district_names[]"
class="form-control" value="{{ item.name }}">
 496                                    </td>
 497
 498                                    <!-- Строки таблицы. Столбец "Субъект РФ (полное наименование)" -->
 499                                    <td>
 500                                        <input type="text" name="regional_district_full_names[]"
class="form-control" value="{{ item.name_full }}">
 501                                    </td>
 502
 503                                    <!-- Строки таблицы. Столбец "Наименование (в родительном падеже)"
-->
 504                                    <td>
 505                                        <input type="text" name="regional_district_rp_names[]"
class="form-control" value="{{ item.name_rp }}">
 506                                    </td>
```

Взаимодействие с другими модулями. Субъекты РФ используются станциями, энергорайонами, энергоузлами, показателями потребности в мощности и территориальными расчетами. Сервис _sync_energy_zone_aggregates_for_ids обновляет территориальные агрегаты энергозон через общий сервис синхронизации при изменении соответствующих связей. При смене внешнего ключа меняется принадлежность объекта к расчетной группировке.

Подтверждающие ссылки на потребителей модели:
app/energy_balance/models/regional_energy_system_transfer_model.py:5 – Источник перетока – RegionalEnergySystem или RegionalDistrict (субъект РФ), ровно одна связь.
app/energy_consumption/models/territories/regional_district_energy_consumption_parameter_model.py:26 – "RegionalDistrict",
app/fuel/models/fue_equipment_group_fuel_param_model.py:23 – from app.refdata.models.territories.regional_district_model import RegionalDistrict
app/generation/models/station/station_model.py:85 – regional_district = db.relationship('RegionalDistrict', back_populates='stations')
app/power_demand/models/territories/regional_district_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для субъекта РФ (RegionalDistrict)."""

4.3.2. Форма добавления

Адрес: GET, POST /refdata/add_regional_district. Обработчик: add_regional_district().

Назначение: создание записи справочника «Субъекты РФ». Вводимые поля формы: name, name_full, name_rp, name_dp, federal_district. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – RegionalDistrict, модель и ее связи приведены выше в подразделе 4.3.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Наименование субъекта РФ', validators=[DataRequired(message='Поле «Наименование субъекта РФ» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_full = StringField('Полное наименование субъекта РФ', validators=[DataRequired(message='Поле «Полное наименование субъекта РФ» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_rp = StringField('Наименование (в

родительном падеже)', validators=[DataRequired(message='Поле «Наименование (в родительном падеже)» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_dp = StringField('Наименование (в дательном падеже)', validators=[DataRequired(message='Поле «Наименование (в дательном падеже)» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); federal_district = SelectField('Федеральный округ', choices=[], coerce=int, validators=[DataRequired(message='Выберите федеральный округ.')]); energy_zone = SelectField('Энергозона', choices=[], coerce=int, validators=[Optional()]); synchronous_area = SelectField('Синхронная зона', choices=[], coerce=int, validators=[Optional()]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта RegionalDistrict – основной реквизит. Источник: app/refdata/models/territories/regional_district_model.py, строки 28–28.

```
28      name = db.Column(db.String(80), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/territories/regional_district_routes.py, строки 245–257.

```
245 @refdata_bp.route("/add_regional_district", methods=["GET", "POST"])
246 @login_required
247 def add_regional_district():
248     """ Маршрут для добавления нового субъекта РФ."""
249
250     user = session.get("username", "Неизвестный пользователь")
251     log_to_db(user, "Открыта страница добавления субъекта РФ", entity_type="regional_district")
252
253     # Создание формы
254     form = AddRegionalDistrictForm()
255
256     # Сохранение текущих фильтров и параметров отображения
257     page                        = request.args.get("page", 1, type=int)
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/territories/regional_district_routes.py, строки 302–315.

```
302                             synchronous_area_filter=synchronous_area_filter,
303                     ))
304                 add_regional_district_all_versions_service(payload, user)
305                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
306             else:
307                 add_regional_district_service(payload, user)
308                 flash("Новая запись успешно добавлена.", "success")
309
310             # Перенаправление на список с сохранением параметров и переходом к новой записи
311             total_records = regional_district_query(
312                                 regional_district_filter,
313                                 federal_district_filter,
314                                 energy_zone_filter,
315                                 synchronous_area_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/territories/regional_district_services.py, строки 457–482.

```
                    raise ValueError(f"Запись с полным наименованием «{name_full}» уже
существует.")

                # Создаем новую запись
                obj = RegionalDistrict(
                    name=name,
                    name_full=name_full or None,
                    name_rp=name_rp or None,
                    name_dp=name_dp or None,
                    id_federal_district=federal_district_id,
                )
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()  # получить id без полного коммита

                log_to_db(
                    user,
                    "Создан субъект РФ",
                    (
                        f"Наименование: {name};"
                        f"Полное наименование: {_dash(name_full)};"
                        f"Наименование (в родительном падеже): {_dash(name_rp)};"
                        f"Наименование (в дательном падеже): {_dash(name_dp)};"
                        f"Федеральный округ: {get_federal_district_name(federal_district_id)}",
                    ),
                    entity_type="regional_district",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/territories/regional_district/regional_district_add.html, строки 23–37.

```
        <div class="mb-3">
            <label for="name" class="form-label">Наименование субъекта РФ</label>
            {{ form.name(class="form-control", id="name") }}

            <!-- Ошибки валидации -->
            {% if form.name.errors %}
                <div class="text-danger">
                    {{ form.name.errors|join(', ') }}
                </div>
            {% endif %}
        </div>

        <!-- Полное наименование субъекта РФ -->
        <div class="mb-3">
            <label for="name_full" class="form-label">Полное наименование субъекта РФ</label>
```

Взаимодействие при добавлении. Субъекты РФ используются станциями, энергорайонами, энергоузлами, показателями потребности в мощности и территориальными расчетами. Сервис _sync_energy_zone_aggregates_for_ids обновляет территориальные агрегаты энергозон через общий сервис синхронизации при изменении соответствующих связей. При смене внешнего ключа меняется принадлежность объекта к расчетной группировке. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

В текущей форме создания субъекта РФ вводятся наименования и ФО. Связи с энергозоной и синхронной зоной редактируются в списке; наличие этих полей в модели не означает их наличие в форме добавления.

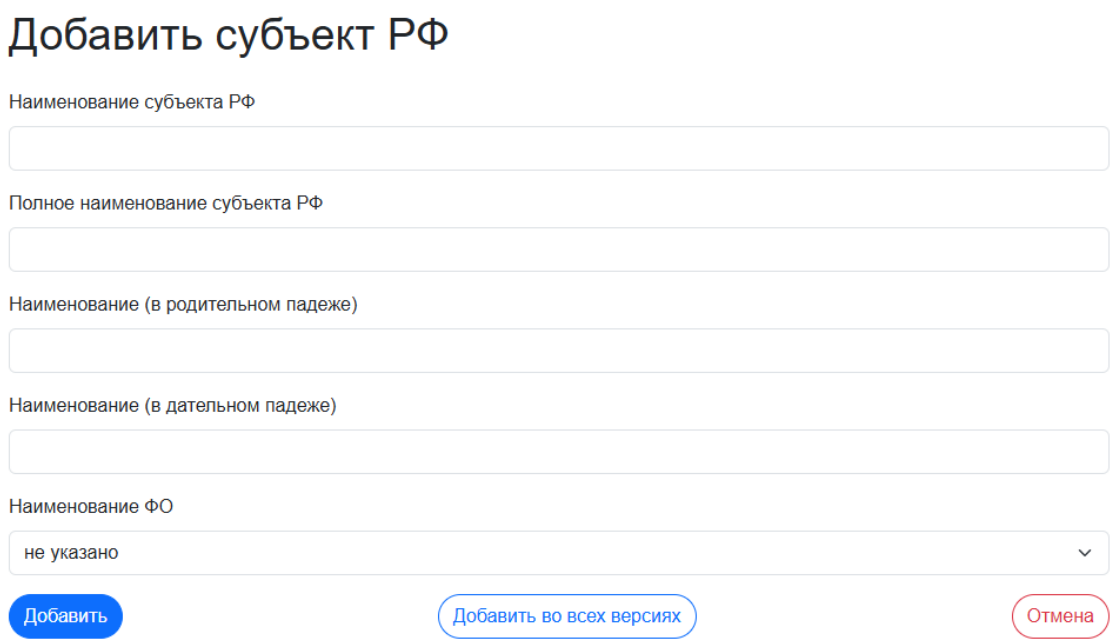


Рисунок 4. Форма добавления субъекта РФ. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

4.3.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
POST /refdata/import_regional_district	import_regional_district	Обработка загруженного файла и сообщение о результате
GET /refdata/export_regional_district	export_regional_district	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

4.4. Зарубежные страны, имеющие общие границы с РФ

4.4.1. Страница списка

Адрес: GET, POST /refdata/foreign_border_country. Обработчик: foreign_border_country_list().

Ведет упорядоченный перечень сопредельных зарубежных стран. Список поддерживает редактирование названия и порядка отображения; отдельная форма служит для добавления страны.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, foreign_border_country_filter, all_versions, foreign_border_country_ids[], display_orders[], foreign_border_country_names[], foreign_border_country_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: ForeignBorderCountry. Поля SQLAlchemy: id, display_order, name, created_at, updated_at, database_version_id.

Модель данных – фрагмент объявления. Источник: app/refdata/models/territories/foreign_border_country_model.py, строки 13–31.

```
  13  class ForeignBorderCountry(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_foreign_border_countries'
  15      __table_args__ = (
  16          db.UniqueConstraint(
  17              'database_version_id',
  18              'name',
  19              name='uq_gs_sys_foreign_border_countries_ver_name',
  20          ),
  21          {"schema": SCHEMA_REFDATA},
  22      )
  23
  24      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  25
  26      # Порядок отображения
  27      display_order = db.Column(db.Integer, nullable=True)
  28
  29      # Наименование
  30      name = db.Column(db.String(255), nullable=False, index=True)
  31
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/territories/foreign_border_country_routes.py, строки 37–52.

```
  37  @refdata_bp.route("/foreign_border_country", methods=["GET", "POST"])
  38  @login_required
  39  def foreign_border_country_list():
  40      """Маршрут для отображения списка зарубежных стран."""
  41
  42      user = session.get('username', 'Неизвестный пользователь')
  43      log_to_db(
  44          user,
  45          "Открыта страница зарубежных стран, имеющих общие границы с РФ",
  46          entity_type="foreign_border_country",
  47      )
  48
  49      form = ForeignBorderCountryFilterForm()
  50
  51      page = request.args.get("page", 1, type=int)
  52      per_page = request.args.get("per_page", 25, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/territories/foreign_border_country_routes.py, строки 147–152.

```
 147                          sort_dir=sort_dir,
 148                      ))
 149                  update_foreign_border_country_all_versions_service(foreign_border_country_data,
user)
 150                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 151              else:
 152                  update_foreign_border_country_service(foreign_border_country_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/territories/foreign_border_country_services.py, строки 71–86.

```
@no_autoflush
def get_foreign_border_country_list(
    page,
    per_page,
    foreign_border_country_filter=None,
    sort_by="display_order",
    sort_dir="asc",
):
    """Получает список стран с пагинацией, фильтрацией и сортировкой."""

    query = foreign_border_country_query(
        foreign_border_country_filter=foreign_border_country_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/territories/foreign_border_country_services.py, строки 185–198.

```

        db.session.flush()

    try:
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по зарубежным странам",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="foreign_border_country",
                entity_id=foreign_border_country_id,
            )
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/territories/foreign_border_country/foreign_border_country.html, строки 164–177.

```

                                <td>
                                    <input type="text" name="foreign_border_country_names[]"
class="form-control" value="{{ country.name }}">
                                </td>

                                <td class="text-center align-middle">
                                    <input type="checkbox" name="foreign_border_country_delete[]"
value="{{ country.id }}" class="form-check-input">
                                </td>
                            </tr>
                            {% else %}
                            <tr>
                                <td colspan="4" class="text-center align-middle">Нет данных для
отображения</td>
                            </tr>
                            {% endfor %}
```

Взаимодействие с другими модулями. Записи используются энергетическими балансами при описании межгосударственных перетоков и импортно-экспортных связей. Связь с конкретным потребителем подтверждается перечнем ссылок на модель в этом разделе; изменение записи справочника не создает автоматически строку баланса.

Подтверждающие ссылки на потребителей модели:
app/energy_balance/models/ee_import_export_oes_value_model.py:68 – foreign_border_country = db.relationship("ForeignBorderCountry")
app/energy_balance/models/regional_energy_system_transfer_model.py:6 – Получатель – RegionalEnergySystem, ForeignBorderCountry, RegionalDistrict или EnergyUnit
app/energy_balance/services/ee_import_export_value_services.py:21 – from app.refdata.models.territories.foreign_border_country_model import ForeignBorderCountry

app/energy_balance/services/ee_transfers_page_services.py:46 – from app.refdata.models.territories.foreign_border_country_model import ForeignBorderCountry
app/energy_balance/services/ee_transfers_resolve_services.py:13 – from app.refdata.models.territories.foreign_border_country_model import ForeignBorderCountry

4.4.2. Форма добавления

Адрес: GET, POST /refdata/add_foreign_border_country. Обработчик: add_foreign_border_country().

Назначение: создание записи справочника «Зарубежные страны, имеющие общие границы с РФ». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – ForeignBorderCountry, модель и ее связи приведены выше в подразделе 4.4.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Наименование страны', validators=[DataRequired(message='Поле «Наименование страны» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта ForeignBorderCountry – основной реквизит. Источник: app/refdata/models/territories/foreign_border_country_model.py, строки 30–30.

```
30      name = db.Column(db.String(255), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/territories/foreign_border_country_routes.py, строки 189–201.

```
189 @refdata_bp.route("/add_foreign_border_country", methods=["GET", "POST"])
190 @login_required
191 def add_foreign_border_country():
192     """Маршрут для добавления новой зарубежной страны."""
193
194     user = session.get('username', 'Неизвестный пользователь')
195     log_to_db(
196         user,
197         "Открыта страница добавления зарубежных стран",
198         entity_type="foreign_border_country",
199     )
200
201     form = AddForeignBorderCountryForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/territories/foreign_border_country_routes.py, строки 233–242.

```
233                         sort_dir=sort_dir,
234                     ))
235                 add_foreign_border_country_all_versions_service(payload, user)
236                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
237             else:
238                 add_foreign_border_country_service(payload, user)
239                 flash("Новая запись успешно добавлена.", "success")
240
241             total_records = foreign_border_country_query(foreign_border_country_filter).count()
242             last_page = (total_records + per_page - 1) // per_page
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/territories/foreign_border_country_services.py, строки 272–286.

```
                                f"Запись с порядком отображения «{display_order}» уже существует."
                            )

                obj = ForeignBorderCountry(name=name, display_order=display_order)
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()

                log_to_db(
                    user,
                    "Создана запись зарубежной страны",
                    f"Наименование: {name}",
                    entity_type="foreign_border_country",
                    entity_id=obj.id,
                )
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/territories/foreign_border_country/foreign_border_country_add.html, строки 19–33.

```
        <div class="mb-3">
            <label for="name" class="form-label">Наименование страны</label>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>

        <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
            <button type="submit" class="btn btn-primary">Добавить</button>
            {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
            <a href="{{ url_for('refdata_bp.foreign_border_country_list',
                                page=page,
                                per_page=per_page,
                                sort_by=sort_by,
```

Взаимодействие при добавлении. Записи используются энергетическими балансами при описании межгосударственных перетоков и импортно-экспортных связей. Связь с конкретным потребителем подтверждается перечнем ссылок на модель в этом разделе; изменение записи справочника не создает автоматически строку баланса. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.4.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_foreign_border_country	export_foreign_border_country	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.5. Объединенные энергосистемы

4.5.1. Страница списка

Адрес: GET, POST /refdata/union_energy_system. Обработчик: union_energy_system_list().

Хранит краткое и полное наименования ОЭС, порядок отображения и принадлежность к типу энергосистемы. Предусмотрены фильтрация, табличное редактирование, создание, импорт и экспорт данных.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, energy_system_type_filter, union_energy_system_filter, all_versions, union_energy_system_ids[], display_orders[], union_energy_system_names[], union_energy_system_full_names[], energy_system_types[], union_energy_system_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: UnionEnergySystem. Поля SQLAlchemy: id, display_order, name, name_full, id_energy_system_type, created_at, updated_at, database_version_id, regional_energy_systems, energy_system_type, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/union_energy_system_model.py, строки 15–33.

```
15  class UnionEnergySystem(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
16      __tablename__ = 'gs_sys_union_energy_systems'
17      __table_args__ = {"schema": SCHEMA_REFDATA}
18
19      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
20
21      # Порядок отображения
22      display_order = db.Column(db.Integer, nullable=True)
23
24      # Наименование и полное наименование
25      name = db.Column(db.String(80), nullable=False, index=True)
26
27      # Устанавливаем по умолчанию name_full := name на уровне Python-контекста вставки
28      name_full = db.Column(
29          db.String(80),
30          nullable=False,
31          default=lambda context: context.get_current_parameters().get("name")
32      )
33
```

Связь модели с другой сущностью. Источник: app/refdata/models/energy_systems/union_energy_system_model.py, строки 60–63.

```
60      regional_energy_systems = db.relationship(
61          'RegionalEnergySystem',
62          back_populates='union_energy_system'
63      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/union_energy_system_routes.py, строки 40–55.

```
40  @refdata_bp.route("/union_energy_system", methods=["GET", "POST"])
41  @login_required
42  def union_energy_system_list():
43      """Маршрут для отображения списка ОЭС."""
44
45      user = session.get('username', 'Неизвестный пользователь')
46      log_to_db(
47          user,
48          "Открыта страница ОЭС",
49          entity_type="union_energy_system")
50
51      # Создание формы
52      form = UnionEnergySystemFilterForm()
53
54      # Получение параметров запроса
55      page                        = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/union_energy_system_routes.py, строки 155–160.

```
                            sort_dir=sort_dir,
                        ))
                update_union_energy_system_all_versions_service(union_energy_system_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_union_energy_system_service(union_energy_system_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/energy_systems/union_energy_system_services.py, строки 184–203.

```
@no_autoflush
def get_union_energy_system_list(
    page,
    per_page,
    union_energy_system_filter=None,
    energy_system_type_filter=None,
    sort_by="display_order",
    sort_dir="asc"):
    """ Получает список ОЭС с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = union_energy_system_query(
        union_energy_system_filter=union_energy_system_filter,
        energy_system_type_filter=energy_system_type_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/energy_systems/union_energy_system_services.py, строки 318–329.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по ОЭС",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="union_energy_system")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/energy_systems/union_energy_system/union_energy_system.html, строки 9–22.

```
    <h1 class="text-center mb-4">Список объединенных энергосистем</h1>

    <form method="GET"
        action="{{ url_for('refdata_bp.union_energy_system_list') }}"
        class="mb-3"
        id="filterForm">

        <div class="row g-2 align-items-center">
            <!-- ЛЕВАЯ СТОРОНА: поиск + кнопки -->
            <div class="col-12 col-lg-8">
                <div class="d-flex align-items-center gap-2 flex-wrap flex-lg-nowrap">
                    <div class="flex-grow-1" style="min-width: 220px;">
                        {{ form.union_energy_system_filter(
                            class="form-control",
```

Взаимодействие с другими модулями. RegionalEnergySystem.id_union_energy_system задает принадлежность РЭС к ОЭС. Иерархия используется для группировки генерации, электропотребления, потребности в мощности и энергетических балансов. Наименования и внешние

ключи справочника используются повторно, без дублирования классификатора в расчетных модулях.

Подтверждающие ссылки на потребителей модели:
app/energy_balance/models/ee_import_export_oes_value_model.py:67 – union_energy_system = db.relationship("UnionEnergySystem")
app/energy_consumption/models/energy_systems/union_energy_system_energy_consumption_parameter_model.py:26 – "UnionEnergySystem",
app/fuel/models/fue_distribution_parameter_model.py:28 – "UnionEnergySystem",
app/power_demand/models/energy_systems/union_energy_system_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для объединенной энергосистемы (UnionEnergySystem)."""
app/api/services/generation_objects_exchange_services.py:37 – from app.refdata.models.energy_systems.union_energy_system_model import UnionEnergySystem

4.5.2. Форма добавления

Адрес: GET, POST /refdata/add_union_energy_system. Обработчик: add_union_energy_system().

Назначение: создание записи справочника «Объединенные энергосистемы». Вводимые поля формы: name, name_full, energy_system_type. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – UnionEnergySystem, модель и ее связи приведены выше в подразделе 4.5.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Наименование ОЭС', validators=[DataRequired(message='Поле «Наименование ОЭС» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_full = StringField('Полное наименование ОЭС', validators=[DataRequired(message='Поле «Полное наименование ОЭС» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); energy_system_type = SelectField('Тип энергосистемы', choices=[], coerce=int, validators=[DataRequired(message='Выберите тип энергосистемы.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта UnionEnergySystem – основной реквизит. Источник: app/refdata/models/energy_systems/union_energy_system_model.py, строки 25–25.

```
25      name = db.Column(db.String(80), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/union_energy_system_routes.py, строки 203–215.

```
203 @refdata_bp.route("/add_union_energy_system", methods=["GET", "POST"])
204 @login_required
205 def add_union_energy_system():
206     """ Маршрут для добавления новой ОЭС. """
207
208     user = session.get('username', 'Неизвестный пользователь')
209     log_to_db(
210         user,
211         "Открыта страница добавления ОЭС",
212         entity_type="union_energy_system")
213
214     # Создание формы
215     form = AddUnionEnergySystemForm()
```

Маршрут добавления – вызов сервиса. Источник:
app/refdata/routes/energy_systems/union_energy_system_routes.py, строки 258–269.

```
                            sort_dir=sort_dir,
                        ))
                    add_union_energy_system_all_versions_service(payload, user)
                    flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
                else:
                    add_union_energy_system_service(payload, user)
                    flash("Новая запись успешно добавлена.", "success")

                # Перенаправление на список с сохранением параметров и переходом к новой записи
                total_records = union_energy_system_query(
                                    union_energy_system_filter,
                                    energy_system_type_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/energy_systems/union_energy_system_services.py, строки 407–430.

```
                        raise ValueError(f"Запись с порядком отображения «{display_order}» уже
существует.")

                # Создаем новую запись
                obj = UnionEnergySystem(
                    display_order=display_order,
                    name=name,
                    name_full=name_full or None,
                    id_energy_system_type=energy_system_type_id,
                )
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()  # получить id без полного коммита

                log_to_db(
                    user,
                    "Создана ОЭС",
                    (
                        f"Порядок отображения: {display_order};"
                        f"Наименование: {name}; "
                        f"Полное наименование: {_dash(name_full)};"
                        f"Часть энергосистемы России:
{get_energy_system_type_name(energy_system_type_id)}"
                    ),
                    entity_type="union_energy_system",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/energy_systems/union_energy_system/union_energy_system_add.html, строки 23–37.

```
        <div class="mb-3">
            <label for="name" class="form-label">Наименование ОЭС</label>
            {{ form.name(class="form-control", id="name") }}

            <!-- Ошибки валидации -->
            {% if form.name.errors %}
                <div class="text-danger">
                    {{ form.name.errors|join(', ') }}
                </div>
            {% endif %}
        </div>

        <!-- Полное наименование ОЭС -->
        <div class="mb-3">
            <label for="name_full" class="form-label">Полное наименование ОЭС</label>
```

Взаимодействие при добавлении. RegionalEnergySystem.id_union_energy_system задает принадлежность РЭС к ОЭС. Иерархия используется для группировки генерации, электропотребления, потребности в мощности и энергетических балансов. Наименования и внешние ключи справочника используются повторно, без дублирования классификатора в расчетных модулях. После создания запись становится доступна для выбора в

зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.5.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
POST /refdata/import_union_energy_system	import_union_energy_system	Обработка загруженного файла и сообщение о результате
GET /refdata/export_union_energy_system	export_union_energy_system	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

4.6. Региональные энергосистемы

4.6.1. Страница списка

Адрес: GET, POST /refdata/regional_energy_system. Обработчик: regional_energy_system_list().

Ведет наименования РЭС и их принадлежность к ОЭС. Поддерживает связь одной РЭС с несколькими субъектами РФ через таблицу regional_district_regional_energy_system. Список и форма добавления используют варианты выбора из связанных справочников.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, energy_area_filter, union_energy_system_filter, regional_energy_system_filter, all_versions, regional_energy_system_ids[], regional_energy_system_names[], regional_energy_system_full_names[], regional_energy_system_rp_names[], union_energy_system[], regional_energy_system_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: RegionalEnergySystem. Поля SQLAlchemy: id, name, name_full, name_rp, id_union_energy_system, union_energy_system, created_at, updated_at, database_version_id, regional_districts, energy_units, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/regional_energy_system_model.py, строки 16–35.

```
  16  class RegionalEnergySystem(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  17      __tablename__ = 'gs_sys_regional_energy_systems'
  18      __table_args__ = {"schema": SCHEMA_REFDATA}
  19
  20      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  21
  22      # Короткое наименование
  23      name = db.Column(db.String(255), nullable=False, index=True)
  24      # Полное наименование
  25      name_full = db.Column(db.String(255), nullable=False)
  26      # Полное наименование в родительном падеже
  27      name_rp = db.Column(db.String(255), nullable=False)
  28
  29      # FK -> UnionEnergySystem
  30      id_union_energy_system = db.Column(
  31          db.Integer,
  32          db.ForeignKey(f'{SCHEMA_REFDATA}.gs_sys_union_energy_systems.id', ondelete='RESTRICT'),
  33          index=True,
  34          nullable=True
  35      )
```

Связь модели с другой сущностью. Источник: app/refdata/models/energy_systems/regional_energy_system_model.py, строки 36–39.

```
  36      union_energy_system = db.relationship(
  37          'UnionEnergySystem',
  38          back_populates='regional_energy_systems'
  39      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/regional_energy_system_routes.py, строки 44–59.

```
  44  @refdata_bp.route("/regional_energy_system", methods=["GET", "POST"])
  45  @login_required
  46  def regional_energy_system_list():
  47      """Маршрут для отображения списка региональных энергосистем."""
  48
  49      user = session.get('username', 'Неизвестный пользователь')
  50      log_to_db(
  51          user,
  52          "Открыта страница региональных энергосистем",
  53          entity_type="regional_energy_system")
  54
  55      # Создание формы
  56      form = RegionalEnergySystemFilterForm()
  57
  58      # Получение параметров запроса
  59      page                            = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/regional_energy_system_routes.py, строки 163–168.

```
 163                              union_energy_system_filter=union_energy_system_filter,
 164                      ))
 165                  update_regional_energy_system_all_versions_service(regional_energy_system_data,
user)
 166                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 167              else:
 168                  update_regional_energy_system_service(regional_energy_system_data, user)
```

Сервис списка – выборка и пагинация. Источник:
app/refdata/services/energy_systems/regional_energy_system_services.py, строки 123–143.

```
123  @no_autoflush
124  def get_regional_energy_system_list(
125      page,
126      per_page,
127      sort_by="id",
128      sort_dir="asc",
129      regional_energy_system_filter=None,
130      union_energy_system_filter=None,
131      ):
132      """ Получает список региональных энергосистем с пагинацией, фильтрацией и сортировкой, загружая
связи с субъектами РФ. """
133
134      # Базовый запрос
135      query = regional_energy_system_query(
136          sort_by=sort_by,
137          sort_dir=sort_dir,
138          regional_energy_system_filter=regional_energy_system_filter,
139          union_energy_system_filter=union_energy_system_filter,
140      )
141
142      # Пагинация
143      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник:
app/refdata/services/energy_systems/regional_energy_system_services.py, строки 303–314.

```
303
304      try:
305          # Сохранение изменений в базе данных
306          # Фиксация транзакции (устойчивый коммит)
307          _commit_with_retry()
308
309          if updated_ids:
310              log_to_db(
311                  user,
312                  "Сохранены изменения по региональной энергосистеме",
313                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
314                  entity_type="regional_energy_system")
```

HTML/Jinja2 списка – элементы формы. Источник:
app/templates/refdata/energy_systems/regional_energy_system/regional_energy_system.html, строки 10–23.

```
 10      <h1 class="text-center mb-4">Список региональных энергосистем</h1>
 11
 12      <form method="GET"
 13          action="{{ url_for('refdata_bp.regional_energy_system_list') }}"
 14          class="mb-3"
 15          id="filterForm"
 16          >
 17          <div class="row g-2 align-items-center">
 18              <!-- ЛЕВАЯ СТОРОНА: поиск + кнопки -->
 19              <div class="col-12 col-lg-8">
 20                  <div class="d-flex align-items-center gap-2 flex-wrap flex-lg-nowrap">
 21                      <div class="flex-grow-1" style="min-width: 220px;">
 22                          {{ form.regional_energy_system_filter(
 23                              class="form-control",
```

Взаимодействие с другими модулями. РЭС объединяют энергорайоны и территориальные показатели. Связь РЭС с субъектами является многие-ко-многим: административная и энергетическая иерархии не обязаны совпадать. Эта связь участвует в отборе и агрегировании данных электропотребления и потребности в мощности.

Подтверждающие ссылки на потребителей модели:
app/energy_balance/models/espp_energy_generation_model.py:80 – "RegionalEnergySystem",
app/energy_consumption/models/energy_systems/regional_energy_system_energy_consumption_parameter_model.py:29 – "RegionalEnergySystem",
app/fuel/models/fue_equipment_group_fuel_param_model.py:21 – from app.refdata.models.energy_systems.regional_energy_system_model import RegionalEnergySystem
app/generation/models/station/station_model.py:96 – 'RegionalEnergySystem',

app/power_demand/models/energy_systems/regional_energy_system_demand_parameter_model.py:42 – "RegionalEnergySystem",

4.6.2. Форма добавления

Адрес: GET, POST /refdata/add_regional_energy_system. Обработчик: add_regional_energy_system().

Назначение: создание записи справочника «Региональные энергосистемы». Вводимые поля формы: name, name_full, name_rp. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – RegionalEnergySystem, модель и ее связи приведены выше в подразделе 4.6.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Региональная энергосистема', validators=[DataRequired(message='Поле «Региональная энергосистема» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_full = StringField('Полное наименование', validators=[DataRequired(message='Поле «Полное наименование» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_rp = StringField('Наименование (в родительном падеже)', validators=[DataRequired(message='Поле «Наименование (в родительном падеже)» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); union_energy_system = SelectField('ОЭС', choices=[], coerce=int, validators=[DataRequired(message='Выберите ОЭС.')]); regional_districts = SelectMultipleField('Субъекты РФ', choices=[], coerce=int, validators=[Optional()], render_kw={'multiple': True}).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта RegionalEnergySystem – основной реквизит. Источник: app/refdata/models/energy_systems/regional_energy_system_model.py, строки 23–23.

```
23      name = db.Column(db.String(255), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/regional_energy_system_routes.py, строки 216–228.

```
216  @refdata_bp.route("/add_regional_energy_system", methods=["GET", "POST"])
217  @login_required
218  def add_regional_energy_system():
219      """ Маршрут для добавления новой региональной энергосистемы. """
220
221      user = session.get('username', 'Неизвестный пользователь')
222      log_to_db(
223          user,
224          "Открыта страница добавления региональной энергосистемы",
225          entity_type="regional_energy_system")
226
227      # Создание формы
228      form = AddRegionalEnergySystemForm()
```

Маршрут добавления – вызов сервиса. Источник:
app/refdata/routes/energy_systems/regional_energy_system_routes.py, строки 265–276.

```
265                             union_energy_system_filter=union_energy_system_filter,
266                         ))
267                     add_regional_energy_system_all_versions_service(payload, user)
268                     flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
269                 else:
270                     add_regional_energy_system_service(payload, user)
271                     flash("Новая запись успешно добавлена.", "success")
272 
273                 # Перенаправление на список с сохранением параметров и переходом к новой записи
274                 total_records = regional_energy_system_query(
275                                     regional_energy_system_filter,
276                                     union_energy_system_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/energy_systems/regional_energy_system_services.py, строки 389–403.

```
389                     raise ValueError(f"Запись с полным наименованием «{name_full}» уже
существует.")
390 
391                 # Создаем новую запись
392                 obj = RegionalEnergySystem(
393                     name=name,
394                     name_full=name_full or None,
395                     name_rp=name_rp or None,
396                     id_union_energy_system=union_energy_system_id,
397                 )
398 
399                 # Обновление связей «многие ко многим»
400                 existing_districts = {district.id for district in obj.regional_districts}
401                 new_districts = set(regional_district_ids)
402 
403                 # Добавить новые связи
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/energy_systems/regional_energy_system/regional_energy_system_add.html, строки 93–107.

```
 93                     <li>
 94                         <label class="dropdown-item">
 95                             <input type="checkbox" name="{{ form.regional_districts.name }}"
value="{{ regional_district[0] }}"
 96                                    data-region-name="{{ regional_district[1] }}"
 97                                    class="region-checkbox"
 98                                    {% if form.regional_districts.data and regional_district[0] in
form.regional_districts.data %}checked{% endif %}>
 99                                 {{ regional_district[1] }}
100                             </label>
101                     </li>
102                     {% endfor %}
103                 </ul>
104             </div>
105 
106             <!-- Ошибки валидации -->
107             {% if form.regional_districts.errors %}
```

Взаимодействие при добавлении. РЭС объединяют энергорайоны и территориальные показатели. Связь РЭС с субъектами является многие-ко-многим: административная и энергетическая иерархии не обязаны совпадать. Эта связь участвует в отборе и агрегировании данных электропотребления и потребности в мощности. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.6.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
POST /refdata/import_regional_energy_system	import_regional_energy_system	Обработка загруженного файла и

Адрес и метод	Обработчик	Результат
		сообщение о результате
GET /refdata/export_regional_energy_system	export_regional_energy_system	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

4.7. Энергорайоны

4.7.1. Страница списка

Адрес: GET, POST /refdata/energy_units. Обработчик: energy_unit_list().

Ведет энергорайоны EnergyUnit, их наименования и грамматические формы, принадлежность к субъекту РФ и региональной энергосистеме. Список позволяет отбирать записи по территориальным связям, редактировать их и переходить к созданию новой записи.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, regional_district_filter, regional_energy_system_filter, union_energy_system_filter, energy_unit_filter, all_versions, energy_unit_ids[], energy_unit_names[], energy_unit_names_rp[], energy_unit_names_dp[], energy_unit_names_dat[], regional_district_ids[], regional_energy_system_ids[], energy_unit_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: EnergyUnit. Поля SQLAlchemy: id, name, name_rp, name_dp, name_dat, id_regional_district, regional_district, id_regional_energy_system, regional_energy_system, created_at, updated_at, database_version_id, stations, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/energy_unit_model.py, строки 13–33.

```
13  class EnergyUnit(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
14      __tablename__ = 'gs_sys_energy_units'
15      __table_args__ = {"schema": SCHEMA_REFDATA}
16
17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
18      name = db.Column(db.String(256), nullable=False, index=True)
19
20      # Полное наименование в родительном падеже
21      name_rp = db.Column(db.String(255), nullable=False)
22      # Полное наименование в предложном падеже
23      name_dp = db.Column(db.String(255), nullable=False)
24      # Наименование в дательном падеже (для «Итого по …»)
25      name_dat = db.Column(db.String(255), nullable=False, server_default="")
26
27      # FK -> RegionalDistrict
28      id_regional_district = db.Column(
29          db.Integer,
30          db.ForeignKey(f'{SCHEMA_REFDATA}.gs_sys_regional_districts.id', ondelete='RESTRICT'),
31          index=True,
32          nullable=False
33      )
```

Связь модели с другой сущностью. Источник: app/refdata/models/energy_systems/energy_unit_model.py, строки 34–37.

```
34      regional_district = db.relationship(
35          'RegionalDistrict',
36          back_populates='energy_units'
37      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/energy_unit_routes.py, строки 47–62.

```
47  @refdata_bp.route("/energy_units", methods=["GET", "POST"])
48  @login_required
49  def energy_unit_list():
50      """Маршрут для отображения списка энергоузлов."""
51
52      user = session.get('username', 'Неизвестный пользователь')
53      log_to_db(
54          user,
55          "Открыта страница энергоузла",
56          entity_type="energy_unit")
57
58      # Создание формы
59      form = EnergyUnitFilterForm()
60
61      # Получение параметров запроса
62      page                            = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/energy_unit_routes.py, строки 184–189.

```
184                             union_energy_system_filter=union_energy_system_filter,
185                         ))
186                 update_energy_unit_all_versions_service(energy_unit_data, user)
187                 flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
188             else:
189                 update_energy_unit_service(energy_unit_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/energy_systems/energy_unit_services.py, строки 191–212.

```
191  @no_autoflush
192  def get_energy_unit_list(
193      page,
194      per_page,
195      energy_unit_filter=None,
196      regional_district_filter=None,
197      regional_energy_system_filter=None,
198      union_energy_system_filter=None,
199      sort_by="id",
200      sort_dir="asc"):
201      """ Получает список энергоузлов с пагинацией, фильтрацией и сортировкой. """
202
203      # Базовый запрос
204      query = energy_unit_query(
205          energy_unit_filter=energy_unit_filter,
206          regional_district_filter=regional_district_filter,
207          regional_energy_system_filter=regional_energy_system_filter,
208          union_energy_system_filter=union_energy_system_filter,
209          sort_by=sort_by,
210          sort_dir=sort_dir,
211      )
212
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/energy_systems/energy_unit_services.py, строки 330–341.

```
330
331     try:
332         # Сохранение изменений в базе данных
333         # Фиксация транзакции (устойчивый коммит)
334         _commit_with_retry()
335
336         if updated_ids:
337             log_to_db(
338                 user,
339                 "Сохранены изменения по энергоузлам",
340                 f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
341                 entity_type="energy_unit")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/energy_systems/energy_unit/energy_unit.html, строки 10–23.

```
10      <h1 class="text-center mb-4">Список энергорайонов</h1>
11
12      <form method="GET"
13          action="{{ url_for('refdata_bp.energy_unit_list') }}"
14          class="mb-3"
15          id="filterForm"
16          >
17          <div class="row g-2 align-items-center">
18              <!-- ЛЕВАЯ СТОРОНА: поиск + кнопки -->
19              <div class="col-12 col-lg-8">
20                  <div class="d-flex align-items-center gap-2 flex-wrap flex-lg-nowrap">
21                      <div class="flex-grow-1" style="min-width: 220px;">
22                          {{ form.energy_unit_filter(
23                              class="form-control",
```

Взаимодействие с другими модулями. Station.id_energy_unit связывает электростанцию с энергорайоном. EnergyUnitDemandParameter хранит связанные показатели потребности в мощности. При группировке станций используются одновременно энергетическая и административная принадлежности. В интерфейсе EnergyUnit называется «Энергорайон», а EnergyArea – «Энергоузел».

Подтверждающие ссылки на потребителей модели:
app/energy_balance/models/regional_energy_system_transfer_model.py:6 – Получатель – RegionalEnergySystem, ForeignBorderCountry, RegionalDistrict или EnergyUnit
app/energy_consumption/models/energy_systems/energy_unit_energy_consumption_parameter_model.py:26 – "EnergyUnit",
app/fuel/models/fue_equipment_group_model.py:133 – energy_unit = db.relationship("EnergyUnit", foreign_keys=[energy_unit_id])
app/generation/models/station/station_model.py:107 – energy_unit = db.relationship('EnergyUnit', back_populates='stations')
app/power_demand/models/energy_systems/energy_unit_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для энергоузла (EnergyUnit)."""

4.7.2. Форма добавления

Адрес: GET, POST /refdata/add_energy_unit. Обработчик: add_energy_unit().

Назначение: создание записи справочника «Энергорайоны». Вводимые поля формы: name, name_rp, name_dp, name_dat. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – EnergyUnit, модель и ее связи приведены выше в подразделе 4.7.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Энергорайон', validators=[DataRequired(message='Поле «Энергорайон» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_rp = StringField('Энергорайон (род. пад.)', validators=[DataRequired(message='Поле «Энергорайон (род. пад.)» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); name_dp = StringField('Энергорайон (предл. пад.)', validators=[DataRequired(message='Поле «Энергорайон (предл. пад.)» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); name_dat = StringField('Наименование (в дательном падеже)', validators=[Optional(), Length(max=255, message='Длина до 255 символов.')]); regional_district = SelectField('Субъект РФ', choices=[], coerce=int, validators=[DataRequired(message='Выберите субъект РФ.')]); regional_energy_system = SelectField('Региональная энергосистема', choices=[], coerce=int, validators=[Optional()]); union_energy_system = SelectField('ОЭС', choices=[], coerce=int, validators=[Optional()]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта EnergyUnit – основной реквизит. Источник: app/refdata/models/energy_systems/energy_unit_model.py, строки 18–18.

```
18      name = db.Column(db.String(256), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/energy_unit_routes.py, строки 252–264.

```
252 @refdata_bp.route("/add_energy_unit", methods=["GET", "POST"])
253 @login_required
254 def add_energy_unit():
255     """ Маршрут для добавления нового энергоузла. """
256
257     user = session.get('username', 'Неизвестный пользователь')
258     log_to_db(
259         user,
260         "Открыта страница добавления энергоузла",
261         entity_type="energy_unit")
262
263     # Создание формы
264     form = AddEnergyUnitForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/energy_systems/energy_unit_routes.py, строки 316–329.

```
316                                 union_energy_system_filter=union_energy_system_filter,
317                         ))
318                 add_energy_unit_all_versions_service(payload, user)
319                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
320             else:
321                 add_energy_unit_service(payload, user)
322                 flash("Новая запись успешно добавлена.", "success")
323
324             # Перенаправление на список с сохранением параметров и переходом к новой записи
325             total_records = energy_unit_query(
326                                 energy_unit_filter,
327                                 regional_district_filter,
328                                 regional_energy_system_filter,
329                                 union_energy_system_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/energy_systems/energy_unit_services.py, строки 411–435.

```
411                     raise ValueError(f"Запись с наименованием «{name}» уже существует.")
412
413                 # Создаем новую запись
414                 obj = EnergyUnit(
415                     name=name,
416                     name_rp=name_rp,
417                     name_dp=name_dp,
418                     name_dat=name_dat,
419                     id_regional_district=rd_obj.id,
420                     id_regional_energy_system=res_obj.id,
421                 )
422                 set_db_version_on_create(obj)
423                 db.session.add(obj)
424                 db.session.flush()  # получить id без полного коммита
425
426                 log_to_db(
427                     user,
428                     "Создана региональная энергосистема",
429                     (
430                         f"Наименование: {name}; "
431                         f"Субъект РФ: {rd_obj.name}; "
432                         f"Региональная энергосистема: {res_obj.name} "
433                     ),
434                     entity_type="energy_unit",
435                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/energy_systems/energy_unit/energy_unit_add.html, строки 7–21.

```
  7  <div class="container mt-5">
  8      <h1 class="mb-4"><i class="fas fa-plus-circle"></i> Добавить энергорайон</h1>
  9
 10      <!-- Форма ввода данных -->
 11      <form action="{{ url_for('refdata_bp.add_energy_unit') }}" method="POST">
 12          {{ form.hidden_tag() }}
 13
 14          <!-- Скрытые поля для сохранения параметров предыдущей страницы -->
 15          <input type="hidden" name="page"                            value="{{ pagination.page if
pagination else 1 }}">
 16          <input type="hidden" name="per_page"                        value="{{ per_page }}">
 17          <input type="hidden" name="sort_by"                         value="{{ sort_by }}">
 18          <input type="hidden" name="sort_dir"                        value="{{ sort_dir }}">
 19          <input type="hidden" name="energy_unit_filter"              value="{{ energy_unit_filter
}}">
 20          <input type="hidden" name="regional_district_filter"        value="{{
regional_district_filter }}">
 21          <input type="hidden" name="regional_energy_system_filter"   value="{{
regional_energy_system_filter }}">
```

Взаимодействие при добавлении. Station.id_energy_unit связывает электростанцию с энергорайоном. EnergyUnitDemandParameter хранит связанные показатели потребности в мощности. При группировке станций используются одновременно энергетическая и административная принадлежности. В интерфейсе EnergyUnit называется «Энергорайон», а EnergyArea – «Энергоузел». После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.7.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_energy_unit	export_energy_unit	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.8. Энергоузлы

4.8.1. Страница списка

Адрес: GET, POST /refdata/energy_areas. Обработчик: energy_area_list().

Ведет энергоузлы EnergyArea с наименованием и привязкой к субъекту РФ. Предусмотрены отбор, редактирование в таблице и отдельная форма добавления; экспорт формирует файл Excel.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, regional_district_filter, regional_energy_system_filter, union_energy_system_filter, energy_area_filter, all_versions, energy_area_ids[], energy_area_names[], regional_district_ids[], energy_area_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: EnergyArea. Поля SQLAlchemy: id, name, id_regional_district, regional_district, created_at, updated_at, database_version_id, machines, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/energy_area_model.py, строки 13–31.

```
  13  class EnergyArea(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_energy_areas'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      name = db.Column(db.String(256), nullable=False, index=True)
  19
  20      # FK -> RegionalDistrict (обязательный)
  21      id_regional_district = db.Column(
  22          db.Integer,
  23          db.ForeignKey(f'{SCHEMA_REFDATA}.gs_sys_regional_districts.id', ondelete='RESTRICT'),
  24          index=True,
  25          nullable=False
  26      )
  27      regional_district = db.relationship('RegionalDistrict', back_populates='energy_areas')
  28
  29      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  30      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  31
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/energy_area_routes.py, строки 45–60.

```
  45  @refdata_bp.route("/energy_areas", methods=["GET", "POST"])
  46  def energy_area_list():
  47      """Маршрут для отображения списка энергорайонов"""
  48
  49      user = session.get('username', 'Неизвестный пользователь')
  50      log_to_db(
  51          user,
  52          "Открыта страница энергорайонов",
  53          entity_type="energy_area")
  54
  55      # Создание формы
  56      form = EnergyAreaFilterForm()
  57
  58      # Получение параметров запроса
  59      page                            = request.args.get("page", 1, type=int)
  60      per_page                        = request.args.get("per_page", 25, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/energy_area_routes.py, строки 164–169.

```
                            union_energy_system_filter=union_energy_system_filter,
                        ))
                update_energy_area_all_versions_service(energy_area_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_energy_area_service(energy_area_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/energy_systems/energy_area_services.py, строки 199–220.

```
@no_autoflush
def get_energy_area_list(
        page,
        per_page,
        energy_area_filter=None,
        regional_district_filter=None,
        regional_energy_system_filter=None,
        union_energy_system_filter=None,
        sort_by="id",
        sort_dir="asc"):
    """ Получает список энергорайонов с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = energy_area_query(
        energy_area_filter=energy_area_filter,
        regional_district_filter=regional_district_filter,
        regional_energy_system_filter=regional_energy_system_filter,
        union_energy_system_filter=union_energy_system_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/energy_systems/energy_area_services.py, строки 311–322.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по энергорайонам",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="energy_area")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/energy_systems/energy_area/energy_area.html, строки 10–23.

```
    <h1 class="text-center mb-4">Список энергоузлов</h1>

    <form method="GET"
        action="{{ url_for('refdata_bp.energy_area_list') }}"
        class="mb-3"
        id="filterForm"
        >
        <div class="row g-2 align-items-center">
            <!-- ЛЕВАЯ СТОРОНА: поиск + кнопки -->
            <div class="col-12 col-lg-8">
                <div class="d-flex align-items-center gap-2 flex-wrap flex-lg-nowrap">
                    <div class="flex-grow-1" style="min-width: 220px;">
                        {{ form.energy_area_filter(
                            class="form-control",
```

Взаимодействие с другими модулями. Machine.id_energy_area связывает агрегат с энергоузлом. EnergyAreaDemandParameter содержит параметры потребности в мощности этого уровня. Энергоузел не следует отождествлять с энергорайоном EnergyUnit: модели и точки подключения различаются.

Подтверждающие ссылки на потребителей модели:
app/energy_consumption/models/energy_systems/energy_area_energy_consumption_parameter_model.py:2 – """Параметры потребления для энергорайона (EnergyArea)."""
app/generation/models/machine/machine_model.py:82 – energy_area = db.relationship('EnergyArea', back_populates='machines')
app/power_demand/models/energy_systems/energy_area_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для энергоузла (EnergyArea)."""
app/energy_consumption/routes/energy_consumption_parameter_routes.py:67 – from app.refdata.models.energy_systems.energy_area_model import EnergyArea
app/generation/services/machine_services/machine_services.py:27 – from app.refdata.models.energy_systems.energy_area_model import EnergyArea

4.8.2. Форма добавления

Адрес: GET, POST /refdata/add_energy_area. Обработчик: add_energy_area().

Назначение: создание записи справочника «Энергоузлы». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – EnergyArea, модель и ее связи приведены выше в подразделе 4.8.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Энергоузел', validators=[DataRequired(message='Поле «Энергоузел» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); regional_district = SelectField('Субъект РФ', choices=[], coerce=int, validators=[DataRequired(message='Выберите субъект РФ.')]); regional_energy_system = SelectField('Региональная энергосистема', choices=[], coerce=int, validators=[Optional()]); union_energy_system = SelectField('ОЭС', choices=[], coerce=int, validators=[Optional()]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта EnergyArea – основной реквизит. Источник: app/refdata/models/energy_systems/energy_area_model.py, строки 18–18.

```
18      name = db.Column(db.String(256), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/energy_area_routes.py, строки 228–240.

```
228 @refdata_bp.route("/add_energy_area", methods=["GET", "POST"])
229 def add_energy_area():
230     """Маршрут для добавления нового энергорайона."""
231
232     user = session.get('username', 'Неизвестный пользователь')
233     log_to_db(
234         user,
235         "Открыта страница добавления энергорайона",
236         entity_type="energy_area")
237
238     # Создание формы
239     form = AddEnergyAreaForm()
240
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/energy_systems/energy_area_routes.py, строки 284–297.

```
284                             union_energy_system_filter=union_energy_system_filter,
285                         ))
286                     add_energy_area_all_versions_service(payload, user)
287                     flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
288                 else:
289                     add_energy_area_service(payload, user)
290                     flash("Новая запись успешно добавлена.", "success")
291 
292                 # Перенаправление на список с сохранением параметров и переходом к новой записи
293                 total_records = energy_area_query(
294                                     energy_area_filter,
295                                     regional_district_filter,
296                                     regional_energy_system_filter,
297                                     union_energy_system_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/energy_systems/energy_area_services.py, строки 382–401.

```
382                     raise ValueError(f"Запись с именем «{name}» уже существует.")
383 
384                 # Создаем новую запись
385                 obj = EnergyArea(
386                     name=name,
387                     id_regional_district=regional_district_id,
388                 )
389                 set_db_version_on_create(obj)
390                 db.session.add(obj)
391                 db.session.flush()  # получить id без полного коммита
392 
393                 log_to_db(
394                     user,
395                     "Создан энергорайон",
396                     (
397                         f"Наименование: {name}; "
398                         f"Субъект РФ: {get_regional_district_name(regional_district_id)}"
399                     ),
400                     entity_type="energy_area",
401                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/energy_systems/energy_area/energy_area_add.html, строки 7–21.

```
  7  <div class="container mt-5">
  8      <h1 class="mb-4"><i class="fas fa-plus-circle"></i> Добавить энергоузел</h1>
  9 
 10      <!-- Форма ввода данных -->
 11      <form action="{{ url_for('refdata_bp.add_energy_area') }}" method="POST">
 12          {{ form.hidden_tag() }}
 13 
 14          <!-- Скрытые поля для сохранения параметров предыдущей страницы -->
 15          <input type="hidden" name="page"                              value="{{ pagination.page if
pagination else 1 }}">
 16          <input type="hidden" name="per_page"                          value="{{ per_page }}">
 17          <input type="hidden" name="sort_by"                           value="{{ sort_by }}">
 18          <input type="hidden" name="sort_dir"                          value="{{ sort_dir }}">
 19          <input type="hidden" name="energy_area_filter"                value="{{ energy_area_filter
}}">
 20          <input type="hidden" name="regional_district_filter"          value="{{
regional_district_filter }}">
 21          <input type="hidden" name="regional_energy_system_filter"     value="{{
regional_energy_system_filter }}">
```

Взаимодействие при добавлении. Machine.id_energy_area связывает агрегат с энергоузлом. EnergyAreaDemandParameter содержит параметры потребности в мощности этого уровня. Энергоузел не следует отождествлять с энергорайоном EnergyUnit: модели и точки подключения различаются. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.8.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_energy_area_to_excel	export_energy_area	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.9. Типы энергосистем

4.9.1. Страница списка

Адрес: GET, POST /refdata/energy_system_type. Обработчик: energy_system_type_list().

Содержит классификатор типов энергосистем. Наименование редактируется в общей таблице; новый тип вводится через отдельную форму. Тип служит признаком классификации ОЭС.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, energy_system_type_filter, all_versions, energy_system_type_ids[], energy_system_type_names[], energy_system_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: EnergySystemType. Поля SQLAlchemy: id, name, created_at, updated_at, database_version_id, union_energy_systems, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/energy_system_type_model.py, строки 14–32.

```
14  class EnergySystemType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
15      __tablename__ = "gs_sys_energy_system_types"
16      __table_args__ = {"schema": SCHEMA_REFDATA}
17
18      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
19      name = db.Column(db.String(255), unique=True, nullable=False, index=True)
20
21      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
22      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
23
24      # Поле для связи с версией БД
25      database_version_id = db.Column(
26          db.Integer,
27          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
28          nullable=True,
29          index=True
30      )
31
32      # двусторонняя связь с UnionEnergySystem
```

Связь модели с другой сущностью. Источник: app/refdata/models/energy_systems/energy_system_type_model.py, строки 33–37.

```
33      union_energy_systems = db.relationship(
34          "UnionEnergySystem",
35          back_populates="energy_system_type",
36          foreign_keys="UnionEnergySystem.id_energy_system_type"
37      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/energy_system_type_routes.py, строки 35–50.

```
@refdata_bp.route("/energy_system_type", methods=["GET", "POST"])
@login_required
def energy_system_type_list():
    """Маршрут для отображения списка типов частей энергосистемы России."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница типов частей энергосистемы России",
        entity_type="energy_system_type")

    # Создание формы
    form = EnergySystemTypeFilterForm()

    # Получение параметров запроса
    page                        = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/energy_system_type_routes.py, строки 135–140.

```
                            sort_dir=sort_dir,
                    ))
                update_energy_system_type_all_versions_service(energy_system_type_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_energy_system_type_service(energy_system_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/energy_systems/energy_system_type_services.py, строки 82–99.

```
@no_autoflush
def get_energy_system_type_list(
    page,
    per_page,
    energy_system_type_filter=None,
    sort_by="id",
    sort_dir="asc"):
    """ Получает список частей энергосистемы России с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = energy_system_type_query(
        energy_system_type_filter=energy_system_type_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/energy_systems/energy_system_type_services.py, строки 173–184.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по частям энергосистемы России",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="energy_system_type")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/energy_systems/energy_system_type/energy_system_type.html, строки 164–177.

```
 164                                <!-- Строки таблицы. Столбец "Тип части энергосистемы России" -->
 165                                <td>
 166                                    <input type="text" name="energy_system_type_names[]" class="form-
control" value="{{ energy_system_type.name|e }}" maxlength="255">
 167                                </td>
 168
 169                                <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
 170                                <td class="text-center align-middle">
 171                                    <input type="checkbox" name="energy_system_type_delete[]" value="{{
energy_system_type.id }}" class="form-check-input" aria-label="Удалить запись {{ energy_system_type.id
}}">
 172                                </td>
 173                            </tr>
 174                        {% else %}
 175                            <tr>
 176                                <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
 177                            </tr>
```

Взаимодействие с другими модулями. UnionEnergySystem.id_energy_system_type ссылается на этот классификатор. EnergySystemTypeDemandParameter связывает тип с параметрами потребности в мощности. Расчетные модули используют тип при отборе и группировке энергетических объектов.

Подтверждающие ссылки на потребителей модели:
app/energy_consumption/models/energy_systems/energy_system_type_energy_consumption_parameter_model.py:26 – "EnergySystemType",
app/power_demand/models/energy_systems/energy_system_type_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для типа энергосистемы (EnergySystemType)."""
app/api/services/gentypes_info_exchange_services.py:94 – from app.refdata.models.energy_systems.energy_system_type_model import EnergySystemType
app/energy_balance/services/ee_generation_hierarchy_control_services.py:264 – EnergySystemType as entity_model,
app/fuel/services/stations/stations_equipment_groups_services.py:3681 – from app.refdata.models.energy_systems.energy_system_type_model import EnergySystemType

4.9.2. Форма добавления

Адрес: GET, POST /refdata/add_energy_system_type. Обработчик: add_energy_system_type().

Назначение: создание записи справочника «Типы энергосистем». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – EnergySystemType, модель и ее связи приведены выше в подразделе 4.9.1.

Поля WTForms и валидаторы: name = StringField('Наименование типа части энергосистемы России', validators=[DataRequired(message='Укажите наименование типа части энергосистемы России.'), Length(max=255)], render_kw={'placeholder': 'Наименование типа части энергосистемы России'}); submit = SubmitField('Сохранить').

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта EnergySystemType – основной реквизит. Источник: app/refdata/models/energy_systems/energy_system_type_model.py, строки 19–19.

```
  19      name = db.Column(db.String(255), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/energy_system_type_routes.py, строки 174–186.

```
 174  @refdata_bp.route("/add_energy_system_type", methods=["GET", "POST"])
 175  @login_required
 176  def add_energy_system_type():
 177      """ Маршрут для добавления новой части энергосистемы России. """
 178
 179      user = session.get('username', 'Неизвестный пользователь')
 180      log_to_db(
 181          user,
 182          "Открыта страница добавления части энергосистемы России",
 183          entity_type="energy_system_type")
 184
 185      # Создание формы
 186      form = AddEnergySystemTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/energy_systems/energy_system_type_routes.py, строки 219–229.

```
 219                                 sort_dir=sort_dir,
 220                         ))
 221                 add_energy_system_type_all_versions_service(payload, user)
 222                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
 223             else:
 224                 add_energy_system_type_service(payload, user)
 225                 flash("Новая запись успешно добавлена.", "success")
 226
 227             # Перенаправление на список с сохранением параметров и переходом к новой записи
 228             total_records = energy_system_type_query(
 229                                 energy_system_type_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/energy_systems/energy_system_type_services.py, строки 238–253.

```
 238                     raise ValueError(f"Запись с именем «{name}» уже существует.")
 239
 240                 # Создаем новую запись
 241                 obj = EnergySystemType(
 242                     name=name,
 243                 )
 244                 set_db_version_on_create(obj)
 245                 db.session.add(obj)
 246                 db.session.flush()  # получить id без полного коммита
 247
 248                 log_to_db(
 249                     user,
 250                     "Создана часть энергосистемы России",
 251                     (f"Наименование: {name};"),
 252                     entity_type="energy_system_type",
 253                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/energy_systems/energy_system_type/energy_system_type_add.html, строки 22–36.

```
  22          <div class="mb-3">
  23              <label for="name" class="form-label">Наименование типа энергосистемы</label>
  24              {{ form.name(class="form-control", id="name") }}
  25          </div>
  26
  27          <!-- Кнопки сохранения записи и возвращения назад -->
  28          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
  29              <button type="submit" class="btn btn-primary">Добавить</button>
  30              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
 class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
 версиях</button>{% endif %}
  31
  32              <a href="{{ url_for('refdata_bp.energy_system_type_list',
  33                                  page=page,
  34                                  per_page=per_page,
  35                                  sort_by=sort_by,
  36                                  sort_dir=sort_dir,
```

Взаимодействие при добавлении. UnionEnergySystem.id_energy_system_type ссылается на этот классификатор. EnergySystemTypeDemandParameter связывает тип с параметрами потребности в мощности. Расчетные модули используют тип при отборе и группировке энергетических объектов. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.9.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_energy_system_type	export_energy_system_type	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.10. Синхронные зоны

4.10.1. Страница списка

Адрес: GET, POST /refdata/synchronous_area. Обработчик: synchronous_area_list().

Ведет номер, краткое и полное наименования синхронной зоны и порядок ее отображения. Список позволяет искать, сортировать и изменять записи; отдельная форма добавляет зону.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, synchronous_area_filter, all_versions, synchronous_area_ids[], synchronous_area_numbers[], display_orders[], synchronous_area_names[], synchronous_area_full_names[], synchronous_area_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: SynchronousArea. Поля SQLAlchemy: id, display_order, number, name, name_full, created_at, updated_at, database_version_id, regional_districts, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/synchronous_area_model.py, строки 13–33.

```
class SynchronousArea(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_synchronous_areas'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    # Порядок отображения в справочнике
    display_order = db.Column(db.Integer, nullable=True)
    number = db.Column(db.String(256), unique=True, nullable=True, index=True)
    name = db.Column(db.String(256), unique=True, nullable=False, index=True)
    name_full = db.Column(db.String(256), nullable=True, index=True)

    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)

    # Поле для связи с версией БД
    database_version_id = db.Column(
        db.Integer,
        db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
        nullable=True,
        index=True
    )
```

Связь модели с другой сущностью. Источник: app/refdata/models/energy_systems/synchronous_area_model.py, строки 36–40.

```
    regional_districts = db.relationship(
        'RegionalDistrict',
        back_populates='synchronous_area',
        foreign_keys='RegionalDistrict.id_synchronous_area',
    )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/synchronous_area_routes.py, строки 35–50.

```
@refdata_bp.route("/synchronous_area", methods=["GET", "POST"])
@login_required
def synchronous_area_list():
    """Маршрут для отображения списка синхронных зон."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница синхронных зон",
        entity_type="synchronous_area")

    # Создание формы
    form = SynchronousAreaFilterForm()

    # Получение параметров запроса
    page                        = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/synchronous_area_routes.py, строки 167–172.

```
                            sort_dir=sort_dir,
                        ))
                update_synchronous_area_all_versions_service(synchronous_area_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_synchronous_area_service(synchronous_area_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/energy_systems/synchronous_area_services.py, строки 97–114.

```
 97  @no_autoflush
 98  def get_synchronous_area_list(
 99      page,
100      per_page,
101      synchronous_area_filter=None,
102      sort_by="display_order",
103      sort_dir="asc"):
104      """ Получает список синхронных зон с пагинацией, фильтрацией и сортировкой."""
105
106      # Базовый запрос
107      query = synchronous_area_query(
108          synchronous_area_filter=synchronous_area_filter,
109          sort_by=sort_by,
110          sort_dir=sort_dir,
111      )
112
113      # Пагинация
114      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/energy_systems/synchronous_area_services.py, строки 236–247.

```
236
237      try:
238          # Сохранение изменений в базе данных
239          # Фиксация транзакции (устойчивый коммит)
240          _commit_with_retry()
241
242          if updated_ids:
243              log_to_db(
244                  user,
245                  "Сохранены изменения по синхронным зонам",
246                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
247                  entity_type="synchronous_area")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/energy_systems/synchronous_area/synchronous_area.html, строки 202–215.

```
202                                  <!-- Строки таблицы. Столбец "Наименование синхронной зоны" -->
203                                  <td>
204                                      <input type="text" name="synchronous_area_names[]" class="form-
control" value="{{ synchronous_area.name }}">
205                                  </td>
206
207                                  <!-- Строки таблицы. Столбец "Полное наименование синхронной зоны"
-->
208                                  <td>
209                                      <input type="text" name="synchronous_area_full_names[]"
class="form-control" value="{{ synchronous_area.name_full or '' }}">
210                                  </td>
211
212
213                                  <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
214                                  <td class="text-center align-middle">
215                                      <input type="checkbox" name="synchronous_area_delete[]"
value="{{ synchronous_area.id }}" class="form-check-input">
```

Взаимодействие с другими модулями. RegionalDistrict.id_synchronous_area определяет принадлежность субъекта к синхронной зоне. SynchronousAreaDemandParameter связывает зону с показателями потребности в мощности; территориальная агрегация опирается на соответствующие внешние ключи.

Подтверждающие ссылки на потребителей модели:
app/energy_consumption/models/energy_systems/synchronous_area_energy_consumption_parameter_model.py:26 – "SynchronousArea",
app/power_demand/models/energy_systems/synchronous_area_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для синхронной зоны (SynchronousArea)."""
app/api/services/gentypes_info_exchange_services.py:99 – from app.refdata.models.energy_systems.synchronous_area_model import SynchronousArea

app/energy_balance/services/ee_generation_hierarchy_control_services.py:268 – SynchronousArea as entity_model, app/generation/services/station_changes_services/station_changes_services.py:1705 – from app.refdata.models.energy_systems.synchronous_area_model import SynchronousArea

4.10.2. Форма добавления

Адрес: GET, POST /refdata/add_synchronous_area. Обработчик: add_synchronous_area().

Назначение: создание записи справочника «Синхронные зоны». Вводимые поля формы: number, name, name_full. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – SynchronousArea, модель и ее связи приведены выше в подразделе 4.10.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); number = StringField('Номер синхронной зоны', validators=[DataRequired(message='Поле «Номер синхронной зоны» обязательно.'), Length(min=1, max=255, message='Длина от 1 до 255 символов.')]); name = StringField('Наименование синхронной зоны', validators=[Optional(), Length(max=100)], render_kw={'placeholder': 'Введите наименование синхронной зоны'}); name_full = StringField('Полное наименование синхронной зоны', validators=[Optional(), Length(max=256)], render_kw={'placeholder': 'Введите полное наименование синхронной зоны'}).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта SynchronousArea – основной реквизит. Источник: app/refdata/models/energy_systems/synchronous_area_model.py, строки 21–21.

```
21      name = db.Column(db.String(256), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/synchronous_area_routes.py, строки 206–218.

```
206 @refdata_bp.route("/add_synchronous_area", methods=["GET", "POST"])
207 @login_required
208 def add_synchronous_area():
209     """ Маршрут для добавления новой синхронной зоны. """
210
211     user = session.get('username', 'Неизвестный пользователь')
212     log_to_db(
213         user,
214         "Открыта страница добавления синхронной зоны",
215         entity_type="synchronous_area")
216
217     # Создание формы
218     form = AddSynchronousAreaForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/energy_systems/synchronous_area_routes.py, строки 254–264.

```
                            sort_dir=sort_dir,
                        ))
                add_synchronous_area_all_versions_service(payload, user)
                flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
            else:
                add_synchronous_area_service(payload, user)
                flash("Новая запись успешно добавлена.", "success")

             # Перенаправление на список с сохранением параметров и переходом к новой записи
            total_records = synchronous_area_query(
                                synchronous_area_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/energy_systems/synchronous_area_services.py, строки 334–356.

```
                        )

                # Создаем новую запись
                obj = SynchronousArea(
                    name=name,
                    name_full=name_full,
                    number=number or None,
                    display_order=display_order,
                )
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()  # получить id без полного коммита

                log_to_db(
                    user,
                    "Создана синхронная зона",
                    (
                        f"Номер: {_dash(number)}; "
                        f"Наименование: {name}; "
                        f"Полное наименование: {_dash(name_full)}; "
                    ),
                    entity_type="synchronous_area",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/energy_systems/synchronous_area/synchronous_area_add.html, строки 22–36.

```
        <div class="mb-3">
            <label for="name" class="form-label">Номер синхронной зоны</label>
            {{ form.number(class="form-control", id="number") }}

            <label for="name" class="form-label">Наименование синхронной зоны</label>
            {{ form.name(class="form-control", id="name") }}

            <label for="name_full" class="form-label">Полное наименование синхронной зоны</label>
            {{ form.name_full(class="form-control", id="name_full") }}

        </div>

        <!-- Кнопки сохранения записи и возвращения назад -->
        <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
            <button type="submit" class="btn btn-primary">Добавить</button>
```

Взаимодействие при добавлении. RegionalDistrict.id_synchronous_area определяет принадлежность субъекта к синхронной зоне. SynchronousAreaDemandParameter связывает зону с показателями потребности в мощности; территориальная агрегация опирается на соответствующие внешние ключи. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.10.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат

Адрес и метод	Обработчик	Результат
GET /refdata/export_synchronous_area	export_synchronous_area	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.11. Энергозоны

4.11.1. Страница списка

Адрес: GET, POST /refdata/energy_zone. Обработчик: energy_zone_list().

Ведет номера и наименования энергозон. В таблице выполняется поиск и изменение записей, отдельная форма создает новую зону. Модель содержит ограничения уникальности номера и наименования внутри версии БД.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, energy_zone_filter, all_versions, energy_zone_ids[], energy_zone_numbers[], energy_zone_names[], energy_zone_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: EnergyZone. Поля SQLAlchemy: id, number, name, created_at, updated_at, database_version_id, demand_parameters, regional_districts.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/energy_zone_model.py, строки 13–31.

```
 13  class EnergyZone(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
 14      __tablename__ = 'gs_sys_energy_zones'
 15      __table_args__ = {"schema": SCHEMA_REFDATA}
 16
 17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
 18      number = db.Column(db.String(256), nullable=False, index=True)
 19      name = db.Column(db.String(256), nullable=False, index=True)
 20
 21      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
 22      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
 23
 24      # Поле для связи с версией БД
 25      database_version_id = db.Column(
 26          db.Integer,
 27          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
 28          nullable=True,
 29          index=True
 30      )
 31
```

Связь модели с другой сущностью. Источник: app/refdata/models/energy_systems/energy_zone_model.py, строки 38–42.

```
 38      demand_parameters = db.relationship(
 39          "EnergyZoneDemandParameter",
 40          back_populates="energy_zone",
 41          cascade="all, delete-orphan",
 42      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/energy_zone_routes.py, строки 33–48.

```
@refdata_bp.route("/energy_zone", methods=["GET", "POST"])
@login_required
def energy_zone_list():
    """Маршрут для отображения списка энергозон."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница: энергозоны",
        entity_type="energy_zone")

    # Создание формы
    form = EnergyZoneFilterForm()

    # Получение параметров запроса
    page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/energy_zone_routes.py, строки 136–141.

```
                            sort_dir=sort_dir,
                    ))
                update_energy_zone_all_versions_service(energy_zone_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_energy_zone_service(energy_zone_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/energy_systems/energy_zone_services.py, строки 99–116.

```
@no_autoflush
def get_energy_zone_list(
    page,
    per_page,
    energy_zone_filter=None,
    sort_by="id",
    sort_dir="asc"):
    """ Получает список энергозон с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = energy_zone_query(
        energy_zone_filter=energy_zone_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/energy_systems/energy_zone_services.py, строки 199–210.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по энергозонам",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="energy_zone")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/energy_systems/energy_zone/energy_zone.html, строки 169–182.

```
 169                                    <!-- Строки таблицы. Столбец "Номер энергозоны" -->
 170                                    <td class="text-center align-middle">
 171                                        <input type="text" name="energy_zone_numbers[]" class="form-
control" value="{{ energy_zone.number }}">
 172                                    </td>
 173
 174                                    <!-- Строки таблицы. Столбец "Наименование энергозоны" -->
 175                                    <td>
 176                                        <input type="text" name="energy_zone_names[]" class="form-
control" value="{{ energy_zone.name }}">
 177                                    </td>
 178
 179
 180                                    <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
 181                                    <td class="text-center align-middle">
 182                                        <input type="checkbox" name="energy_zone_delete[]" value="{{
energy_zone.id }}" class="form-check-input">
```

Взаимодействие с другими модулями. RegionalDistrict.id_energy_zone связывает субъекты РФ с энергозонами; EnergyZoneDemandParameter хранит показатели потребности в мощности по этим зонам. Согласованность территориальных агрегатов обеспечивается общими сервисами и сервисом субъектов РФ, а не копированием названий в расчетные таблицы.

Подтверждающие ссылки на потребителей модели:
app/energy_consumption/models/energy_systems/energy_zone_energy_consumption_parameter_model.py:26 – "EnergyZone",
app/power_demand/models/energy_systems/energy_zone_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для энергозоны (EnergyZone)."""
app/rational_structure/models/territory_technology_model.py:18 – energy_zone = db.relationship("EnergyZone")
app/fuel/routes/fuel_routes.py:394 – from app.refdata.models.energy_systems.energy_zone_model import EnergyZone
app/generation/services/station_changes_services/station_changes_services.py:16 – from app.refdata.models.energy_systems.energy_zone_model import EnergyZone

4.11.2. Форма добавления

Адрес: GET, POST /refdata/add_energy_zone. Обработчик: add_energy_zone().

Назначение: создание записи справочника «Энергозоны». Вводимые поля формы: number, name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – EnergyZone, модель и ее связи приведены выше в подразделе 4.11.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); number = StringField('Номер энергозоны', validators=[DataRequired(message='Поле «Номер энергозоны» обязательно.'), Length(min=1, max=255, message='Длина от 1 до 255 символов.')]); name = StringField('Наименование энергозоны', validators=[Optional(), Length(max=100)], render_kw={'placeholder': 'Введите наименование энергозоны'}).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта EnergyZone – основной реквизит. Источник: app/refdata/models/energy_systems/energy_zone_model.py, строки 19–19.

```
19      name = db.Column(db.String(256), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/energy_zone_routes.py, строки 174–186.

```
174 @refdata_bp.route("/add_energy_zone", methods=["GET", "POST"])
175 @login_required
176 def add_energy_zone():
177     """ Маршрут для добавления новой энергозоны». """
178 
179     user = session.get('username', 'Неизвестный пользователь')
180     log_to_db(
181         user,
182         "Открыта страница добавления энергозоны",
183         entity_type="energy_zone")
184 
185     # Создание формы
186     form = AddEnergyZoneForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/energy_systems/energy_zone_routes.py, строки 219–229.

```
219                               sort_dir=sort_dir,
220                     ))
221                 add_energy_zone_all_versions_service(payload, user)
222                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
223             else:
224                 add_energy_zone_service(payload, user)
225                 flash("Новая запись успешно добавлена.", "success")
226 
227             # Перенаправление на список с сохранением параметров и переходом к новой записи
228             total_records = energy_zone_query(
229                                 energy_zone_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/energy_systems/energy_zone_services.py, строки 272–291.

```
272                     raise ValueError(f"Запись с полным наименованием «{number}» уже существует.")
273 
274                 # Создаем новую запись
275                 obj = EnergyZone(
276                     name=name,
277                     number=number or None,
278                 )
279                 set_db_version_on_create(obj)
280                 db.session.add(obj)
281                 db.session.flush()  # получить id без полного коммита
282 
283                 log_to_db(
284                     user,
285                     "Создана энергозона",
286                     (
287                         f"Номер: {_dash(number)};"
288                         f"Наименование: {name}. "
289                     ),
290                     entity_type="energy_zone",
291                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/energy_systems/energy_zone/energy_zone_add.html, строки 22–36.

```
  22          <div class="mb-3">
  23              <label for="name" class="form-label">Номер энергозоны</label>
  24              {{ form.number(class="form-control", id="number") }}
  25
  26              <label for="name" class="form-label">Наименование энергозоны</label>
  27              {{ form.name(class="form-control", id="name") }}
  28          </div>
  29
  30          <!-- Кнопки сохранения записи и возвращения назад -->
  31          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
  32              <button type="submit" class="btn btn-primary">Добавить</button>
  33              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
  34
  35              <a href="{{ url_for('refdata_bp.energy_zone_list',
  36                                  page=page,
```

Взаимодействие при добавлении. RegionalDistrict.id_energy_zone связывает субъекты РФ с энергозонами; EnergyZoneDemandParameter хранит показатели потребности в мощности по этим зонам. Согласованность территориальных агрегатов обеспечивается общими сервисами и сервисом субъектов РФ, а не копированием названий в расчетные таблицы. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.11.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_energy_zone	export_energy_zone	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.12. Генерирующие компании

4.12.1. Страница списка

Адрес: GET, POST /refdata/gen_company. Обработчик: gen_company_list().

Ведет полное и сокращенное наименования генерирующих компаний. Реализованы табличное редактирование, создание записи, импорт и экспорт Excel.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, gen_company_filter, all_versions, gen_company_ids[], gen_company_names[], gen_company_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: GenCompany. Поля SQLAlchemy: id, name, name_short, created_at, updated_at, database_version_id, machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/gen_companies/gen_company_model.py, строки 13–31.

```
class GenCompany(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_companies'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    name = db.Column(db.String(80), unique=True, nullable=False, index=True)
    name_short = db.Column(db.String(80), unique=False, nullable=True, index=True)

    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)

    # Поле для связи с версией БД
    database_version_id = db.Column(
        db.Integer,
        db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
        nullable=True,
        index=True
    )

```

Связь модели с другой сущностью. Источник: app/refdata/models/gen_companies/gen_company_model.py, строки 33–33.

```
    machines = db.relationship('Machine', back_populates='gen_company')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/gen_companies/gen_company_routes.py, строки 36–51.

```
@refdata_bp.route("/gen_company", methods=["GET", "POST"])
@login_required
def gen_company_list():
    """Маршрут для отображения генерирующих компаний"""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница генерирующих компаний",
        entity_type="gen_company")

    # Создание формы
    form = GenCompanyFilterForm()

    # Получение параметров запроса
    page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/gen_companies/gen_company_routes.py, строки 127–132.

```
                               sort_dir=sort_dir,
                      ))
                update_gen_company_all_versions_service(gen_company_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_gen_company_service(gen_company_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/gen_companies/gen_company_services.py, строки 90–107.

```
 90  @no_autoflush
 91  def get_gen_company_list(
 92      page,
 93      per_page,
 94      gen_company_filter=None,
 95      sort_by="id",
 96      sort_dir="asc"):
 97      """ Получает список генерирующих компаний с пагинацией, фильтрацией и сортировкой. """
 98
 99      # Базовый запрос
100      query = gen_company_query(
101          gen_company_filter=gen_company_filter,
102          sort_by=sort_by,
103          sort_dir=sort_dir,
104      )
105
106      # Пагинация
107      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/gen_companies/gen_company_services.py, строки 176–187.

```
176
177      try:
178          # Сохранение изменений в базе данных
179          # Фиксация транзакции (устойчивый коммит)
180          _commit_with_retry()
181
182          if updated_ids:
183              log_to_db(
184                  user,
185                  "Сохранены изменения по генерирующим компаниям",
186                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
187                  entity_type="gen_company")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/gen_companies/gen_company.html, строки 159–172.

```
159                                  <!-- Строки таблицы. Столбец "Генерирующая копмания" -->
160                                  <td>
161                                      <input type="text" name="gen_company_names[]" class="form-
control" value="{{ gen_company.name }}">
162                                  </td>
163
164                                  <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
165                                  <td class="text-center align-middle">
166                                      <input type="checkbox" name="gen_company_delete[]" value="{{
gen_company.id }}" class="form-check-input">
167                                  </td>
168                              </tr>
169                          {% else %}
170                              <tr>
171                                  <td colspan="4" class="text-center align-middle">Нет данных для
отображения</td>
172                              </tr>
```

Взаимодействие с другими модулями. Machine.id_gen_company связывает агрегат с компанией. Данные используются в разделе генерирующих мощностей для представления владельца и группировки оборудования; изменение справочника меняет отображаемое наименование компании при чтении связанной модели.

Подтверждающие ссылки на потребителей модели:
app/fuel/models/external_mapping/fue_em_gen_company_branch_model.py:6 – Поле `name` используется для поиска и сопоставления с GenCompany.name.
app/generation/models/machine/machine_model.py:64 – gen_company = db.relationship('GenCompany', back_populates='machines')
app/rational_structure/services/technical_characteristic_machine_services.py:12 – from app.refdata.models.gen_companies.gen_company_model import GenCompany
app/fuel/routes/fuel_routes.py:385 – from app.refdata.models.gen_companies.gen_company_model import GenCompany

app/fuel/services/fuel_imports/import_fuel_db_equipment_groups_services.py:26 – from app.refdata.models.gen_companies.gen_company_model import GenCompany

4.12.2. Форма добавления

Адрес: GET, POST /refdata/add_gen_company. Обработчик: add_gen_company().

Назначение: создание записи справочника «Генерирующие компании». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – GenCompany, модель и ее связи приведены выше в подразделе 4.12.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Наименование генерирующей компании', validators=[DataRequired(message='Поле «Наименование генерирующей компании» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта GenCompany – основной реквизит. Источник: app/refdata/models/gen_companies/gen_company_model.py, строки 18–18.

```
18      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/gen_companies/gen_company_routes.py, строки 166–178.

```
166 @refdata_bp.route("/add_gen_company", methods=["GET", "POST"])
167 @login_required
168 def add_gen_company():
169     """ Маршрут для добавления новой генерирующей компании. """
170
171     user = session.get('username', 'Неизвестный пользователь')
172     log_to_db(
173         user,
174         "Открыта страница добавления генерирующей компании",
175         entity_type="gen_company")
176
177     # Создание формы
178     form = AddGenCompanyForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/gen_companies/gen_company_routes.py, строки 212–222.

```
212                                 sort_dir=sort_dir,
213                     ))
214                 add_gen_company_all_versions_service(payload, user)
215                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
216             else:
217                 add_gen_company_service(payload, user)
218                 flash("Новая запись успешно добавлена.", "success")
219
220             # Перенаправление на список с сохранением параметров и переходом к новой записи
221             total_records = gen_company_query(
222                                     gen_company_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/gen_companies/gen_company_services.py, строки 244–259.

```
244                     raise ValueError(f"Запись с наименованием «{name}» уже существует.")
245
246                 # Создаем новую запись
247                 obj = GenCompany(
248                     name=name,
249                 )
250                 set_db_version_on_create(obj)
251                 db.session.add(obj)
252                 db.session.flush()  # получить id без полного коммита
253
254                 log_to_db(
255                     user,
256                     "Создана генерирующая компания",
257                     f"Наименование: {name}.",
258                     entity_type="gen_company",
259                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/gen_companies/gen_company_add.html, строки 22–36.

```
22          <div class="mb-3">
23              <label for="name" class="form-label">Наименование генерирующей компании</label>
24              {{ form.name(class="form-control", id="name") }}
25              {% if form.name.errors %}
26                  <div class="text-danger">
27                      {{ form.name.errors|join(', ') }}
28                  </div>
29              {% endif %}
30          </div>
31
32          <!-- Кнопки сохранения записи и возвращения назад -->
33          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
34              <button type="submit" class="btn btn-primary">Добавить</button>
35              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
36
```

Взаимодействие при добавлении. Machine.id_gen_company связывает агрегат с компанией. Данные используются в разделе генерирующих мощностей для представления владельца и группировки оборудования; изменение справочника меняет отображаемое наименование компании при чтении связанной модели. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.12.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
POST /refdata/import_gen_company	import_gen_company	Обработка загруженного файла и сообщение о результате
GET /refdata/export_gen_company	export_gen_company	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

4.13. Типы топлива

4.13.1. Страница списка

Адрес: GET, POST /refdata/fuel_type. Обработчик: fuel_type_list().

Хранит тип топлива, порядок отображения и внешнее наименование nazvl. Предусмотрены редактирование, добавление, импорт и экспорт. Тип топлива является родительским классификатором для видов топлива.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, nazvl_filter, fuel_type_filter, all_versions, fuel_ids[], display_orders[], fuel_type_names[], fuel_type_nazvl[], fuel_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: FuelType. Поля SQLAlchemy: id, display_order, name, nazvl, created_at, updated_at, database_version_id, fuels.

Модель данных – фрагмент объявления. Источник: app/refdata/models/fuels/fuel_type_model.py, строки 13–34.

```
class FuelType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_fuel_types'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    # Порядок отображения в справочнике
    display_order = db.Column(db.Integer, nullable=True)
    name = db.Column(db.String(80), unique=True, nullable=False, index=True)

    # Название типа топлива из базы Топливо
    nazvl = db.Column(db.String(80), nullable=True)

    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)

    # Поле для связи с версией БД
    database_version_id = db.Column(
        db.Integer,
        db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
        nullable=True,
        index=True
    )
```

Связь модели с другой сущностью. Источник: app/refdata/models/fuels/fuel_type_model.py, строки 37–37.

```
    fuels = db.relationship('Fuel', back_populates='fuel_type')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/fuels/fuel_type_routes.py, строки 37–52.

```
@refdata_bp.route("/fuel_type", methods=["GET", "POST"])
@login_required
def fuel_type_list():
    """Маршрут для отображения списка видов топлива."""

    def _normalize_filter(value):
        if value in (None, "", "None"):
            return None
        return value

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(user, "Открыта страница видов топлива", entity_type="fuel_type")

    # Создание формы
    form = FuelTypeFilterForm()

```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/fuels/fuel_type_routes.py, строки 151–156.

```
                            sort_dir=sort_dir,
                    ))
                update_fuel_type_all_versions_service(fuel_type_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_fuel_type_service(fuel_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/fuels/fuel_type_services.py, строки 110–129.

```
@no_autoflush
def get_fuel_type_list(
    page,
    per_page,
    fuel_type_filter=None,
    nazvl_filter=None,
    sort_by="display_order",
    sort_dir="asc"):
    """ Получает список видов топлива с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = fuel_type_query(
        fuel_type_filter=fuel_type_filter,
        nazvl_filter=nazvl_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/fuels/fuel_type_services.py, строки 232–244.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()
        _invalidate_fuel_type_caches()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по видам топлива",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="fuel_type")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/fuels/fuel_type/fuel_type.html, строки 261–274.

```
                            <!-- Строки таблицы. Столбец "Тип топлива" -->
                            <td>
                                <input type="text" name="fuel_type_names[]" class="form-control"
value="{{ fuel_type.name }}">
                            </td>

                            <!-- Строки таблицы. Столбец "Короткое наименование" -->
                            <td>
                                <input type="text" name="fuel_type_nazvl[]" class="form-control"
value="{{ fuel_type.nazvl or '' }}">
                            </td>

                            <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
                            <td class="text-center align-middle">
                                <input type="checkbox" name="fuel_type_delete[]" value="{{
fuel_type.id }}" class="form-check-input">
                            </td>
```

Взаимодействие с другими модулями. Fuel.id_fuel_type ссылается на FuelType. Модули топлива, генерации и экологии используют эту классификацию для отбора и сопоставления исходных данных. Поле nazvl служит для согласования обозначений с внешними источниками данных.

Подтверждающие ссылки на потребителей модели:
app/api/services/generation_objects_exchange_services.py:39 – from app.refdata.models.fuels.fuel_type_model import FuelType
app/fuel/refdata/routes/fuel_routes.py:25 – from app.refdata.models.fuels.fuel_type_model import FuelType

app/generation/routes/totals_summary_routes.py:193 – from app.refdata.models.fuels.fuel_type_model import FuelType
app/fuel/refdata/routes/fuel_type_routes.py:21 – from app.refdata.models.fuels.fuel_type_model import FuelType
app/generation/services/station_changes_services/station_changes_services.py:18 – from app.refdata.models.fuels.fuel_type_model import FuelType

4.13.2. Форма добавления

Адрес: GET, POST /refdata/add_fuel_type. Обработчик: add_fuel_type().

Назначение: создание записи справочника «Типы топлива». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – FuelType, модель и ее связи приведены выше в подразделе 4.13.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Вид топлива', validators=[DataRequired(message='Поле Вид топлива» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта FuelType – основной реквизит. Источник: app/refdata/models/fuels/fuel_type_model.py, строки 20–20.

```
20      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/fuels/fuel_type_routes.py, строки 212–224.

```
212  @refdata_bp.route("/add_fuel_type", methods=["GET", "POST"])
213  @login_required
214  def add_fuel_type():
215      """ Маршрут для добавления нового вида топлива. """
216
217      user = session.get('username', 'Неизвестный пользователь')
218      log_to_db(
219          user,
220          "Открыта страница добавления видов топлива",
221          entity_type="fuel_type")
222
223      # Создание формы
224      form = AddFuelTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/fuels/fuel_type_routes.py, строки 260–270.

```
260                                 sort_dir=sort_dir,
261                     ))
262                 add_fuel_type_all_versions_service(payload, user)
263                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
264             else:
265                 add_fuel_type_service(payload, user)
266                 flash("Новая запись успешно добавлена.", "success")
267
268             # Перенаправление на список с сохранением параметров и переходом к новой записи
269             total_records = fuel_type_query(
270                                 fuel_type_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/fuels/fuel_type_services.py, строки 307–320.

```
                                f"Запись с порядком отображения «{display_order}» уже существует."
                            )

                obj = FuelType(name=name, display_order=display_order)
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()

                log_to_db(
                    user,
                    "Создан вид топлива",
                    f"Наименование: {name}",
                    entity_type="fuel_type",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/fuels/fuel_type/fuel_type_add.html, строки 20–34.

```
        <div class="mb-3">
            <label for="name" class="form-label">Тип топлива</label>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>

        <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
            <button type="submit" class="btn btn-primary">Добавить</button>
            {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
            <a href="{{ url_for('refdata_bp.fuel_type_list',
                                page=page,
                                per_page=per_page,
                                sort_by=sort_by,
```

Взаимодействие при добавлении. Fuel.id_fuel_type ссылается на FuelType. Модули топлива, генерации и экологии используют эту классификацию для отбора и сопоставления исходных данных. Поле nazvl служит для согласования обозначений с внешними источниками данных. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.13.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
POST /refdata/import_fuel_type	import_fuel_type	Обработка загруженного файла и сообщение о результате
GET /refdata/export_fuel_type	export_fuel_type	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

4.14. Виды топлива

4.14.1. Страница списка

Адрес: GET, POST /refdata/fuel. Обработчик: fuel_list().

Хранит наименование, код kod, внешнее наименование nazvl, признак угольного топлива kmbur, тип топлива и родительский вид parent_id. Список поддерживает иерархическое представление; служебный GET /fuel/hierarchy

возвращает дерево в JSON. При выборе родителя сервис проверяет допустимость связи, включая недопустимость циклической иерархии.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, fuel_type_filter, nazvl_filter, kmbur_filter, fuel_filter, all_versions, fuel_ids[], fuel_kods[], fuel_names[], fuel_types[], fuel_parents[], fuel_nazvl[], fuel_kmbur[], fuel_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

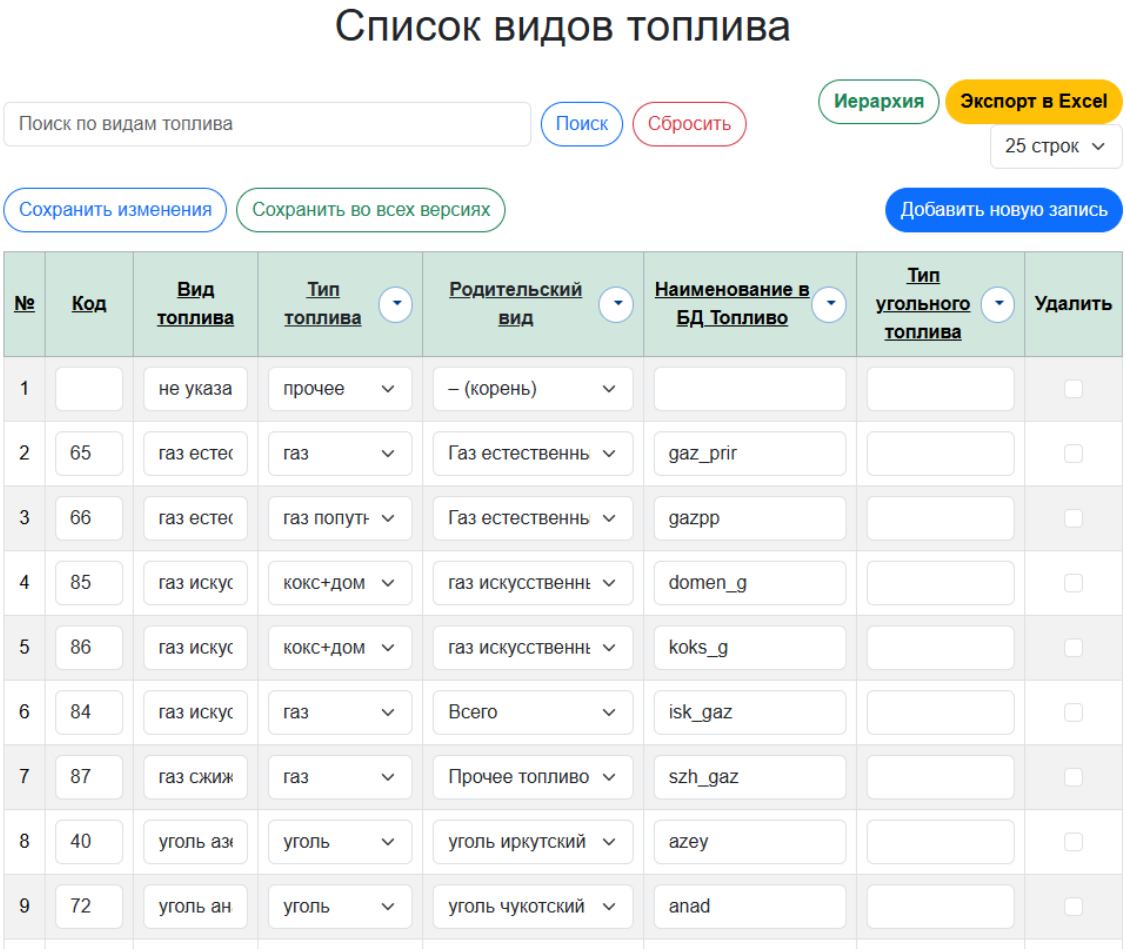


Рисунок 5. Виды топлива. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Модель: Fuel. Поля SQLAlchemy: id, name, kod, nazvl, kmbur, parent_id, parent, id_fuel_type, fuel_type, created_at, updated_at, database_version_id, machine_fuels.

Модель данных – фрагмент объявления. Источник: app/refdata/models/fuels/fuel_model.py, строки 13–35.

```
class Fuel(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_fuels'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    name = db.Column(db.String(80), unique=True, nullable=False, index=True)

    # Код вида топлива из Excel «Состав видов топлива» (столбец Kod)
    kod = db.Column(db.Integer, nullable=True, index=True)

    # Название вида топлива из базы Топливо
    nazvl = db.Column(db.String(80), nullable=True)

    # Название типа угольного топлива из базы Топливо (каменный/бурый)
    kmbur = db.Column(db.String(80), nullable=True)

    # FK -> Fuel (родительский вид в иерархии состава)
    parent_id = db.Column(
        db.Integer,
        db.ForeignKey(f'{SCHEMA_REFDATA}.gs_sys_fuels.id', ondelete='SET NULL'),
        nullable=True,
        index=True,
    )
```

Связь модели с другой сущностью. Источник: app/refdata/models/fuels/fuel_model.py, строки 36–41.

```
    parent = db.relationship(
        'Fuel',
        remote_side=[id],
        backref=db.backref('children', lazy='dynamic'),
        foreign_keys=[parent_id],
    )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/fuels/fuel_routes.py, строки 62–77.

```
@refdata_bp.route("/fuel", methods=["GET", "POST"])
@login_required
def fuel_list():
    """Маршрут для отображения списка типов топлива."""

    def _normalize_filter(value):
        if value in (None, "", "None"):
            return None
        return value

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(user, "Открыта страница типов топлива", entity_type="fuel")

    # Создание формы
    form = FuelFilterForm()

```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/fuels/fuel_routes.py, строки 196–201.

```
                               sort_dir=sort_dir,
                    ))
                update_fuel_all_versions_service(fuel_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_fuel_service(fuel_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/fuels/fuel_services.py, строки 145–167.

```
145  @no_autoflush
146  def get_fuel_list(
147      page,
148      per_page,
149      fuel_filter=None,
150      fuel_type_filter=None,
151      nazvl_filter=None,
152      kmbur_filter=None,
153      sort_by="id",
154      sort_dir="asc",
155      parent_filter=None):
156      """ Получает список типов топлива с пагинацией, фильтрацией и сортировкой. """
157
158      # Базовый запрос
159      query = fuel_query(
160          fuel_filter=fuel_filter,
161          fuel_type_filter=fuel_type_filter,
162          nazvl_filter=nazvl_filter,
163          kmbur_filter=kmbur_filter,
164          parent_filter=parent_filter,
165          sort_by=sort_by,
166          sort_dir=sort_dir,
167      ).options(joinedload(Fuel.fuel_type), joinedload(Fuel.parent))
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/fuels/fuel_services.py, строки 374–385.

```
374
375      try:
376          # Сохранение изменений в базе данных
377          # Фиксация транзакции (устойчивый коммит)
378          _commit_with_retry()
379
380          if updated_ids:
381              log_to_db(
382                  user,
383                  "Сохранены изменения по типам топлива",
384                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
385                  entity_type="fuel")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/fuels/fuel/fuel.html, строки 326–339.

```
326                                          <ul class="dropdown-menu dropdown-menu-end fuel-parent-
filter-menu" aria-labelledby="parentFilterDropdownFuelAdmin">
327                                              <li class="fuel-parent-filter-search-wrap"
onclick="event.stopPropagation()">
328                                                  <input type="text"
329                                                         class="form-control form-control-sm fuel-
parent-filter-search"
330                                                         placeholder="Поиск..."
331                                                         autocomplete="off">
332                                              </li>
333                                              <li>
334                                                  <a href="{{ url_for('refdata_bp.fuel_list',
335                                                              page=pagination.page,
336                                                              fuel_filter=fuel_filter,
337                                                              fuel_type_filter=fuel_type_filter,
338                                                              parent_filter='',
339                                                              nazvl_filter=nazvl_filter,
```

Взаимодействие с другими модулями. MachineFuel связывает виды топлива с агрегатами и годами. Модули топлива и экологии используют единый классификатор, коды и внешние обозначения при импорте, сопоставлении и расчетах. Изменение parent_id меняет структуру состава топлива, но не подменяет исходные значения расхода топлива.

Подтверждающие ссылки на потребителей модели:

app/generation/models/machine/machine_fuel_model.py:4 – - Связи сохранены: year (Year.machine_fuels), machine_fuel (Machine.machine_fuels), fuel (Fuel.machine_fuels).

app/api/services/generation_objects_exchange_services.py:38 – from app.refdata.models.fuels.fuel_model import Fuel

app/energy_balance/services/station_ee_generation_page_services.py:54 – from app.refdata.models.fuels.fuel_model import Fuel
app/fuel/refdata/routes/fuel_routes.py:26 – from app.refdata.models.fuels.fuel_model import Fuel
app/fuel/routes/equipment_group_fuel_batch_ui_routes.py:454 – from app.refdata.models.fuels.fuel_model import Fuel

4.14.2. Форма добавления

Адрес: GET, POST /refdata/add_fuel. Обработчик: add_fuel().

Назначение: создание записи справочника «Виды топлива». Вводимые поля формы: name, kod, fuel_type, parent. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – Fuel, модель и ее связи приведены выше в подразделе 4.14.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип топлива', validators=[DataRequired(message='Поле «Тип топлива» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); kod = StringField('Код', validators=[Optional(), Length(max=20)]); fuel_type = SelectField('Вид топлива', choices=[], coerce=int, validators=[DataRequired(message='Выберите вид топлива.')]); parent = SelectField('Родительский вид', choices=[], coerce=int, validators=[Optional()]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта Fuel – основной реквизит. Источник: app/refdata/models/fuels/fuel_model.py, строки 18–18.

```
18      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/fuels/fuel_routes.py, строки 290–302.

```
290 @refdata_bp.route("/add_fuel", methods=["GET", "POST"])
291 @login_required
292 def add_fuel():
293     """ Маршрут для добавления нового топлива. """
294
295     user = session.get('username', 'Неизвестный пользователь')
296     log_to_db(
297         user,
298         "Открыта страница добавления типов топлива",
299         entity_type="fuel")
300
301     # Создание формы
302     form = AddFuelForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/fuels/fuel_routes.py, строки 341–354.

```
                            sort_dir=sort_dir,
                        ))
                add_fuel_all_versions_service(payload, user)
                flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
            else:
                add_fuel_service(payload, user)
                flash("Новая запись успешно добавлена.", "success")

            # Перенаправление на список с сохранением параметров и переходом к новой записи
            total_records = fuel_query(
                                fuel_filter,
                                fuel_type_filter,
                                nazvl_filter,
                                kmbur_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/fuels/fuel_services.py, строки 449–465.

```
                    raise ValueError(f"Запись с наименованием «{name}» уже существует.")

                # Создаем новую запись
                obj = Fuel(
                    name=name,
                    kod=kod,
                    nazvl=_normalize_fuel_nazvl(record.get("nazvl")),
                    id_fuel_type=fuel_type_id,
                    parent_id=parent_id,
                )
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()  # получить id без полного коммита

                parent_name = (
                    db.session.get(Fuel, parent_id).name if parent_id else "не указано"
                )
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/fuels/fuel/fuel_add.html, строки 21–35.

```
        <div class="mb-3">
            <label for="name" class="form-label">Вид топлива</label>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>


        <div class="mb-3">
            <label for="kod" class="form-label">Код</label>
            {{ form.kod(class="form-control", id="kod") }}
            {% if form.kod.errors %}
                <div class="text-danger">{{ form.kod.errors|join(', ') }}</div>
            {% endif %}
```

Взаимодействие при добавлении. MachineFuel связывает виды топлива с агрегатами и годами. Модули топлива и экологии используют единый классификатор, коды и внешние обозначения при импорте, сопоставлении и расчетах. Изменение parent_id меняет структуру состава топлива, но не подменяет исходные значения расхода топлива. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.14.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/fuel/hierarchy	fuel_hierarchy	JSON-дерево видов топлива
POST /refdata/import_fuel	import_fuel	Обработка загруженного

Адрес и метод	Обработчик	Результат
		файла и сообщение о результате
GET /refdata/export_fuel	export_fuel	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

Кнопка «Иерархия» открывает модальное окно на странице видов топлива, а не отдельную HTML-страницу. Дерево загружается через GET /refdata/fuel/hierarchy. Пользователь может искать по названию, коду и NazvL, раскрывать и сворачивать ветви. Сервис строит дерево текущей версии; parent_id определяет отношение родитель–потомок.

Маршрут JSON-дерева топлива. Источник: app/refdata/routes/fuels/fuel_routes.py, строки 43–59.

```
43  @refdata_bp.route("/fuel/hierarchy", methods=["GET"])
44  @login_required
45  def fuel_hierarchy():
46      """JSON-дерево иерархии видов топлива текущей версии БД."""
47      user = session.get("username", "Неизвестный пользователь")
48      try:
49          tree = get_fuel_hierarchy_tree()
50          log_to_db(
51              user,
52              "Открыта визуализация иерархии видов топлива",
53              f"Узлов: {tree.get('total', 0)}; корней: {tree.get('roots_count', 0)}",
54              entity_type="fuel",
55          )
56          return jsonify(ok=True, **tree)
57      except Exception as e:
58          current_app.logger.exception("Ошибка построения иерархии топлива")
59          return jsonify(ok=False, error=str(e) or "Ошибка построения иерархии."), 500
```

Проверка допустимости родительского вида топлива. Источник: app/refdata/services/fuels/fuel_services.py, строки 121–142.

```
121  def _assert_valid_fuel_parent(fuel_id, parent_id):
122      """Проверяет, что parent_id существует и не создает цикл."""
123      if parent_id is None:
124          return
125
126      parent_obj = db.session.get(Fuel, parent_id)
127      if not parent_obj:
128          raise ValueError(f"Родительский вид топлива с id={parent_id} не найден.")
129
130      if fuel_id is not None and int(parent_id) == int(fuel_id):
131          raise ValueError("Вид топлива не может быть родителем самого себя.")
132
133      seen = {int(fuel_id)} if fuel_id is not None else set()
134      current_id = int(parent_id)
135      while current_id is not None:
136          if current_id in seen:
137              raise ValueError("Обнаружена циклическая ссылка в иерархии видов топлива.")
138          seen.add(current_id)
139          current_obj = db.session.get(Fuel, current_id)
140          if not current_obj:
141              raise ValueError(f"Родительский вид топлива с id={current_id} не найден.")
142          current_id = current_obj.parent_id
```

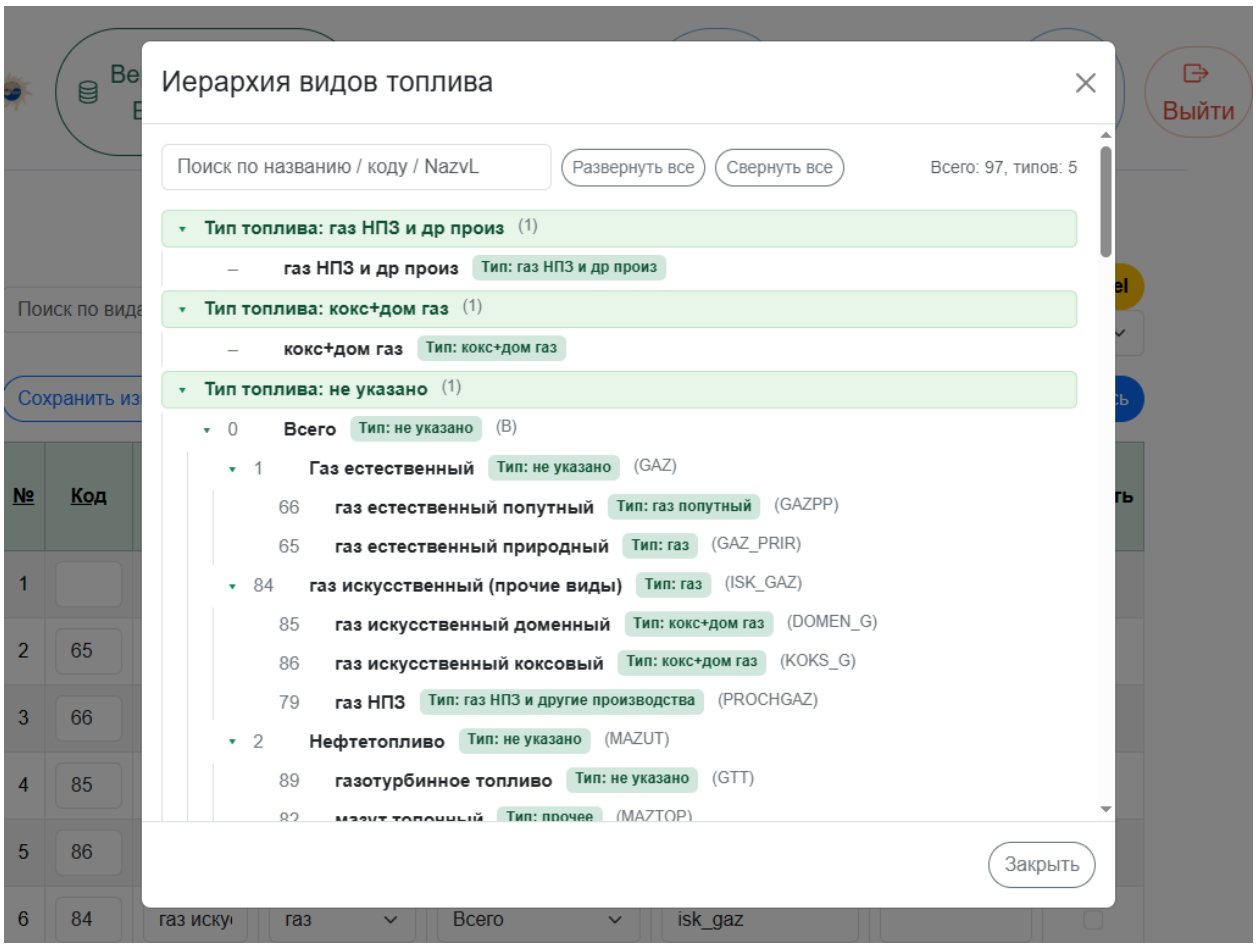


Рисунок 6. Модальное окно иерархии топлива. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

4.15. Типы электростанций

4.15.1. Страница списка

Адрес: GET, POST /refdata/station_type. Обработчик: station_type_list().

Ведет перечень типов электростанций и порядок их отображения. Список позволяет изменять классификационные записи, отдельная форма добавляет новый тип.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, station_type_filter, all_versions, station_ids[], display_orders[], station_type_names[], station_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: StationType. Поля SQLAlchemy: id, display_order, name, created_at, updated_at, database_version_id, stations.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/station/station_type_model.py, строки 13–31.

```
  13  class StationType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_station_types'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      # Порядок отображения в справочнике
  19      display_order = db.Column(db.Integer, nullable=True)
  20      name = db.Column(db.String(255), unique=True, nullable=False, index=True)
  21
  22      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  23      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  24
  25      # Поле для связи с версией БД
  26      database_version_id = db.Column(
  27          db.Integer,
  28          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
  29          nullable=True,
  30          index=True
  31      )
```

Связь модели с другой сущностью. Источник: app/refdata/models/refdata_for_stations/station/station_type_model.py, строки 33–33.

```
  33      stations = db.relationship('Station', back_populates='station_type')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/stations/station_type_routes.py, строки 42–57.

```
  42  @refdata_bp.route("/station_type", methods=["GET", "POST"])
  43  @login_required
  44  def station_type_list():
  45      """Маршрут для отображения списка типов электростанций."""
  46
  47      user = session.get('username', 'Неизвестный пользователь')
  48      log_to_db(
  49          user,
  50          "Открыта страница типов электростанций",
  51          entity_type="station_type")
  52
  53      # Создание формы
  54      form = StationTypeFilterForm()
  55
  56      # Получение параметров запроса
  57      page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/stations/station_type_routes.py, строки 145–150.

```
 145                                sort_dir=sort_dir,
 146                      ))
 147                  update_station_type_all_versions_service(station_type_data, user)
 148                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 149              else:
 150                  update_station_type_service(station_type_data, user)
```

Сервис списка – выборка и пагинация. Источник:
app/refdata/services/refdata_for_stations/stations/station_type_services.py, строки 81–98.

```
  81  @no_autoflush
  82  def get_station_type_list(
  83      page,
  84      per_page,
  85      station_type_filter=None,
  86      sort_by="display_order",
  87      sort_dir="asc"):
  88      """ Получает список типов электростанций с пагинацией, фильтрацией и сортировкой. """
  89
  90      # Базовый запрос
  91      query = station_type_query(
  92          station_type_filter=station_type_filter,
  93          sort_by=sort_by,
  94          sort_dir=sort_dir,
  95      )
  96
  97      # Пагинация
  98      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник:
app/refdata/services/refdata_for_stations/stations/station_type_services.py, строки 187–199.

```
 187
 188      try:
 189          # Сохранение изменений в базе данных
 190          # Фиксация транзакции (устойчивый коммит)
 191          _commit_with_retry()
 192
 193          if updated_ids:
 194              log_to_db(
 195                  user,
 196                  "Сохранены изменения по типам электростанций",
 197                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
 198                  entity_type="station_type",
 199                  entity_id=station_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник:
app/templates/refdata/refdata_for_stations/stations/station_type/station_type.html, строки 192–205.

```
 192                              <!-- Строки таблицы. Столбец "Тип  электростанции" -->
 193                              <td>
 194                                  <input type="text" name="station_type_names[]" class="form-control"
value="{{ station_type.name }}">
 195                              </td>
 196
 197                              <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
 198                              <td class="text-center align-middle">
 199                                  <input type="checkbox" name="station_type_delete[]" value="{{
station_type.id }}" class="form-check-input">
 200                              </td>
 201                          </tr>
 202                          {% else %}
 203                          <tr>
 204                              <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
 205                          </tr>
```

Взаимодействие с другими модулями. Station.id_station_type связывает станцию с типом. Модуль генерации и энергетические балансы используют тип станции для отбора и группировки объектов. Справочник хранит классификацию, а мощности и выработка хранятся в профильных моделях.

Подтверждающие ссылки на потребителей модели:
app/generation/models/station/station_model.py:116 – station_type = db.relationship('StationType', back_populates='stations')
app/energy_balance/services/power_balance_installed_capacity_services.py:886 – from app.refdata.models.refdata_for_stations.station.station_type_model import StationType
app/energy_consumption/services/energy_consumption_summary_services.py:125 – from app.refdata.models.refdata_for_stations.station.station_type_model import StationType

app/fuel/routes/fuel_routes.py:317 – from app.refdata.models.refdata_for_stations.station.station_type_model import StationType
app/rational_structure/services/equipment_runtime_machine_services.py:25 – from app.refdata.models.refdata_for_stations.station.station_type_model import StationType

4.15.2. Форма добавления

Адрес: GET, POST /refdata/add_station_type. Обработчик: add_station_type().

Назначение: создание записи справочника «Типы электростанций». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – StationType, модель и ее связи приведены выше в подразделе 4.15.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип электростанции', validators=[DataRequired(message='Поле «Тип электростанции» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта StationType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/station/station_type_model.py, строки 20–20.

```
20      name = db.Column(db.String(255), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/stations/station_type_routes.py, строки 185–197.

```
185  @refdata_bp.route("/add_station_type", methods=["GET", "POST"])
186  @login_required
187  def add_station_type():
188      """ Маршрут для добавления нового типа  электростанции. """
189
190      user = session.get('username', 'Неизвестный пользователь')
191      log_to_db(
192          user,
193          "Открыта страница добавления типов электростанций",
194          entity_type="station_type")
195
196      # Создание формы
197      form = AddStationTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/stations/station_type_routes.py, строки 231–241.

```
231                                 sort_dir=sort_dir,
232                             ))
233                     add_station_type_all_versions_service(payload, user)
234                     flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
235                 else:
236                     add_station_type_service(payload, user)
237                     flash("Новая запись успешно добавлена.", "success")
238
239                 # Перенаправление на список с сохранением параметров и переходом к новой записи
240                 total_records = station_type_query(
241                                     station_type_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/refdata_for_stations/stations/station_type_services.py, строки 260–273.

```
260                             f"Запись с порядком отображения «{display_order}» уже существует."
261                         )
262
263                 obj = StationType(name=name, display_order=display_order)
264                 set_db_version_on_create(obj)
265                 db.session.add(obj)
266                 db.session.flush()
267
268                 log_to_db(
269                     user,
270                     "Создан тип  электростанции",
271                     f"Наименование: {name}",
272                     entity_type="station_type",
273                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/refdata_for_stations/stations/station_type/station_type_add.html, строки 20–34.

```
20          <div class="mb-3">
21              <label for="name" class="form-label">Тип  электростанции</label>
22              {{ form.name(class="form-control", id="name") }}
23              {% if form.name.errors %}
24                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
25              {% endif %}
26          </div>
27
28          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
29              <button type="submit" class="btn btn-primary">Добавить</button>
30              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
31              <a href="{{ url_for('refdata_bp.station_type_list',
32                                  page=page,
33                                  per_page=per_page,
34                                  sort_by=sort_by,
```

Взаимодействие при добавлении. Station.id_station_type связывает станцию с типом. Модуль генерации и энергетические балансы используют тип станции для отбора и группировки объектов. Справочник хранит классификацию, а мощности и выработка хранятся в профильных моделях. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.15.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_station_type	export_station_type	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.16. Типы состояния оборудования

4.16.1. Страница списка

Адрес: GET, POST /refdata/condition_type. Обработчик: condition_type_list().

Ведет классификатор состояний оборудования. Пользователь просматривает, ищет и редактирует наименования состояний либо добавляет новое состояние в отдельной форме.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, condition_type_filter, all_versions, condition_type_ids[], condition_type_names[], condition_type_delete[].

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: ConditionType. Поля SQLAlchemy: id, name, created_at, updated_at, database_version_id, stations, machines, pgu_machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/condition_type_model.py, строки 13–35.

```
  13  class ConditionType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_condition_types'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
  19
  20      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  22
  23      # Поле для связи с версией БД
  24      database_version_id = db.Column(
  25          db.Integer,
  26          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
  27          nullable=True,
  28          index=True
  29      )
  30
  31      stations = db.relationship(
  32          'Station',
  33          back_populates='condition_type',
  34          primaryjoin="ConditionType.id == Station.id_condition_type"
  35      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/condition_type_routes.py, строки 35–50.

```
  35  @refdata_bp.route("/condition_type", methods=["GET", "POST"])
  36  @login_required
  37  def condition_type_list():
  38      """Маршрут для отображения списка типов состояния."""
  39
  40      user = session.get('username', 'Неизвестный пользователь')
  41      log_to_db(
  42          user,
  43          "Открыта страница типов состояния",
  44          entity_type="condition_type"
  45      )
  46
  47      # Создание формы
  48      form = ConditionTypeFilterForm()
  49
  50      # Получение параметров запроса
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/condition_type_routes.py, строки 124–129.

```
 124                                              sort_by=sort_by,
 125                                              sort_dir=sort_dir))
 126                  update_condition_type_all_versions_service(condition_type_payload, user)
 127                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 128              else:
 129                  update_condition_type_service(condition_type_payload, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/refdata_for_stations/condition_type_services.py, строки 65–81.

```
  65  @no_autoflush
  66  def get_condition_type_list(
  67      page: int,
  68      per_page: int,
  69      condition_type_filter: str | None = None,
  70      sort_by: str = "id",
  71      sort_dir: str = "asc"):
  72      """ Получает список типов состояния с пагинацией."""
  73
  74      # Базовый запрос
  75      query = condition_type_query(
  76          condition_type_filter=condition_type_filter,
  77          sort_by=sort_by,
  78          sort_dir=sort_dir)
  79
  80      # Пагинация
  81      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/refdata_for_stations/condition_type_services.py, строки 149–161.

```
 149
 150      try:
 151          # Сохранение изменений в базе данных
 152          # Фиксация транзакции (устойчивый коммит)
 153          _commit_with_retry()
 154
 155          if updated_ids:
 156              log_to_db(
 157                  user,
 158                  "Сохранены изменения по типам состояния",
 159                  f"Изменено записей: {len(updated_ids)}",
 160                  entity_type="condition_type"
 161                  )
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/refdata_for_stations/condition_type/condition_type.html, строки 158–171.

```
 158                              <!-- Строки таблицы. Столбец "Тип состояния" -->
 159                              <td>
 160                                  <input type="text" name="condition_type_names[]" class="form-
control" value="{{ condition_type.name }}">
 161                              </td>
 162
 163                              <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
 164                              <td class="text-center align-middle">
 165                                  <input type="checkbox" name="condition_type_delete[]" value="{{
condition_type.id }}" class="form-check-input">
 166                              </td>
 167                          </tr>
 168                          {% else %}
 169                          <tr>
 170                              <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
 171                          </tr>
```

Взаимодействие с другими модулями. Классификатор применяется в модуле генерации при хранении состояний оборудования. Он задает значение признака, а не даты ввода, демонтажа или ремонтные интервалы; конкретные ссылки потребителей приведены ниже.

Подтверждающие ссылки на потребителей модели:

app/generation/models/machine/machine_model.py:55 – condition_type = db.relationship('ConditionType', back_populates='machines')

app/generation/models/pgu_machine/pgu_machine_model.py:32 – condition_type = db.relationship('ConditionType', back_populates='pgu_machines')

app/generation/models/station/station_model.py:70 – condition_type = db.relationship('ConditionType', back_populates='stations')

app/generation/routes/stations/station_details_routes.py:38 – from app.refdata.models.refdata_for_stations.condition_type_model import ConditionType

app/generation/services/machine_services/machine_all_versions_services.py:61 – from app.refdata.models.refdata_for_stations.condition_type_model import ConditionType

4.16.2. Форма добавления

Адрес: GET, POST /refdata/add_condition_type. Обработчик: add_condition_type().

Назначение: создание записи справочника «Типы состояния оборудования». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – ConditionType, модель и ее связи приведены выше в подразделе 4.16.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип состояния', validators=[DataRequired(message='Поле «Тип состояния» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта ConditionType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/condition_type_model.py, строки 18–18.

```
18      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/condition_type_routes.py, строки 165–177.

```
165 @refdata_bp.route("/add_condition_type", methods=["GET", "POST"])
166 @login_required
167 def add_condition_type():
168     """ Маршрут для добавления нового вида топлива. """
169 
170     user = session.get('username', 'Неизвестный пользователь')
171     log_to_db(
172         user,
173         "Открыта страница добавления типа состояния",
174         entity_type="condition_type"
175         )
176 
177     # Создание формы
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/condition_type_routes.py, строки 212–222.

```
212                                 sort_dir=sort_dir,
213                     ))
214                 add_condition_type_all_versions_service(payload, user)
215                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
216             else:
217                 add_condition_type_service(payload, user)
218                 flash("Новая запись успешно добавлена.", "success")
219 
220             # Перенаправление на список с сохранением параметров и переходом к новой записи
221             total_records = condition_type_query(
222                                 condition_type_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/refdata_for_stations/condition_type_services.py, строки 215–230.

```
215                     raise ValueError(f"Запись с наименованием «{name}» уже существует.")
216
217                 # Создаем новую запись
218                 obj = ConditionType(
219                     name=name
220                     )
221                 set_db_version_on_create(obj)
222                 db.session.add(obj)
223                 db.session.flush()  # получить id без полного коммита
224
225                 log_to_db(
226                     user,
227                     "Создан тип состояния",
228                     f"Наименование: {name}",
229                     entity_type="condition_type",
230                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/refdata_for_stations/condition_type/condition_type_add.html, строки 22–36.

```
22          <div class="mb-3">
23              <label for="name" class="form-label">Наименование типа состояния</label>
24              {{ form.name(class="form-control", id="name") }}
25              {% if form.name.errors %}
26                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
27              {% endif %}
28          </div>
29
30          <!-- Кнопки сохранения записи и возвращения назад -->
31          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
32              <button type="submit" class="btn btn-primary">Добавить</button>
33              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
34
35              <a href="{{ url_for('refdata_bp.condition_type_list',
36                                  page=page,
```

Взаимодействие при добавлении. Классификатор применяется в модуле генерации при хранении состояний оборудования. Он задает значение признака, а не даты ввода, демонтажа или ремонтные интервалы; конкретные ссылки потребителей приведены ниже. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.16.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_condition_type	export_condition_type	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.17. Типы агрегатов

4.17.1. Страница списка

Адрес: GET, POST /refdata/machine_type. Обработчик: machine_type_list().

Содержит общий классификатор типов агрегатов. Предусмотрены табличное редактирование и отдельная форма добавления типа.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, machine_type_filter, all_versions, machine_ids[], machine_type_names[], machine_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: MachineType. Поля SQLAlchemy: id, name, created_at, updated_at, database_version_id, machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/machine/machine_type_model.py, строки 13–31.

```
  13  class MachineType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_machine_types'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      # Сохраняем исходную nullable=True, чтобы не трогать существующие данные.
  19      name = db.Column(db.String(80), unique=True, nullable=True, index=True)
  20
  21      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  22      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  23
  24      # Поле для связи с версией БД
  25      database_version_id = db.Column(
  26          db.Integer,
  27          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
  28          nullable=True,
  29          index=True
  30      )
  31
```

Связь модели с другой сущностью. Источник: app/refdata/models/refdata_for_stations/machine/machine_type_model.py, строки 32–32.

```
  32      machines = db.relationship('Machine', back_populates='machine_type')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/machines/machine_type_routes.py, строки 35–50.

```
  35  @refdata_bp.route("/machine_type", methods=["GET", "POST"])
  36  @login_required
  37  def machine_type_list():
  38      """Маршрут для отображения списка типов агрегатов."""
  39
  40      user = session.get('username', 'Неизвестный пользователь')
  41      log_to_db(
  42          user,
  43          "Открыта страница типов агрегатов",
  44          entity_type="machine_type")
  45
  46      # Создание формы
  47      form = MachineTypeFilterForm()
  48
  49      # Получение параметров запроса
  50      page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/machines/machine_type_routes.py, строки 124–129.

```
 124                              sort_dir=sort_dir,
 125                      ))
 126                  update_machine_type_all_versions_service(machine_type_data, user)
 127                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 128              else:
 129                  update_machine_type_service(machine_type_data, user)
```

Сервис списка – выборка и пагинация. Источник:
app/refdata/services/refdata_for_stations/machines/machine_type_services.py, строки 69–86.

```
@no_autoflush
def get_machine_type_list(
    page,
    per_page,
    machine_type_filter=None,
    sort_by="id",
    sort_dir="asc"):
    """ Получает список типов агрегатов с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = machine_type_query(
        machine_type_filter=machine_type_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник:
app/refdata/services/refdata_for_stations/machines/machine_type_services.py, строки 154–166.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по типам агрегатов",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="machine_type",
                entity_id=machine_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник:
app/templates/refdata/refdata_for_stations/machines/machine_type/machine_type.html, строки 160–173.

```
                            <!-- Строки таблицы. Столбец "Тип агрегата" -->
                            <td>
                                <input type="text" name="machine_type_names[]" class="form-control"
value="{{ machine_type.name }}">
                            </td>

                            <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
                            <td class="text-center align-middle">
                                <input type="checkbox" name="machine_type_delete[]" value="{{
machine_type.id }}" class="form-check-input">
                            </td>
                        </tr>
                        {% else %}
                        <tr>
                            <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
                        </tr>
```

Взаимодействие с другими модулями. Machine.id_machine_type связывает агрегат с общим типом. Модуль генерации использует эту классификацию при представлении состава оборудования; специализированные типы ТЭС и ПГУ ведутся в самостоятельных справочниках.

Подтверждающие ссылки на потребителей модели:
app/generation/models/machine/machine_model.py:97 – machine_type = db.relationship('MachineType', back_populates='machines')
app/generation/services/machine_services/machine_services.py:31 – from app.refdata.models.refdata_for_stations.machine.machine_type_model import MachineType
app/generation/services/station_services/import_station_services.py:34 – from app.refdata.models.refdata_for_stations.machine.machine_type_model import MachineType

4.17.2. Форма добавления

Адрес: GET, POST /refdata/add_machine_type. Обработчик: add_machine_type().

Назначение: создание записи справочника «Типы агрегатов». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – MachineType, модель и ее связи приведены выше в подразделе 4.17.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип агрегата', validators=[DataRequired(message='Поле «Тип агрегата» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта MachineType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/machine/machine_type_model.py, строки 19–19.

```
19      name = db.Column(db.String(80), unique=True, nullable=True, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/machines/machine_type_routes.py, строки 164–176.

```
164  @refdata_bp.route("/add_machine_type", methods=["GET", "POST"])
165  @login_required
166  def add_machine_type():
167      """ Маршрут для добавления нового типа агрегата. """
168
169      user = session.get('username', 'Неизвестный пользователь')
170      log_to_db(
171          user,
172          "Открыта страница добавления типов агрегатов",
173          entity_type="machine_type")
174
175      # Создание формы
176      form = AddMachineTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/machines/machine_type_routes.py, строки 210–220.

```
210                              sort_dir=sort_dir,
211                          ))
212                  add_machine_type_all_versions_service(payload, user)
213                  flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
214              else:
215                  add_machine_type_service(payload, user)
216                  flash("Новая запись успешно добавлена.", "success")
217
218              # Перенаправление на список с сохранением параметров и переходом к новой записи
219              total_records = machine_type_query(
220                                  machine_type_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/refdata_for_stations/machines/machine_type_services.py, строки 213–226.

```
213                 if dup:
214                     raise ValueError(f"Запись с наименованием «{name}» уже существует.")
215
216                 obj = MachineType(name=name)
217                 set_db_version_on_create(obj)
218                 db.session.add(obj)
219                 db.session.flush()
220
221                 log_to_db(
222                     user,
223                     "Создан тип агрегата",
224                     f"Наименование: {name}",
225                     entity_type="machine_type",
226                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/refdata_for_stations/machines/machine_type/machine_type_add.html, строки 20–34.

```
20          <div class="mb-3">
21              <label for="name" class="form-label">Тип агрегата</label>
22              {{ form.name(class="form-control", id="name") }}
23              {% if form.name.errors %}
24                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
25              {% endif %}
26          </div>
27
28          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
29              <button type="submit" class="btn btn-primary">Добавить</button>
30              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
31              <a href="{{ url_for('refdata_bp.machine_type_list',
32                                  page=page,
33                                  per_page=per_page,
34                                  sort_by=sort_by,
```

Взаимодействие при добавлении. Machine.id_machine_type связывает агрегат с общим типом. Модуль генерации использует эту классификацию при представлении состава оборудования; специализированные типы ТЭС и ПГУ ведутся в самостоятельных справочниках. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.17.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_machine_type	export_machine_type	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.18. Типы ТЭС

4.18.1. Страница списка

Адрес: GET, POST /refdata/tes_type. Обработчик: tes_type_list().

Ведет типы тепловых электростанций с порядком отображения. Классификатор редактируется в списке; новые записи вводятся в отдельной форме.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, tes_type_filter, all_versions, tes_ids[], display_orders[], tes_type_names[], tes_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: TesType. Поля SQLAlchemy: id, display_order, name, created_at, updated_at, database_version_id, machine_tes_types.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/machine/tes_type_model.py, строки 13–31.

```
  13  class TesType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_tes_types'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      # Порядок отображения в справочнике
  19      display_order = db.Column(db.Integer, nullable=True)
  20      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
  21
  22      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  23      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  24
  25      # Поле для связи с версией БД
  26      database_version_id = db.Column(
  27          db.Integer,
  28          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
  29          nullable=True,
  30          index=True
  31      )
```

Связь модели с другой сущностью. Источник: app/refdata/models/refdata_for_stations/machine/tes_type_model.py, строки 33–37.

```
  33      machine_tes_types = db.relationship(
  34          'MachineTesType',
  35          back_populates='tes_type',
  36          cascade="all, delete-orphan"
  37      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/machines/tes_type_routes.py, строки 42–57.

```
  42  @refdata_bp.route("/tes_type", methods=["GET", "POST"])
  43  @login_required
  44  def tes_type_list():
  45      """Маршрут для отображения списка типов ТЭС."""
  46
  47      user = session.get('username', 'Неизвестный пользователь')
  48      log_to_db(
  49          user,
  50          "Открыта страница типов ТЭС",
  51          entity_type="tes_type")
  52
  53      # Создание формы
  54      form = TesTypeFilterForm()
  55
  56      # Получение параметров запроса
  57      page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/machines/tes_type_routes.py, строки 145–150.

```
 145                               sort_dir=sort_dir,
 146                      ))
 147                  update_tes_type_all_versions_service(tes_type_data, user)
 148                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 149              else:
 150                  update_tes_type_service(tes_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/refdata_for_stations/machines/tes_type_services.py, строки 82–99.

```
 82  @no_autoflush
 83  def get_tes_type_list(
 84      page,
 85      per_page,
 86      tes_type_filter=None,
 87      sort_by="display_order",
 88      sort_dir="asc"):
 89      """ Получает список типов ТЭС с пагинацией, фильтрацией и сортировкой. """
 90
 91      # Базовый запрос
 92      query = tes_type_query(
 93          tes_type_filter=tes_type_filter,
 94          sort_by=sort_by,
 95          sort_dir=sort_dir,
 96      )
 97
 98      # Пагинация
 99      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/refdata_for_stations/machines/tes_type_services.py, строки 188–200.

```
188
189      try:
190          # Сохранение изменений в базе данных
191          # Фиксация транзакции (устойчивый коммит)
192          _commit_with_retry()
193
194          if updated_ids:
195              log_to_db(
196                  user,
197                  "Сохранены изменения по типам ТЭС",
198                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
199                  entity_type="tes_type",
200                  entity_id=tes_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/refdata_for_stations/machines/tes_type/tes_type.html, строки 192–205.

```
192                              <!-- Строки таблицы. Столбец "Тип ТЭС" -->
193                              <td>
194                                  <input type="text" name="tes_type_names[]" class="form-control"
value="{{ tes_type.name }}">
195                              </td>
196
197                              <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
198                              <td class="text-center align-middle">
199                                  <input type="checkbox" name="tes_type_delete[]" value="{{
tes_type.id }}" class="form-check-input">
200                              </td>
201                          </tr>
202                          {% else %}
203                          <tr>
204                              <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
205                          </tr>
```

Взаимодействие с другими модулями. MachineTesType использует классификатор для привязки типа ТЭС к оборудованию во времени. Это позволяет модулю генерации учитывать годовую классификацию без хранения типа только в текстовом наименовании агрегата.

Подтверждающие ссылки на потребителей модели:
app/generation/models/machine/machine_tes_type_model.py:4 – - Связи сохранены: year (Year.machine_tes_types), machine_tes_type (Machine.machine_tes_types), tes_type (TesType.machine_tes_types).
app/generation/routes/totals_summary_routes.py:189 – from app.refdata.models.refdata_for_stations.machine.tes_type_model import TesType
app/generation/services/machine_services/machine_services.py:30 – from app.refdata.models.refdata_for_stations.machine.tes_type_model import TesType

app/generation/services/station_changes_services/station_changes_services.py:23 – from app.refdata.models.refdata_for_stations.machine.tes_type_model import TesType
app/generation/services/station_services/export_totals_summary_services.py:30 – from app.refdata.models.refdata_for_stations.machine.tes_type_model import TesType

4.18.2. Форма добавления

Адрес: GET, POST /refdata/add_tes_type. Обработчик: add_tes_type().

Назначение: создание записи справочника «Типы ТЭС». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – TesType, модель и ее связи приведены выше в подразделе 4.18.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип ТЭС', validators=[DataRequired(message='Поле «Тип ТЭС обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта TesType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/machine/tes_type_model.py, строки 20–20.

```
20      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/machines/tes_type_routes.py, строки 185–197.

```
185  @refdata_bp.route("/add_tes_type", methods=["GET", "POST"])
186  @login_required
187  def add_tes_type():
188      """ Маршрут для добавления нового типа ТЭС. """
189
190      user = session.get('username', 'Неизвестный пользователь')
191      log_to_db(
192          user,
193          "Открыта страница добавления типов ТЭС",
194          entity_type="tes_type")
195
196      # Создание формы
197      form = AddTesTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/machines/tes_type_routes.py, строки 231–241.

```
231                                sort_dir=sort_dir,
232                      ))
233                  add_tes_type_all_versions_service(payload, user)
234                  flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
235              else:
236                  add_tes_type_service(payload, user)
237                  flash("Новая запись успешно добавлена.", "success")
238
239              # Перенаправление на список с сохранением параметров и переходом к новой записи
240              total_records = tes_type_query(
241                                    tes_type_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/refdata_for_stations/machines/tes_type_services.py, строки 261–274.

```
                                f"Запись с порядком отображения «{display_order}» уже существует."
                            )

                obj = TesType(name=name, display_order=display_order)
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()

                log_to_db(
                    user,
                    "Создан тип ТЭС",
                    f"Наименование: {name}",
                    entity_type="tes_type",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/refdata_for_stations/machines/tes_type/tes_type_add.html, строки 20–34.

```
        <div class="mb-3">
            <label for="name" class="form-label">Тип ТЭС</label>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>

        <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
            <button type="submit" class="btn btn-primary">Добавить</button>
            {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
            <a href="{{ url_for('refdata_bp.tes_type_list',
                                page=page,
                                per_page=per_page,
                                sort_by=sort_by,
```

Взаимодействие при добавлении. MachineTesType использует классификатор для привязки типа ТЭС к оборудованию во времени. Это позволяет модулю генерации учитывать годовую классификацию без хранения типа только в текстовом наименовании агрегата. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.18.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_tes_type	export_tes_type	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.19. Типы агрегатов ТЭС

4.19.1. Страница списка

Адрес: GET, POST /refdata/tes_machine_type. Обработчик: tes_machine_type_list().

Ведет специализированные типы агрегатов тепловых электростанций и порядок отображения. Страница поддерживает поиск и табличное редактирование, форма добавления создает новый классификационный элемент.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, tes_machine_type_filter, all_versions, tes_machine_ids[], display_orders[], tes_machine_type_names[], tes_machine_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: TesMachineType. Поля SQLAlchemy: id, display_order, name, created_at, updated_at, database_version_id, machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/machine/tes_machine_type_model.py, строки 13–31.

```
class TesMachineType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_tes_machine_types'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    # Порядок отображения в справочнике
    display_order = db.Column(db.Integer, nullable=True)
    name = db.Column(db.String(80), unique=True, nullable=False, index=True)

    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)

    # Поле для связи с версией БД
    database_version_id = db.Column(
        db.Integer,
        db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
        nullable=True,
        index=True
    )
```

Связь модели с другой сущностью. Источник: app/refdata/models/refdata_for_stations/machine/tes_machine_type_model.py, строки 33–37.

```
    machines = db.relationship(
        'Machine',
        back_populates='tes_machine_type',
        cascade="all, delete-orphan"
    )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/machines/tes_machine_type_routes.py, строки 42–57.

```
@refdata_bp.route("/tes_machine_type", methods=["GET", "POST"])
@login_required
def tes_machine_type_list():
    """Маршрут для отображения списка типов агрегатов ТЭС."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница типов агрегатов ТЭС",
        entity_type="tes_machine_type")

    # Создание формы
    form = TesMachineTypeFilterForm()

    # Получение параметров запроса
    page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/machines/tes_machine_type_routes.py, строки 145–150.

```
                            sort_dir=sort_dir,
                    ))
                update_tes_machine_type_all_versions_service(tes_machine_type_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_tes_machine_type_service(tes_machine_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/refdata_for_stations/machines/tes_machine_type_services.py, строки 81–98.

```
@no_autoflush
def get_tes_machine_type_list(
    page,
    per_page,
    tes_machine_type_filter=None,
    sort_by="display_order",
    sort_dir="asc"):
    """ Получает список типов агрегатов ТЭС с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = tes_machine_type_query(
        tes_machine_type_filter=tes_machine_type_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/refdata_for_stations/machines/tes_machine_type_services.py, строки 187–199.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по типам агрегатов ТЭС",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="tes_machine_type",
                entity_id=tes_machine_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/refdata_for_stations/machines/tes_machine_type/tes_machine_type.html, строки 192–205.

```
                            <!-- Строки таблицы. Столбец "Тип агрегата" -->
                            <td>
                                <input type="text" name="tes_machine_type_names[]" class="form-
control" value="{{ tes_machine_type.name }}">
                            </td>

                            <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
                            <td class="text-center align-middle">
                                <input type="checkbox" name="tes_machine_type_delete[]" value="{{
tes_machine_type.id }}" class="form-check-input">
                            </td>
                        </tr>
                        {% else %}
                        <tr>
                            <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
                        </tr>
```

Взаимодействие с другими модулями. Специализированные модели оборудования генерации используют TesMachineType в дополнение к общему MachineType. Классификация участвует в представлении теплового оборудования; конкретные модели и сервисы-потребители указаны в ссылках ниже.

Подтверждающие ссылки на потребителей модели:
app/generation/models/machine/machine_model.py:106 – tes_machine_type = db.relationship('TesMachineType', back_populates='machines')
app/generation/routes/totals_summary_routes.py:191 – TesMachineType,
app/generation/models/pgu_machine/pgu_machine_model.py:53 – tes_machine_type = db.relationship('TesMachineType', backref='pgu_machines')

app/generation/services/machine_services/machine_services.py:29 – from app.refdata.models.refdata_for_stations.machine.tes_machine_type_model import TesMachineType
app/generation/services/station_changes_services/station_changes_services.py:24 – from app.refdata.models.refdata_for_stations.machine.tes_machine_type_model import TesMachineType

4.19.2. Форма добавления

Адрес: GET, POST /refdata/add_tes_machine_type. Обработчик: add_tes_machine_type().

Назначение: создание записи справочника «Типы агрегатов ТЭС». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – TesMachineType, модель и ее связи приведены выше в подразделе 4.19.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип агрегата ТЭС', validators=[DataRequired(message='Поле «Тип агрегата ТЭС» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта TesMachineType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/machine/tes_machine_type_model.py, строки 20–20.

```
20      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/machines/tes_machine_type_routes.py, строки 185–197.

```
185  @refdata_bp.route("/add_tes_machine_type", methods=["GET", "POST"])
186  @login_required
187  def add_tes_machine_type():
188      """ Маршрут для добавления нового типа агрегата ТЭС. """
189
190      user = session.get('username', 'Неизвестный пользователь')
191      log_to_db(
192          user,
193          "Открыта страница добавления типов агрегатов ТЭС",
194          entity_type="tes_machine_type")
195
196      # Создание формы
197      form = AddTesMachineTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/machines/tes_machine_type_routes.py, строки 231–241.

```
231                                sort_dir=sort_dir,
232                            ))
233                    add_tes_machine_type_all_versions_service(payload, user)
234                    flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
235                else:
236                    add_tes_machine_type_service(payload, user)
237                    flash("Новая запись успешно добавлена.", "success")
238
239                # Перенаправление на список с сохранением параметров и переходом к новой записи
240                total_records = tes_machine_type_query(
241                                    tes_machine_type_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/refdata_for_stations/machines/tes_machine_type_services.py, строки 263–276.

```
                        )

                # Создаем новую запись
                obj = TesMachineType(name=name, display_order=display_order)
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()

                log_to_db(
                    user,
                    "Создан тип агрегата ТЭС",
                    f"Наименование: {name}",
                    entity_type="tes_machine_type",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/refdata_for_stations/machines/tes_machine_type/tes_machine_type_add.html, строки 20–34.

```
        <div class="mb-3">
            <label for="name" class="form-label">Тип агрегата ТЭС</label>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>

        <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
            <button type="submit" class="btn btn-primary">Добавить</button>
            {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
            <a href="{{ url_for('refdata_bp.tes_machine_type_list',
                                page=page,
                                per_page=per_page,
                                sort_by=sort_by,
```

Взаимодействие при добавлении. Специализированные модели оборудования генерации используют TesMachineType в дополнение к общему MachineType. Классификация участвует в представлении теплового оборудования; конкретные модели и сервисы-потребители указаны в ссылках ниже. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.19.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_tes_machine_type	export_tes_machine_type	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.20. Типы агрегатов ПГУ

4.20.1. Страница списка

Адрес: GET, POST /refdata/pgu_tes_machine_type. Обработчик: pgu_tes_machine_type_list().

Ведет классификатор типов агрегатов парогазовых установок. Страница списка используется для просмотра и изменения наименований; отдельная форма добавляет тип.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, pgu_tes_machine_type_filter, all_versions, pgu_tes_machine_type_ids[], pgu_tes_machine_type_names[], pgu_tes_machine_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: PGUTesMachineType. Поля SQLAlchemy: id, name, created_at, updated_at, database_version_id, pgu_machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/machine/pgu_tes_machine_type_model.py, строки 13–35.

```
13  class PGUTesMachineType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
14      __tablename__ = 'gs_sys_pgu_tes_machine_types'
15      __table_args__ = {"schema": SCHEMA_REFDATA}
16
17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
18      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
19
20      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
22
23      # Поле для связи с версией БД
24      database_version_id = db.Column(
25          db.Integer,
26          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
27          nullable=True,
28          index=True
29      )
30
31      pgu_machines = db.relationship(
32          'PGUMachine',
33          back_populates='pgu_tes_machine_type',
34          cascade="all, delete-orphan"
35      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/machines/pgu_tes_machine_type_routes.py, строки 35–50.

```
35  @refdata_bp.route("/pgu_tes_machine_type", methods=["GET", "POST"])
36  @login_required
37  def pgu_tes_machine_type_list():
38      """Маршрут для отображения списка типов агрегатов ПГУ."""
39
40      user = session.get('username', 'Неизвестный пользователь')
41      log_to_db(
42          user,
43          "Открыта страница типов агрегатов ПГУ",
44          entity_type="pgu_tes_machine_type")
45
46      # Создание формы
47      form = PGUTesMachineTypeFilterForm()
48
49      # Получение параметров запроса
50      page                            = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/machines/pgu_tes_machine_type_routes.py, строки 124–129.

```
124                                             sort_dir=sort_dir,
125                     ))
126                 update_pgu_tes_machine_type_all_versions_service(pgu_tes_machine_type_data, user)
127                 flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
128             else:
129                 update_pgu_tes_machine_type_service(pgu_tes_machine_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/refdata_for_stations/machines/pgu_tes_machine_type_services.py, строки 69–86.

```
 69  @no_autoflush
 70  def get_pgu_tes_machine_type_list(
 71      page,
 72      per_page,
 73      pgu_tes_machine_type_filter=None,
 74      sort_by="id",
 75      sort_dir="asc"):
 76      """ Получает список типов агрегатов ПГУ с пагинацией, фильтрацией и сортировкой. """
 77
 78      # Базовый запрос
 79      query = pgu_tes_machine_type_query(
 80          pgu_tes_machine_type_filter=pgu_tes_machine_type_filter,
 81          sort_by=sort_by,
 82          sort_dir=sort_dir,
 83      )
 84
 85      # Пагинация
 86      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/refdata_for_stations/machines/pgu_tes_machine_type_services.py, строки 154–166.

```
154
155      try:
156          # Сохранение изменений в базе данных
157          # Фиксация транзакции (устойчивый коммит)
158          _commit_with_retry()
159
160          if updated_ids:
161              log_to_db(
162                  user,
163                  "Сохранены изменения по типам агрегатов",
164                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
165                  entity_type="pgu_tes_machine_type",
166                  entity_id=pgu_tes_machine_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/refdata_for_stations/machines/pgu_tes_machine_type/pgu_tes_machine_type.html, строки 160–173.

```
160                              <!-- Строки таблицы. Столбец "Тип агрегата" -->
161                              <td>
162                                  <input type="text" name="pgu_tes_machine_type_names[]" class="form-
control" value="{{ pgu_tes_machine_type.name }}">
163                              </td>
164
165                              <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
166                              <td class="text-center align-middle">
167                                  <input type="checkbox" name="pgu_tes_machine_type_delete[]"
value="{{ pgu_tes_machine_type.id }}" class="form-check-input">
168                              </td>
169                          </tr>
170                          {% else %}
171                          <tr>
172                              <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
173                          </tr>
```

Взаимодействие с другими модулями. PGUMachine.pgu_tes_machine_type связывает состав ПГУ с классификатором. Тип используется при отображении и обработке оборудования в модуле генерации; он не заменяет связи агрегатов ПГУ с их техническими характеристиками.

Подтверждающие ссылки на потребителей модели:
app/generation/models/pgu_machine/pgu_machine_model.py:62 – pgu_tes_machine_type = db.relationship('PGUTesMachineType', back_populates='pgu_machines')
app/generation/services/machine_services/machine_services.py:32 – from app.refdata.models.refdata_for_stations.machine.pgu_tes_machine_type_model import PGUTesMachineType
app/generation/services/station_changes_services/station_changes_services.py:25 – from app.refdata.models.refdata_for_stations.machine.pgu_tes_machine_type_model import PGUTesMachineType

app/generation/services/station_services/filters_services.py:35 – PGUTesMachineType,
app/generation/services/station_services/station_services.py:3548 – from
app.refdata.models.refdata_for_stations.machine.pgu_tes_machine_type_model import PGUTesMachineType

4.20.2. Форма добавления

Адрес: GET, POST /refdata/add_pgu_tes_machine_type. Обработчик: add_pgu_tes_machine_type().

Назначение: создание записи справочника «Типы агрегатов ПГУ». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – PGUTesMachineType, модель и ее связи приведены выше в подразделе 4.20.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип агрегата', validators=[DataRequired(message='Поле «Тип агрегата» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта PGUTesMachineType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/machine/pgu_tes_machine_type_model.py, строки 18–18.

```
18      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/machines/pgu_tes_machine_type_routes.py, строки 164–176.

```
164  @refdata_bp.route("/add_pgu_tes_machine_type", methods=["GET", "POST"])
165  @login_required
166  def add_pgu_tes_machine_type():
167      """ Маршрут для добавления нового типа агрегата ПГУ. """
168
169      user = session.get('username', 'Неизвестный пользователь')
170      log_to_db(
171          user,
172          "Открыта страница добавления типов агрегатов ПГУ",
173          entity_type="pgu_tes_machine_type")
174
175      # Создание формы
176      form = AddPGUTesMachineTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/machines/pgu_tes_machine_type_routes.py, строки 210–220.

```
210                                             sort_dir=sort_dir,
211                     ))
212                 add_pgu_tes_machine_type_all_versions_service(payload, user)
213                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
214             else:
215                 add_pgu_tes_machine_type_service(payload, user)
216                 flash("Новая запись типа агрегата ПГУ успешно добавлена.", "success")
217
218             # Перенаправление на список с сохранением параметров и переходом к новой записи
219             total_records = pgu_tes_machine_type_query(
220                                 pgu_tes_machine_type_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/refdata_for_stations/machines/pgu_tes_machine_type_services.py, строки 218–231.

```
218                 if dup:
219                     raise ValueError(f"Запись с наименованием «{name}» уже существует.")
220
221                 obj = PGUTesMachineType(name=name)
222                 set_db_version_on_create(obj)
223                 db.session.add(obj)
224                 db.session.flush()
225
226                 log_to_db(
227                     user,
228                     "Создан тип агрегата ПГУ",
229                     f"Наименование: {name}",
230                     entity_type="pgu_tes_machine_type",
231                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/refdata_for_stations/machines/pgu_tes_machine_type/pgu_tes_machine_type_add.html, строки 20–34.

```
20          <div class="mb-3">
21              <label for="name" class="form-label">Тип агрегата ПГУ</label>
22              {{ form.name(class="form-control", id="name") }}
23              {% if form.name.errors %}
24                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
25              {% endif %}
26          </div>
27
28          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
29              <button type="submit" class="btn btn-primary">Добавить ПГУ</button>
30              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
31              <a href="{{ url_for('refdata_bp.pgu_tes_machine_type_list',
32                                  page=page,
33                                  per_page=per_page,
34                                  sort_by=sort_by,
```

Взаимодействие при добавлении. PGUMachine.pgu_tes_machine_type связывает состав ПГУ с классификатором. Тип используется при отображении и обработке оборудования в модуле генерации; он не заменяет связи агрегатов ПГУ с их техническими характеристиками. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.20.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_pgu_tes_machine_type	export_pgu_tes_machine_type	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.21. Типы групп оборудования

4.21.1. Страница списка

Адрес: GET, POST /refdata/equipment_group. Обработчик: equipment_group_list().

Хранит наименование и порядок отображения типа группы оборудования, ссылки на тип технологии и ее доступность. В списке имеются фильтры по связанным классификаторам. Сервисы обрабатывают совпадающие типы групп и

перенос связей на каноническую запись; удаление выполняется с учетом зависимостей.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, technology_type_filter, technology_availability_filter, equipment_group_filter, all_versions, equipment_group_ids[], display_orders[], equipment_group_names[], technology_types[], technology_availabilities[], equipment_group_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

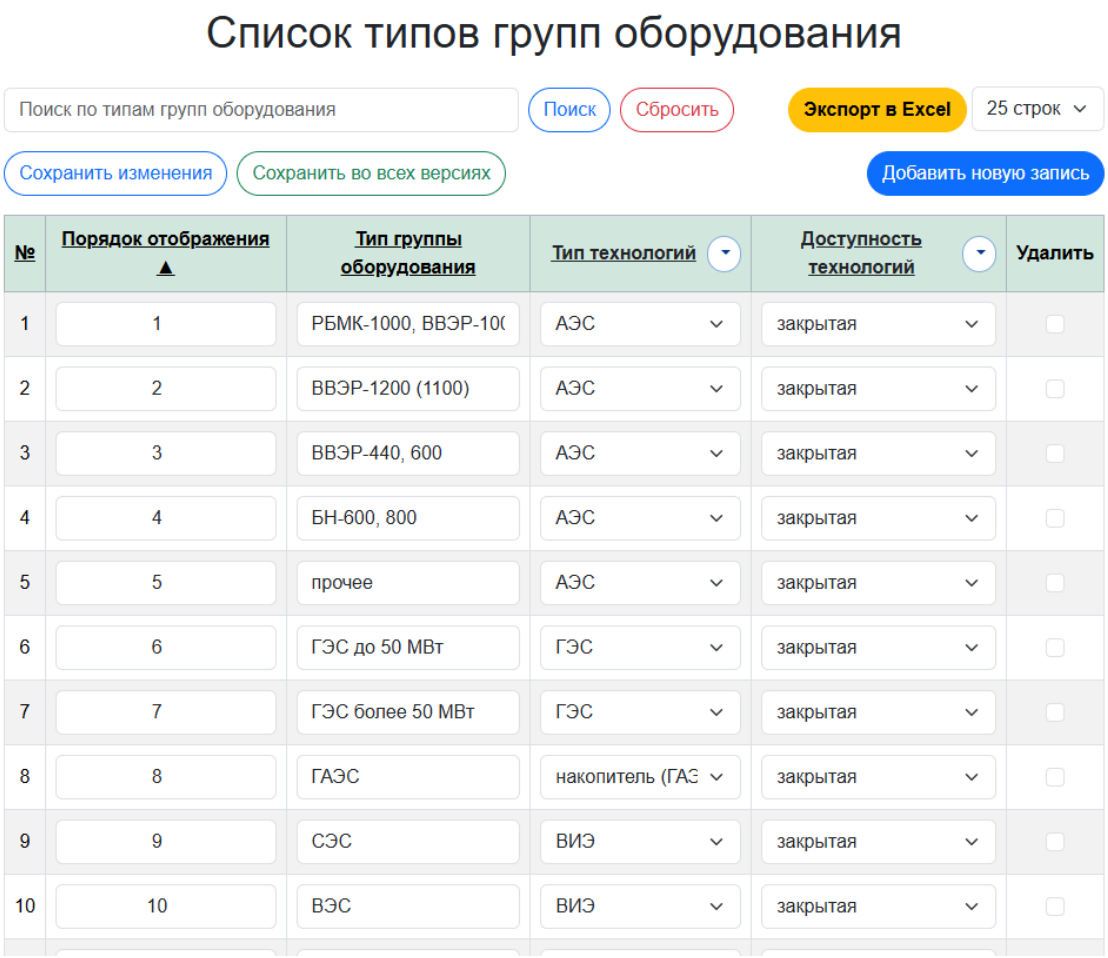


Рисунок 7. Типы групп оборудования. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Модель: EquipmentGroupType. Поля SQLAlchemy: id, display_order, name, id_technology_availability, technology_availability, id_technology_type, technology_type, created_at, updated_at, database_version_id, machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/technologies/equipment_group_model.py, строки 14–32.

```
class EquipmentGroupType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_equipment_groups'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    # Порядок отображения
    display_order = db.Column(db.Integer, nullable=True)
    name = db.Column(db.String(80), unique=True, nullable=False, index=True)

    # FK -> TechnologyAvailability
    id_technology_availability = db.Column(
        db.Integer,
        db.ForeignKey(f'{SCHEMA_REFDATA}.gs_sys_technology_availabilities.id', ondelete='SET
NULL'),
        nullable=True,
        index=True,
    )
    technology_availability = db.relationship('TechnologyAvailability',
back_populates='equipment_groups')

    # FK -> TechnologyType
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/technologies/equipment_group_routes.py, строки 58–73.

```
@refdata_bp.route("/equipment_group", methods=["GET", "POST"])
@login_required
def equipment_group_list():
    """Маршрут для отображения списка типов групп оборудования."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница типов групп оборудования",
        entity_type="equipment_group")

    # Создание формы
    form = EquipmentGroupTypeFilterForm()

    # Получение параметров запроса
    page                            = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/technologies/equipment_group_routes.py, строки 186–193.

```
                                            sort_by=sort_by,
                                            sort_dir=sort_dir))
                update_equipment_group_all_versions_service(
                    equipment_group_data, user
                )
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_equipment_group_service(equipment_group_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/refdata_for_stations/technologies/equipment_group_services.py, строки 345–366.

```
@no_autoflush
def get_equipment_group_list(
    page,
    per_page,
    equipment_group_filter=None,
    technology_type_filter=None,
    technology_availability_filter=None,
    sort_by="display_order",
    sort_dir="asc"):
    """ Получает список типов групп оборудования с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = equipment_group_query(
        equipment_group_filter=equipment_group_filter,
        technology_type_filter=technology_type_filter,
        technology_availability_filter=technology_availability_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/refdata_for_stations/technologies/equipment_group_services.py, строки 496–508.

```
                    updated_ids.append(equipment_group_id)

            db.session.flush()

        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по типам групп оборудования",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="equipment_group",
                entity_id=last_entity_id)
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/refdata_for_stations/technologies/equipment_group/equipment_group.html, строки 333–346.

```
                            <!-- Строки таблицы. Столбец "Группа оборудования" -->
                            <td>
                                <input type="text" name="equipment_group_names[]" class="form-
control" value="{{ equipment_group.name }}">
                            </td>

                            <!-- Строки таблицы. Столбец "Тип технологий" -->
                            <td>
                                <select name="technology_types[]" class="form-select">
                                    {% for technology_type in technology_types %}
                                    <option value="{{ technology_type.0 }}"
                                        {% if technology_type.0 ==
equipment_group.id_technology_type %}selected{% endif %}>
                                        {{ technology_type.1 }}
                                    </option>
                                    {% endfor %}
```

Взаимодействие с другими модулями. Machine.id_equipment_group ссылается на EquipmentGroupType. Классификатор используется генерацией, топливным модулем и моделями рациональной структуры. Это именно тип группы, а не экземпляр топливной группы оборудования: назначение конкретных агрегатов и расчетные параметры ведутся в профильных модулях.

Подтверждающие ссылки на потребителей модели:
app/fuel/models/fue_equipment_group_set_station_model.py:43 – "EquipmentGroupType",
app/generation/models/machine/machine_model.py:133 – equipment_group = db.relationship('EquipmentGroupType', back_populates='machines')

app/rational_structure/models/territory_technology_model.py:26 – equipment_group_type = db.relationship("EquipmentGroupType")
app/api/services/equipment_group_exchange_services.py:29 – EquipmentGroupType,
app/ecology/services/gis_tes_pg_from_fuel_services.py:113 – «Котельная» – это поле «Тип группы оборудования» (EquipmentGroupType.name

4.21.2. Форма добавления

Адрес: GET, POST /refdata/add_equipment_group. Обработчик: add_equipment_group().

Назначение: создание записи справочника «Типы групп оборудования». Вводимые поля формы: name, technology_type, technology_availability. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – EquipmentGroupType, модель и ее связи приведены выше в подразделе 4.21.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип группы оборудования', validators=[DataRequired(message='Поле «Тип группы оборудования» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); technology_type = SelectField('Тип технологии', choices=[], coerce=int, validators=[Optional()]); technology_availability = SelectField('Доступность технологии', choices=[], coerce=int, validators=[Optional()]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта EquipmentGroupType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/technologies/equipment_group_model.py, строки 21–21.

```
21      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/technologies/equipment_group_routes.py, строки 244–256.

```
244 @refdata_bp.route("/add_equipment_group", methods=["GET", "POST"])
245 @login_required
246 def add_equipment_group():
247     """ Маршрут для добавления нового типа группы оборудования. """
248
249     user = session.get('username', 'Неизвестный пользователь')
250     log_to_db(
251         user,
252         "Открыта страница добавления типов групп оборудования",
253         entity_type="equipment_group")
254
255     # Создание формы
256     form = AddEquipmentGroupTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/technologies/equipment_group_routes.py, строки 290–302.

```
290                                             sort_by=sort_by,
291                                             sort_dir=sort_dir))
292                 add_equipment_group_all_versions_service(payload, user)
293                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
294             else:
295                 add_equipment_group_service(payload, user)
296                 flash("Новая запись успешно добавлена.", "success")
297
298             # Перенаправление на список с сохранением параметров и переходом к новой записи
299             total_records = equipment_group_query(
300                                 equipment_group_filter,
301                                 technology_type_filter,
302                                 technology_availability_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/refdata_for_stations/technologies/equipment_group_services.py, строки 590–610.

```
590                     if dup_display:
591                         raise ValueError(f"Запись с порядком отображения «{display_order}» уже
существует.")
592
593                 obj = EquipmentGroupType(
594                     display_order=display_order,
595                     name=name,
596                     id_technology_type=technology_type_id,
597                     id_technology_availability=technology_availability_id
598                 )
599                 set_db_version_on_create(obj)
600                 db.session.add(obj)
601                 db.session.flush()
602
603                 log_to_db(
604                     user,
605                     "Создан тип группы оборудования",
606                     f"Порядок отображения: {_dash(display_order)}; "
607                     f"Тип технологии: {get_technology_type_name(technology_type_id)} "
608                     f"Тип доступности технологии:
{get_technology_availability_name(technology_availability_id)} ",
609                     entity_type="equipment_group",
610                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/refdata_for_stations/technologies/equipment_group/equipment_group_add.html, строки 22–36.

```
22          <div class="mb-3">
23              <label for="name" class="form-label">Тип группы оборудования</label>
24              {{ form.name(class="form-control", id="name") }}
25              {% if form.name.errors %}
26                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
27              {% endif %}
28          </div>
29
30          <div class="mb-3">
31              <label for="technology_type" class="form-label">Тип технологии</label>
32              {{ form.technology_type(class="form-control", id="technology_type") }}
33              {% if form.technology_type.errors %}
34                  <div class="text-danger">{{ form.technology_type.errors|join(', ') }}</div>
35              {% endif %}
36          </div>
```

Взаимодействие при добавлении. Machine.id_equipment_group ссылается на EquipmentGroupType. Классификатор используется генерацией, топливным модулем и моделями рациональной структуры. Это именно тип группы, а не экземпляр топливной группы оборудования: назначение конкретных агрегатов и расчетные параметры ведутся в профильных модулях. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.21.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_equipment_group	export_equipment_group	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.22. Типы технологий

4.22.1. Страница списка

Адрес: GET, POST /refdata/technology_type. Обработчик: technology_type_list().

Ведет типы технологий и порядок отображения. Страница позволяет искать и редактировать записи; новая технология добавляется через самостоятельную форму.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, technology_type_filter, all_versions, technology_ids[], display_orders[], technology_type_names[], technology_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: TechnologyType. Поля SQLAlchemy: id, display_order, name, created_at, updated_at, database_version_id, equipment_groups, machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/technologies/technology_type_model.py, строки 13–32.

```
  13  class TechnologyType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_technology_types'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      # Порядок отображения
  19      display_order = db.Column(db.Integer, nullable=True)
  20      # Сохраняем исходную nullable=True, чтобы не трогать существующие данные.
  21      name = db.Column(db.String(80), unique=True, nullable=True, index=True)
  22
  23      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  24      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  25
  26      # Поле для связи с версией БД
  27      database_version_id = db.Column(
  28          db.Integer,
  29          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
  30          nullable=True,
  31          index=True
  32      )
```

Связь модели с другой сущностью. Источник: app/refdata/models/refdata_for_stations/technologies/technology_type_model.py, строки 34–34.

```
  34      equipment_groups = db.relationship('EquipmentGroupType', back_populates='technology_type')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_type_routes.py, строки 35–50.

```
@refdata_bp.route("/technology_type", methods=["GET", "POST"])
@login_required
def technology_type_list():
    """Маршрут для отображения списка типов технологий."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница типов технологий",
        entity_type="technology_type")

    # Создание формы
    form = TechnologyTypeFilterForm()

    # Получение параметров запроса
    page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_type_routes.py, строки 137–142.

```
                            sort_dir=sort_dir,
                        ))
                update_technology_type_all_versions_service(technology_type_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_technology_type_service(technology_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/refdata_for_stations/technologies/technology_type_services.py, строки 80–97.

```
@no_autoflush
def get_technology_type_list(
    page,
    per_page,
    technology_type_filter=None,
    sort_by="display_order",
    sort_dir="asc"):
    """ Получает список типов технологий с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = technology_type_query(
        technology_type_filter=technology_type_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/refdata_for_stations/technologies/technology_type_services.py, строки 223–235.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по типам технологий",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="technology_type",
                entity_id=technology_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/refdata_for_stations/technologies/technology_type/technology_type.html, строки 193–206.

```
 193                                <!-- Строки таблицы. Столбец "Тип технологии" -->
 194                                <td>
 195                                    <input type="text" name="technology_type_names[]" class="form-
control" value="{{ technology_type.name }}">
 196                                </td>
 197
 198                                <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
 199                                <td class="text-center align-middle">
 200                                    <input type="checkbox" name="technology_type_delete[]" value="{{
technology_type.id }}" class="form-check-input">
 201                                </td>
 202                            </tr>
 203                            {% else %}
 204                            <tr>
 205                                <td colspan="4" class="text-center align-middle">Нет данных для
отображения</td>
 206                            </tr>
```

Взаимодействие с другими модулями. EquipmentGroupType.id_technology_type и связи оборудования используют классификатор для технологической группировки. Модуль рациональной структуры и топливные сервисы обращаются к тем же сущностям в местах, подтвержденных ссылками на исходный код ниже.

Подтверждающие ссылки на потребителей модели:
app/generation/models/machine/machine_model.py:124 – technology_type = db.relationship('TechnologyType', back_populates='machines')
app/rational_structure/models/territory_technology_model.py:25 – technology_type = db.relationship("TechnologyType")
app/rational_structure/services/territory_equipment_services.py:38 – from app.refdata.models.refdata_for_stations.technologies.technology_type_model import TechnologyType
app/rational_structure/services/territory_technology_matching.py:8 – from app.refdata.models.refdata_for_stations.technologies.technology_type_model import TechnologyType

4.22.2. Форма добавления

Адрес: GET, POST /refdata/add_technology_type. Обработчик: add_technology_type().

Назначение: создание записи справочника «Типы технологий». Вводимые поля формы: name, display_order. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – TechnologyType, модель и ее связи приведены выше в подразделе 4.22.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип технологии', validators=[DataRequired(message='Поле «Тип технологии» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); display_order = IntegerField('Порядок отображения', validators=[Optional(), NumberRange(min=0)]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта TechnologyType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/technologies/technology_type_model.py, строки 21–21.

```
21      name = db.Column(db.String(80), unique=True, nullable=True, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_type_routes.py, строки 177–189.

```
177 @refdata_bp.route("/add_technology_type", methods=["GET", "POST"])
178 @login_required
179 def add_technology_type():
180     """ Маршрут для добавления нового типа технологии. """
181
182     user = session.get('username', 'Неизвестный пользователь')
183     log_to_db(
184         user,
185         "Открыта страница добавления типов технологий",
186         entity_type="technology_type")
187
188     # Создание формы
189     form = AddTechnologyTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_type_routes.py, строки 230–240.

```
230                             sort_dir=sort_dir,
231                     ))
232                 add_technology_type_all_versions_service(payload, user)
233                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
234             else:
235                 add_technology_type_service(payload, user)
236                 flash("Новая запись успешно добавлена.", "success")
237
238             # Перенаправление на список с сохранением параметров и переходом к новой записи
239             total_records = technology_type_query(
240                                 technology_type_filter=technology_type_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/refdata_for_stations/technologies/technology_type_services.py, строки 312–325.

```
312                             f"Порядок «{display_order}» уже задан у записи «{other}»
(id={dup_order.id})."
313                         )
314
315                 obj = TechnologyType(name=name, display_order=display_order)
316                 set_db_version_on_create(obj)
317                 db.session.add(obj)
318                 db.session.flush()
319
320                 log_to_db(
321                     user,
322                     "Создан тип технологии",
323                     f"Наименование: {name}",
324                     entity_type="technology_type",
325                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/refdata_for_stations/technologies/technology_type/technology_type_add.html, строки 20–34.

```
20          <div class="mb-3">
21              <label for="name" class="form-label">Тип технологии</label>
22              {{ form.name(class="form-control", id="name") }}
23              {% if form.name.errors %}
24                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
25              {% endif %}
26          </div>
27
28          <div class="mb-3">
29              <label for="display_order" class="form-label">Порядок отображения</label>
30              {{ form.display_order(class="form-control", id="display_order", type="number", min="0",
placeholder="") }}
31              {% if form.display_order.errors %}
32                  <div class="text-danger">{{ form.display_order.errors|join(', ') }}</div>
33              {% endif %}
34          </div>
```

Взаимодействие при добавлении. EquipmentGroupType.id_technology_type и связи оборудования используют классификатор для технологической группировки. Модуль рациональной структуры и топливные сервисы обращаются к тем же сущностям в местах, подтвержденных ссылками на исходный код ниже. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.22.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_technology_type	export_technology_type	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.23. Доступность технологии

4.23.1. Страница списка

Адрес: GET, POST /refdata/technology_availability. Обработчик: technology_availability_list().

Содержит классификатор доступности технологий. Список поддерживает просмотр и изменение записей; отдельная форма создает новое значение доступности.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, technology_availability_filter, all_versions, technology_availability_ids[], technology_availability_names[], technology_availability_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: TechnologyAvailability. Поля SQLAlchemy: id, name, created_at, updated_at, database_version_id, equipment_groups, machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/technologies/technology_availability_model.py, строки 13–31.

```
  13  class TechnologyAvailability(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_technology_availabilities'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      # Сохраняем исходную nullable=True, чтобы не трогать существующие данные.
  19      name = db.Column(db.String(80), unique=True, nullable=True, index=True)
  20
  21      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  22      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  23
  24      # Поле для связи с версией БД
  25      database_version_id = db.Column(
  26          db.Integer,
  27          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
  28          nullable=True,
  29          index=True
  30      )
  31
```

Связь модели с другой сущностью. Источник: app/refdata/models/refdata_for_stations/technologies/technology_availability_model.py, строки 32–32.

```
    equipment_groups = db.relationship('EquipmentGroupType',
back_populates='technology_availability')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_availability_routes.py, строки 35–50.

```
@refdata_bp.route("/technology_availability", methods=["GET", "POST"])
@login_required
def technology_availability_list():
    """Маршрут для отображения списка доступности технологий."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница доступности технологий",
        entity_type="technology_availability")

    # Создание формы
    form = TechnologyAvailabilityFilterForm()

    # Получение параметров запроса
    page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_availability_routes.py, строки 124–129.

```
                               sort_dir=sort_dir,
                    ))
                update_technology_availability_all_versions_service(technology_availability_data,
user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_technology_availability_service(technology_availability_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/refdata_for_stations/technologies/technology_availability_services.py, строки 69–86.

```
@no_autoflush
def get_technology_availability_list(
    page,
    per_page,
    technology_availability_filter=None,
    sort_by="id",
    sort_dir="asc"):
    """ Получает список доступности технологий с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = technology_availability_query(
        technology_availability_filter=technology_availability_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/refdata_for_stations/technologies/technology_availability_services.py, строки 154–166.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по доступности технологий",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="technology_availability",
                entity_id=technology_availability_id)
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/refdata_for_stations/technologies/technology_availability/technology_availability.html, строки 160–173.

```
 160                                <!-- Строки таблицы. Столбец "Доступность технологии" -->
 161                                <td>
 162                                    <input type="text" name="technology_availability_names[]"
class="form-control" value="{{ technology_availability.name }}">
 163                                </td>
 164
 165                                <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
 166                                <td class="text-center align-middle">
 167                                    <input type="checkbox" name="technology_availability_delete[]"
value="{{ technology_availability.id }}" class="form-check-input">
 168                                </td>
 169                            </tr>
 170                            {% else %}
 171                            <tr>
 172                                <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
 173                            </tr>
```

Взаимодействие с другими модулями. EquipmentGroupType.id_technology_availability и Machine.technology_availability связывают признак доступности с типами групп и агрегатами. Признак используется профильными модулями при классификации оборудования и технологий; сам справочник не рассчитывает техническую или экономическую доступность.

Подтверждающие ссылки на потребителей модели: app/generation/models/machine/machine_model.py:115 – technology_availability = db.relationship('TechnologyAvailability', back_populates='machines')

4.23.2. Форма добавления

Адрес: GET, POST /refdata/add_technology_availability. Обработчик: add_technology_availability().

Назначение: создание записи справочника «Доступность технологии». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – TechnologyAvailability, модель и ее связи приведены выше в подразделе 4.23.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Технология', validators=[DataRequired(message='Поле «Технология» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта TechnologyAvailability – основной реквизит. Источник: app/refdata/models/refdata_for_stations/technologies/technology_availability_model.py, строки 19–19.

```
  19      name = db.Column(db.String(80), unique=True, nullable=True, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_availability_routes.py, строки 164–176.

```
@refdata_bp.route("/add_technology_availability", methods=["GET", "POST"])
@login_required
def add_technology_availability():
    """ Маршрут для добавления новой доступности технологии. """

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница добавления доступности технологий",
        entity_type="technology_availability")

    # Создание формы
    form = AddTechnologyAvailabilityForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_availability_routes.py, строки 210–220.

```
                            sort_dir=sort_dir,
                        ))
                add_technology_availability_all_versions_service(payload, user)
                flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
            else:
                add_technology_availability_service(payload, user)
                flash("Новая запись успешно добавлена.", "success")

            # Перенаправление на список с сохранением параметров и переходом к новой записи
            total_records = technology_availability_query(
                                technology_availability_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/refdata_for_stations/technologies/technology_availability_services.py, строки 213–226.

```
                if dup:
                    raise ValueError(f"Запись с наименованием «{name}» уже существует.")

                obj = TechnologyAvailability(name=name)
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()

                log_to_db(
                    user,
                    "Создана доступность технологии",
                    f"Наименование: {name}",
                    entity_type="technology_availability",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/refdata_for_stations/technologies/technology_availability/technology_availability_add.html, строки 20–34.

```
        <div class="mb-3">
            <label for="name" class="form-label">Доступность технологии</label>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>

        <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
            <button type="submit" class="btn btn-primary">Добавить</button>
            {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
            <a href="{{ url_for('refdata_bp.technology_availability_list',
                                page=page,
                                per_page=per_page,
                                sort_by=sort_by,
```

Взаимодействие при добавлении. EquipmentGroupType.id_technology_availability и Machine.technology_availability связывают признак доступности с типами групп и агрегатами. Признак

используется профильными модулями при классификации оборудования и технологий; сам справочник не рассчитывает техническую или экономическую доступность. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.23.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_technology_availability	export_technology_availability	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.24. Типы затрат на производство ЭЭ и ТЭ

4.24.1. Страница списка

Адрес: GET, POST /refdata/electricity_production_cost_type. Обработчик: electricity_production_cost_type_list().

Ведет код cost_code и наименование вида затрат на производство электрической и тепловой энергии. Список позволяет изменять классификацию, отдельная форма добавляет запись; предусмотрен экспорт в Excel.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, electricity_production_cost_type_filter, all_versions, electricity_production_cost_type_ids[], cost_codes[], electricity_production_cost_type_names[], electricity_production_cost_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: ElectricityProductionCostType. Поля SQLAlchemy: id, cost_code, name, created_at, updated_at, database_version_id.

Модель данных – фрагмент объявления. Источник: app/refdata/models/electricity_production_cost/electricity_production_cost_type_model.py, строки 14–37.

```
class ElectricityProductionCostType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_electricity_production_cost_types'
    __table_args__ = (
        db.UniqueConstraint(
            'database_version_id',
            'name',
            name='uq_gs_sys_electricity_production_cost_types_ver_name',
        ),
        {"schema": SCHEMA_REFDATA},
    )

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    cost_code = db.Column(db.Integer, nullable=True)
    name = db.Column(db.String(255), nullable=True, index=True)

    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)

    database_version_id = db.Column(
        db.Integer,
        db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
        nullable=True,
        index=True
    )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/electricity_production_cost/electricity_production_cost_type_routes.py, строки 31–46.

```
@refdata_bp.route("/electricity_production_cost_type", methods=["GET", "POST"])
@login_required
def electricity_production_cost_type_list():
    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница затрат на производство и реализацию электрической энергии (мощности)",
        entity_type="electricity_production_cost_type",
    )

    form = ElectricityProductionCostTypeFilterForm()

    page = request.args.get("page", 1, type=int)
    per_page = request.args.get("per_page", 25, type=int)
    sort_by = request.args.get("sort_by", "cost_code")
    sort_dir = request.args.get("sort_dir", "asc")
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/electricity_production_cost/electricity_production_cost_type_routes.py, строки 132–137.

```
                           sort_dir=sort_dir,
                       ))
                update_electricity_production_cost_type_all_versions_service(record_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_electricity_production_cost_type_service(record_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/electricity_production_cost/electricity_production_cost_type_services.py, строки 78–91.

```
@no_autoflush
def get_electricity_production_cost_type_list(
    page,
    per_page,
    electricity_production_cost_type_filter=None,
    sort_by="cost_code",
    sort_dir="asc",
):
    query = electricity_production_cost_type_query(
        electricity_production_cost_type_filter=electricity_production_cost_type_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/electricity_production_cost/electricity_production_cost_type_services.py, строки 221–234.

```
221
222         db.session.flush()
223
224     try:
225         _commit_with_retry()
226
227         if updated_ids:
228             log_to_db(
229                 user,
230                 "Сохранены изменения по затратам на производство ЭЭ (мощности)",
231                 f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
232                 entity_type="electricity_production_cost_type",
233                 entity_id=record_id,
234             )
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/electricity_production_cost/electricity_production_cost_type/electricity_production_cost_type.html, строки 165–178.

```
165
166                             <td>
167                                 <input type="text" name="electricity_production_cost_type_names[]"
class="form-control" value="{{ item.name }}">
168                             </td>
169
170                             <td class="text-center align-middle">
171                                 <input type="checkbox"
name="electricity_production_cost_type_delete[]" value="{{ item.id }}" class="form-check-input">
172                             </td>
173                         </tr>
174                         {% else %}
175                         <tr>
176                             <td colspan="4" class="text-center align-middle">Нет данных для
отображения</td>
177                         </tr>
178                         {% endfor %}
```

Взаимодействие с другими модулями. Классификатор используется моделями технико-экономических показателей затрат и профильными сервисами. Он задает вид показателя; денежные значения, временная привязка и расчетные коэффициенты хранятся отдельно. Прямые ссылки на потребителей приведены в этом разделе.

Подтверждающие ссылки на потребителей модели:
app/fuel/models/fue_equipment_group_electricity_production_cost_model.py:16 – code_zatr хранится в cost_code; наименование подгружается из ElectricityProductionCostType.
app/fuel/services/equipment_groups/equipment_group_electricity_production_cost_services.py:24 – ElectricityProductionCostType,

4.24.2. Форма добавления

Адрес: GET, POST /refdata/add_electricity_production_cost_type. Обработчик: add_electricity_production_cost_type().

Назначение: создание записи справочника «Типы затрат на производство ЭЭ и ТЭ». Вводимые поля формы: name, cost_code. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – ElectricityProductionCostType, модель и ее связи приведены выше в подразделе 4.24.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Наименование', validators=[DataRequired(message='Поле «Наименование» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); cost_code = IntegerField('Код затрат', validators=[Optional(), NumberRange(min=0)]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта ElectricityProductionCostType – основной реквизит. Источник: app/refdata/models/electricity_production_cost/electricity_production_cost_type_model.py, строки 27–27.

```
27      name = db.Column(db.String(255), nullable=True, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/electricity_production_cost/electricity_production_cost_type_routes.py, строки 174–186.

```
174  @refdata_bp.route("/add_electricity_production_cost_type", methods=["GET", "POST"])
175  @login_required
176  def add_electricity_production_cost_type():
177      user = session.get('username', 'Неизвестный пользователь')
178      log_to_db(
179          user,
180          "Открыта страница добавления затрат на производство ЭЭ (мощности)",
181          entity_type="electricity_production_cost_type",
182      )
183
184      form = AddElectricityProductionCostTypeForm()
185
186      page = request.args.get("page", 1, type=int)
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/electricity_production_cost/electricity_production_cost_type_routes.py, строки 227–237.

```
227                          sort_dir=sort_dir,
228                      ))
229                  add_electricity_production_cost_type_all_versions_service(payload, user)
230                  flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
231              else:
232                  add_electricity_production_cost_type_service(payload, user)
233                  flash("Новая запись успешно добавлена.", "success")
234
235              total_records = electricity_production_cost_type_query(
236                  electricity_production_cost_type_filter=electricity_production_cost_type_filter,
237              ).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/electricity_production_cost/electricity_production_cost_type_services.py, строки 331–345.

```
331                              f"Код затрат «{cost_code}» уже задан у записи «{other}»
(id={dup_code.id})."
332                          )
333
334                  obj = ElectricityProductionCostType(name=name, cost_code=cost_code)
335                  set_db_version_on_create(obj)
336                  db.session.add(obj)
337                  db.session.flush()
338
339                  log_to_db(
340                      user,
341                      "Создана запись затрат на производство ЭЭ (мощности)",
342                      f"Наименование: {name}",
343                      entity_type="electricity_production_cost_type",
344                      entity_id=obj.id,
345                  )
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/electricity_production_cost/electricity_production_cost_type/electricity_production_cost_type_add.html, строки 19–33.

```
  19          <div class="mb-3">
  20              <label for="name" class="form-label">Наименование</label>
  21              {{ form.name(class="form-control", id="name") }}
  22              {% if form.name.errors %}
  23                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
  24              {% endif %}
  25          </div>
  26
  27          <div class="mb-3">
  28              <label for="cost_code" class="form-label">Код затрат</label>
  29              {{ form.cost_code(class="form-control", id="cost_code", type="number", min="0",
placeholder="") }}
  30              {% if form.cost_code.errors %}
  31                  <div class="text-danger">{{ form.cost_code.errors|join(', ') }}</div>
  32              {% endif %}
  33          </div>
```

Взаимодействие при добавлении. Классификатор используется моделями технико-экономических показателей затрат и профильными сервисами. Он задает вид показателя; денежные значения, временная привязка и расчетные коэффициенты хранятся отдельно. Прямые ссылки на потребителей приведены в этом разделе. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.24.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_electricity_production_cost_type	export_electricity_production_cost_type	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

4.25. Виды экономической деятельности (ВЭД)

4.25.1. Страница списка

Адрес: GET, POST /refdata/ved. Обработчик: economic_activity_type_list().

Ведет наименование ВЭД, дополнительное наименование name_2 и порядок отображения. Адрес страницы – /refdata/ved, форма создания – /refdata/add_ved. Список поддерживает редактирование, фильтрацию и экспорт.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, economic_activity_type_filter, all_versions, economic_activity_ids[], display_orders[], economic_activity_type_names[], economic_activity_type_names_2[], economic_activity_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: EconomicActivityType. Поля SQLAlchemy: id, display_order, name, name_2, created_at, updated_at, database_version_id.

Модель данных – фрагмент объявления. Источник: app/refdata/models/economic_activity/economic_activity_type_model.py, строки 13–31.

```
  13  class EconomicActivityType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_economic_activity_types'
  15      __table_args__ = (
  16          db.UniqueConstraint(
  17              'database_version_id',
  18              'name',
  19              name='uq_gs_sys_economic_activity_types_ver_name',
  20          ),
  21          {"schema": SCHEMA_REFDATA},
  22      )
  23
  24      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  25
  26      # Порядок отображения
  27      display_order = db.Column(db.Integer, nullable=True)
  28
  29      # Полное наименование (страницы «Экономики»)
  30      name = db.Column(db.String(255), nullable=True, index=True)
  31
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/economic_activity/economic_activity_type_routes.py, строки 35–50.

```
  35  @refdata_bp.route("/ved", methods=["GET", "POST"])
  36  @login_required
  37  def economic_activity_type_list():
  38      """Маршрут для отображения списка видов экономической деятельности."""
  39
  40      user = session.get('username', 'Неизвестный пользователь')
  41      log_to_db(
  42          user,
  43          "Открыта страница видов экономической деятельности",
  44          entity_type="economic_activity_type")
  45
  46      # Создание формы
  47      form = EconomicActivityTypeFilterForm()
  48
  49      # Получение параметров запроса
  50      page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/economic_activity/economic_activity_type_routes.py, строки 166–171.

```
 166                              sort_dir=sort_dir,
 167                          ))
 168                  update_economic_activity_type_all_versions_service(economic_activity_type_data,
user)
 169                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 170              else:
 171                  update_economic_activity_type_service(economic_activity_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/economic_activity/economic_activity_type_services.py, строки 111–128.

```
111  @no_autoflush
112  def get_economic_activity_type_list(
113      page,
114      per_page,
115      economic_activity_type_filter=None,
116      sort_by="display_order",
117      sort_dir="asc"):
118      """ Получает список видов экономической деятельности с пагинацией, фильтрацией и сортировкой.
"""
119
120      # Базовый запрос
121      query = economic_activity_type_query(
122          economic_activity_type_filter=economic_activity_type_filter,
123          sort_by=sort_by,
124          sort_dir=sort_dir,
125      )
126
127      # Пагинация
128      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/economic_activity/economic_activity_type_services.py, строки 273–285.

```
273
274      try:
275          # Сохранение изменений в базе данных
276          # Фиксация транзакции (устойчивый коммит)
277          _commit_with_retry()
278
279          if updated_ids:
280              log_to_db(
281                  user,
282                  "Сохранены изменения по видам экономической деятельности",
283                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
284                  entity_type="economic_activity_type",
285                  entity_id=economic_activity_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/economic_activity/economic_activity_type/economic_activity_type.html, строки 214–227.

```
214                                 <!-- Строки таблицы. Столбец "Вид экономической деятельности" -->
215                                 <td>
216                                     <input type="text" name="economic_activity_type_names[]"
class="form-control" value="{{ economic_activity_type.name }}">
217                                 </td>
218
219                                 <td>
220                                     <input type="text" name="economic_activity_type_names_2[]"
class="form-control" value="{{ economic_activity_type.name_2 or '' }}">
221                                 </td>
222
223                                 <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
224                                 <td class="text-center align-middle">
225                                     <input type="checkbox" name="economic_activity_type_delete[]"
value="{{ economic_activity_type.id }}" class="form-check-input">
226                                 </td>
227                             </tr>
```

Взаимодействие с другими модулями. ВЭД используются экономическими моделями выпуска продукции, накопления основного капитала и электропотребления, а также расчетами электроемкости. Классификатор связывает отраслевые строки разных модулей по идентификатору; изменение названия не выполняет перерасчет экономических показателей автоматически.

Подтверждающие ссылки на потребителей модели:
app/economics/models/federal_district_accum_fixed_capital_parameter_model.py:46 – "EconomicActivityType",
app/electrical_intensity/models/federal_district_electrical_intensity_coefficient_model.py:35 – "EconomicActivityType",
app/economics/models/federal_district_eat_consumption_parameter_model.py:46 – "EconomicActivityType",
app/economics/models/federal_district_product_output_parameter_model.py:46 – "EconomicActivityType",

app/economics/models/russia_federation_accum_fixed_capital_parameter_model.py:35 – "EconomicActivityType",

4.25.2. Форма добавления

Адрес: GET, POST /refdata/add_ved. Обработчик: add_economic_activity_type().

Назначение: создание записи справочника «Виды экономической деятельности (ВЭД)». Вводимые поля формы: name, name_2, display_order. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – EconomicActivityType, модель и ее связи приведены выше в подразделе 4.25.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Полное наименование', validators=[DataRequired(message='Поле «Полное наименование» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); name_2 = StringField('Краткое наименование', validators=[Optional(), Length(max=255, message='Длина не более 255 символов.')]); display_order = IntegerField('Порядок отображения', validators=[Optional(), NumberRange(min=0)]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта EconomicActivityType – основной реквизит. Источник: app/refdata/models/economic_activity/economic_activity_type_model.py, строки 30–30.

```
30      name = db.Column(db.String(255), nullable=True, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/economic_activity/economic_activity_type_routes.py, строки 206–218.

```
206 @refdata_bp.route("/add_ved", methods=["GET", "POST"])
207 @login_required
208 def add_economic_activity_type():
209     """ Маршрут для добавления нового вида экономической деятельности. """
210
211     user = session.get('username', 'Неизвестный пользователь')
212     log_to_db(
213         user,
214         "Открыта страница добавления видов экономической деятельности",
215         entity_type="economic_activity_type")
216
217     # Создание формы
218     form = AddEconomicActivityTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/economic_activity/economic_activity_type_routes.py, строки 260–270.

```
260                                 sort_dir=sort_dir,
261                     ))
262                 add_economic_activity_type_all_versions_service(payload, user)
263                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
264             else:
265                 add_economic_activity_type_service(payload, user)
266                 flash("Новая запись успешно добавлена.", "success")
267
268             # Перенаправление на список с сохранением параметров и переходом к новой записи
269             total_records = economic_activity_type_query(
270
economic_activity_type_filter=economic_activity_type_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/economic_activity/economic_activity_type_services.py, строки 368–381.

```
                            f"Порядок «{display_order}» уже задан у записи «{other}»
(id={dup_order.id})."
                        )

                obj = EconomicActivityType(name=name, name_2=name_2, display_order=display_order)
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()

                log_to_db(
                    user,
                    "Создан вид экономической деятельности",
                    f"Наименование: {name}",
                    entity_type="economic_activity_type",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/economic_activity/economic_activity_type/economic_activity_type_add.html, строки 20–34.

```
        <div class="mb-3">
            <label for="name" class="form-label">Полное наименование</label>
            <div class="form-text">Отображается на страницах «Экономики».</div>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>

        <div class="mb-3">
            <label for="name_2" class="form-label">Краткое наименование</label>
            <div class="form-text">Отображается на страницах «Электроемкости».</div>
            {{ form.name_2(class="form-control", id="name_2") }}
            {% if form.name_2.errors %}
                <div class="text-danger">{{ form.name_2.errors|join(', ') }}</div>
```

Взаимодействие при добавлении. ВЭД используются экономическими моделями выпуска продукции, накопления основного капитала и электропотребления, а также расчетами электроемкости. Классификатор связывает отраслевые строки разных модулей по идентификатору; изменение названия не выполняет перерасчет экономических показателей автоматически. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.25.3. Экспорт, импорт и дополнительные операции

Адрес и метод	Обработчик	Результат
GET /refdata/export_ved	export_economic_activity_type	Файл Excel для скачивания

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

5. Взаимодействие с другими модулями

Обмен внутри АРМ преимущественно осуществляется через общие модели и внешние ключи в одной БД, а не через переходы пользователя между страницами. Маршруты справочников вызывают собственные сервисы; расчетные модули читают соответствующие сущности напрямую либо через общие get-сервисы. REST-адреса обмена упоминаются только как подтвержденные потребители моделей, а не как самостоятельные страницы справочников.

Модуль/подсистема	Получаемые данные и логика связи
Генерирующие мощности	Территории, энергосистемы, компании, типы станций/агрегатов/ТЭС/ПГУ, состояния, технологии, топливо и годы. Station, Machine, PGUMachine и годовые модели хранят ссылки на классификаторы.
Топливо	Единые Fuel/FuelType, типы групп оборудования и затрат, компании, территории и годы. Классификатор типа группы отделен от экземпляров топливных групп и их расчетных параметров.
Экономика	Федеральные округа, ВЭД и годы для отраслевых и территориальных показателей. Используются идентификаторы ВЭД и версионные/годовые ключи.
Электропотребление и электроемкость	Территориально-энергетическая иерархия, ВЭД и временная ось. Изменение привязки субъекта к энергозоне сопровождается синхронизацией агрегатов потребления через сервис.
Потребность в мощности	Уровни ФО, субъект РФ, ОЭС, РЭС, энергорайон, энергоузел, тип энергосистемы, синхронная зона и энергозона. Модели параметров связывают показатели с соответствующим уровнем справочника.
Энергетические балансы	Энергосистемы, территории, страны, типы станций и годы. Используются для группировки генерации, межсистемных передач и импортно-экспортных значений.
Экология	Территории, энергосистемы, оборудование, топливо и годы для сопоставления исходных данных и построения расчетных представлений. Расчет выбросов не реализуется в

Модуль/подсистема	Получаемые данные и логика связи
	страницах справочников.
Рациональная структура	Типы технологий и групп оборудования для территориально-технологических моделей и сопоставления оборудования.
Версии БД	Выбор контекста database_version_id; межверсионная идентичность ref_uuid и разрешение ссылок на записи соответствующей версии.
Авторизация, журнал и кэши	flask_login/current_user и декораторы доступа; log_to_db для событий; варианты выбора и справочные get-сервисы для зависимых форм.

5.1. Пример сквозного взаимодействия: субъект РФ → энергозона → потребление

Пользователь изменяет энергозону субъекта в списке. POST передает energy_zone_id в сервис субъектов РФ. После изменения принадлежности сервис собирает затронутые зоны, обновляет состояние ORM-связи regional_districts и вызывает sync_energy_zone_consumption_aggregates. В результате данные территориальной привязки используются модулем электропотребления без повторного ручного ввода классификатора.

Вызов синхронизации модуля электропотребления. Источник: app/refdata/services/territories/regional_district_services.py, строки 48–69.

```
48  def _sync_energy_zone_aggregates_for_ids(
49      energy_zone_ids,
50      *,
51      database_version_id=None,
52  ):
53      """Пересчет агрегатов потребления ЭЗ после смены привязки субъектов."""
54      ids = {int(ez_id) for ez_id in energy_zone_ids if ez_id is not None}
55      if not ids:
56          return
57
58      from app.energy_consumption.services.energy_consumption_parameter_services import (
59          sync_energy_zone_consumption_aggregates,
60      )
61
62      for ez_id in sorted(ids):
63          ez = db.session.get(EnergyZone, ez_id)
64          if ez is not None:
65              db.session.expire(ez, ["regional_districts"])
66          sync_energy_zone_consumption_aggregates(
67              database_version_id=database_version_id,
68              id_energy_zone=ez_id,
69          )
```

5.2. Пример сквозного взаимодействия: тип группы → технология → оборудование

Тип группы оборудования связан с TechnologyType и TechnologyAvailability. Модель Machine содержит ссылку на тип группы, а топливные модели используют тот же классификатор в собственных связях. Изменение классификационной записи влияет на последующее чтение зависимых объектов; перенос связей при объединении дубликатов реализуется специализированными сервисами. Это не равнозначно изменению индивидуальных расчетных параметров всех групп.

Объединение дублирующихся типов групп – фрагмент. Источник: app/refdata/services/refdata_for_stations/technologies/equipment_group_services.py, строки 84–109.

```
  84  def _merge_equipment_group_type_into_canonical(*, from_id: int, to_id: int, user) -> None:
  85      """
  86      Переносит все ссылки с типа группы оборудования from_id на to_id и удаляет строку from_id.
  87      Используется при сохранении, когда редактируемая запись дублирует пару (name,
id_technology_type).
  88      """
  89      if from_id == to_id:
  90          return
  91
  92      from_obj = db.session.get(EquipmentGroupType, from_id)
  93      to_obj = db.session.get(EquipmentGroupType, to_id)
  94      if not from_obj or not to_obj:
  95          raise ValueError(f"Слияние типов групп: запись from_id={from_id} или to_id={to_id} не
найдена.")
  96
  97      Machine.query.filter(Machine.id_equipment_group == from_id).update(
  98          {Machine.id_equipment_group: to_id},
  99          synchronize_session=False,
 100      )
 101
 102      # По одной строке + flush: при нескольких записях с (station, from_id, NULL) в UNIQUE
 103      # подзапрос «уже есть to_id» не видит только что обновленные строки до flush.
 104      while True:
 105          old_link = (
 106              EquipmentGroupSetStation.query.filter_by(
 107                  equipment_group_type_id=from_id
 108              ).first()
 109          )
```

5.3. Граница модуля: управление версиями БД

Файл app/refdata/routes/database_versions_routes.py физически расположен рядом со справочниками, но регистрирует страницы на station_bp, в пространстве адресов /generation/. Ссылка «Версии» относится к общей подсистеме версий АРМ. Ее страницы не включены в перечень HTML-страниц /refdata/: в настоящем документе описана именно логика взаимодействия с ними.

6. Полнота материалов и перечень страниц

Описаны 51 самостоятельная HTML-страница: главная страница плюс 25 списков и 25 форм добавления. Сортировка, поиск и пагинация не создают новых страниц по назначению: это состояния соответствующего списка. Модальное окно иерархии топлива описано отдельно внутри справочника. Экспорт, импорт и JSON API перечислены как служебные операции.

№ раздела	Справочник	Список	Добавление
4.1	Годы	/refdata/years	/refdata/add_year
4.2	Федеральные округа	/refdata/federal_district	/refdata/add_federal_district
4.3	Субъекты РФ	/refdata/regional_district	/refdata/add_regional_district
4.4	Зарубежные страны, имеющие общие границы с РФ	/refdata/foreign_border_country	/refdata/add_foreign_border_country
4.5	Объединенные энергосистемы	/refdata/union_energy_system	/refdata/add_union_energy_system
4.6	Региональные энергосистемы	/refdata/regional_energy_system	/refdata/add_regional_energy_system
4.7	Энергорайоны	/refdata/energy_units	/refdata/add_energy_unit
4.8	Энергоузлы	/refdata/energy_areas	/refdata/add_energy_area
4.9	Типы энергосистем	/refdata/energy_system_type	/refdata/add_energy_system_type
4.10	Синхронные зоны	/refdata/synchronous_area	/refdata/add_synchronous_area
4.11	Энергозоны	/refdata/energy_zone	/refdata/add_energy_zone
4.12	Генерирующие компании	/refdata/gen_company	/refdata/add_gen_company
4.13	Типы топлива	/refdata/fuel_type	/refdata/add_fuel_type
4.14	Виды топлива	/refdata/fuel	/refdata/add_fuel
4.15	Типы электроста	/refdata/station_type	/refdata/add_station_type

№ раздела	Справочник	Список	Добавление
	нций		
4.16	Типы состояния оборудования	/refdata/condition_type	/refdata/add_condition_type
4.17	Типы агрегатов	/refdata/machine_type	/refdata/add_machine_type
4.18	Типы ТЭС	/refdata/tes_type	/refdata/add_tes_type
4.19	Типы агрегатов ТЭС	/refdata/tes_machine_type	/refdata/add_tes_machine_type
4.20	Типы агрегатов ПГУ	/refdata/pgu_tes_machine_type	/refdata/add_pgu_tes_machine_type
4.21	Типы групп оборудования	/refdata/equipment_group	/refdata/add_equipment_group
4.22	Типы технологий	/refdata/technology_type	/refdata/add_technology_type
4.23	Доступность технологии	/refdata/technology_availability	/refdata/add_technology_availability
4.24	Типы затрат на производство ЭЭ и ТЭ	/refdata/electricity_production_cost_type	/refdata/add_electricity_production_cost_type
4.25	Виды экономической деятельности (ВЭД)	/refdata/ved	/refdata/add_ved

Вне перечня самостоятельных страниц: модели организаций business_unit, department, economic_region, признаки годов YearFeature, параметры YearService, коэффициенты пересчета цен и другие вспомогательные сущности, не имеющие собственной страницы в навигации /refdata. Их наличие в каталоге моделей не является основанием приписывать модулю дополнительные веб-страницы.

6.1. Сведения об источниках и проверке

Шаблон заявки: «Сведения для регистрации программы.doc», предоставлен пользователем. Структура 12 пунктов сохранена в разделе 1; отдельные листинги и экранные формы размещены в последующих разделах, как допускает шаблон.

Снимки экрана получены с действующего сайта 29.10.2026. Для печатного документа общий навигационный заголовок с именем учетной записи исключен кадрированием; длинные таблицы представлены верхним фрагментом.

Изображения не являются макетами и не заменяют описание полей, скрытых горизонтальной или вертикальной прокруткой.

Функциональные возможности и листинги сверены со структурой маршрутов, моделей, форм, сервисов и шаблонов текущей рабочей копии. Просмотр сайта выполнялся без отправки форм сохранения, добавления, удаления или импорта. Исполнение всех изменяющих операций на рабочей БД не проверялось и в рамках подготовки регистрационного описания не требовалось.

Документ является описанием одного модуля и не заменяет окончательную заявку на регистрацию всей программы. До подачи заявитель подтверждает официальное наименование программы, авторский состав и вклад, контакты, дату завершения, объем заявляемой поставки и сведения об операционной системе.